

Basic Model

Contents

BasicModel	5
Basic Model of Cognition	5
Documentation	5
Overview	6
Files	6
Quick Start	6
XML Configuration	7
Output	7
Installation and Usage	7
Prerequisites	7
Setup	8
Makefile Targets	8
Training	8
Models	8
Testing and Documentation	8
Utilities	8
Makefile Variables	9
train.py Options	9
General	9
Remote Execution (SSH)	9
Environment Variables	9
Remote Training (ArborMini)	10
Workflow	10
SSH Key Setup	10
Quick Start	10
Architecture	10
Overview	11
Spaces	11
Reconstruction Symbols	12
Single Optimizer with Overlapping Weight Spaces	13
Training Loop	13
Modes of operation	13
Pipeline as a unit, two-tier reset	14
Two-File Architecture	14
Language System	15
Short-Term Memory on ConceptualSpace	15
Per-word operational flow (SERIAL mode)	16
Sigma and Pi Layers	16
Dimensionality Constraints	17
Invertible Linear Layer (LDU)	17

Ergodic Exploration	18
Sentence-level AR (InterSentenceLayer)	18
Sentence representation	18
ARMA(p, q) predictor	18
Wiring into the training loop	18
Inference (chat-loop seeding)	19
Configuration	19
Basic Model	20
Inputs and Outputs	21
Perceptual Space	21
Conceptual Space	21
Symbolic Space	21
Reasoning	22
Symmetric Perception: the Single Layer Linear Case	22
Symmetric Perception: the Single Layer Nonlinear Case	22
Distance Metrics	23
Exploration/Exploitation as a function of temperature	23
Transfer Functions	23
Spaces	24
Overview	24
Base Class: Space	25
Reset cascade — hard vs. soft	25
Sigma / Pi ownership (2026-05-27 substrate refactor)	26
Lift / Lower in the new ownership	26
Butterfly mode on GrammarLayer (Stage 5)	27
Normalization and Ranges	27
Codebook Similarity Metric	27
Euclidean (Perceptual / Symbolic)	28
Dot product (Conceptual)	28
Configuring the metric	28
Codebook Uniqueness Contract	28
Lexicon (Projective Unit Ball)	29
Distance and lookup	29
SBOW training: pole (attractor) and antipole (repulsion target)	30
InputSpace	30
Document streaming and valid_mask	31
AR cursor unfold retirement (2026-05-13)	31
Within-sentence AR retirement (2026-05-14)	31
PerceptualSpace	31
ConceptualSpace	32
ShortTermMemory	33
Lift / Lower as binary GrammarLayer subclasses	34
SymbolicSpace	35
Monotonicity of the lift / lower chain	36
SyntacticSpace — retired	36
OutputSpace	36
Short-Term Memory	37
Overview	37
1. STM data model	37
2. Serial sequencing	38
3. Parallel sequencing	38

4. Attentional filtering	39
5. Routing parser: SS-analysis vs CS-execution	40
6. IntraSentenceLayer	40
7. Per-word router firing	41
8. Masked-word reconstruction via priming	41
9. Relative vs absolute end-states	42
10. LTM as the chain of STM end-states	43
11. Inter-sentence prediction	43
12. nanochat comparison framing	44
See also	44
Language	44
Current Parser Surface	45
Retired XML Knobs	45
Grammar	46
Knowledge Artifacts	46
STM Shift/Reduce	46
Syntax And SVO	47
Mereology	47
Parthood as Clipped Cosine Projection	47
The Full Suite	48
Region Partition	48
Mereological Fusion	49
ImpenetrableLayer: Five-Relations Regularizer	49
Penalty	49
Trust	49
Diagnostics	50
Configuration	50
Why This Design	50
Logic	50
Overview	50
1. Subsymbolic Layer	50
Operators	51
2. Symbolization	51
3. Symbolic Layer (Scalars in [-1,1])	51
Parthood as Projection	51
4. Key Insight	52
5. Rationality	52
Truth Statements	52
TruthLayer (WordSpace.truth_layer)	53
Truth Field	53
Consonance and Dissonance	53
Propositional Structure	53
Truth Accumulation Pipeline	53
6. Consistency and Verification	53
Internal Consistency (within a TruthSet)	53
Verification (incoming statement against stored truth)	54
Logical Entailment (symbolic closure)	54
7. Radial Operators for Hypersphere Ternary Logic	55
Luminosity	56
Fusion	56
Supporting measures	57

Semantic Summary	57
8. Cross-references	57
Reasoning System	57
Partitioned Symbolic Space	58
Reasoning Methods	58
isConsistent() -> dict	58
ground(activation, threshold=0.6) -> dict	58
isTrue(activation) -> float	58
extrapolate(grammar, seed_indices, max_new, attenuation) -> dict	58
TruthLoss	58
Bidirectional Reasoning Loop	59
Grammar Learning	59
Parser And Conceptual Order	59
Configuration	59
Contemplative Awareness Methods	60
Testing	60
Training	60
Overview	61
Phase 1: Embedding Pretraining (make train / embed.py train)	61
Pipeline	61
SBOW (Sentence Bag of Words)	61
CBOW (Continuous Bag of Words)	62
Two-Pass Architecture	62
Configuration (Makefile variables)	62
Memory Considerations	62
Phase 2: Network Training (BasicModel.py)	63
Within-sentence training objective (IR-only)	63
<trainEmbedding>: Embedding Update Mode	63
Gradient Flow Through Codebook	63
JOINT: Single Combined Loss	64
Why MSE over Embeddings (not Cross-Entropy)	64
Training Loop	64
SBOW vs CBOW	65
Three-File Architecture	65
Embedding Space Geometry	65
Profiling	66
Ergodic Exploration via Adaptive Bias-Variance Control	66
1. The Ergodic Weights	66
2. Why Standard Adam Fails on α	66
3. The Gradient Energy Sensor	67
3.1 Modified Second-Moment Estimator	67
3.2 Bias Correction	67
3.3 What $\hat{v}_t$ Measures	67
3.4 The Alpha Update Rule	67
3.5 Layer-Local Certainty (Per-Neuron Sigma)	68
4. Comparison: Adam vs. Gradient Energy Sensor	68
5. Derivation: Why Drop m_t	68
6. Dropout via Bias	68
7. The global_temp Sensitivity Knob	69
8. Initialization and Bootstrap	69
9. Layer Architecture	69

ErgodicLayer Base Class	69
InvertibleLinearLayer — factor-level noise injection	69
10. Training Loop Integration	70
11. Summary of the Full Algorithm	70
Machine Minds	70
Abstract	71
Background	71
Part I: What Machine Minds Are — Weight Ergodicity	71
Weights as Ergodic Samples	71
Temperature and Annealing	71
Advantages of Ergodic Weights	72
Part II: What Machine Minds Feel — Network Invertibility	72
Bidirectional Perception	72
Symbols, Worldlines, and Karmic Inertia	72
Implementation	72
Part III: What Machine Minds Know — Output Certainty	72
Certainty vs. Probability	72
Certainty-Weighted Loss	73
Knowing What You Do Not Know	73
Methods	73
Results	74
Discussion	74
Conclusion	75
References	75
XML Configuration Parameters	75
<architecture>	75
<architecture><data>	76
<architecture><training>	77
<architecture><server>	77
BasicModel / MentalModel Quick Reference	78
Space sections	78
<InputSpace>	78
<PerceptualSpace>	79
<ConceptualSpace>	79
<SymbolicSpace>	79
<OutputSpace>	80
<WordSpace>	80
InvertibleLinearLayer (programmatic)	81
Example: XOR Configuration	81
Example: Embedding / Chatbot Configuration	82

BasicModel

Basic Model of Cognition

The basic model of cognition relies on conceptual hyperplanes and perceptual prototypes to synthesize and analyze the input space. It uses a high-dimensional embedding to characterize mental space and integrates symbolic computation. Its three major operations are intersection (which forms percepts from concepts), union (a bidirectional mapping between concepts and percepts), and equality (where symbols are elements that map across perceptual and conceptual domains).

Documentation

Document	Description
Architecture	Pipeline design, layer types, invertible LDU factorisation
Spaces	All six spaces: Input $\rightarrow$ Perceptual $\rightarrow$ Conceptual $\rightarrow$ Symbolic $\rightarrow$ Syntactic $\rightarrow$ Output
Ergodic	Gradient energy sensor, adaptive exploration, factor-level noise injection
Training	Two-phase training, SBOW embeddings, masked prediction modes
Params	Complete XML configuration reference
BasicModel	Cognitive science foundations
Language	Grammar, word encoding, symbolic and syntactic spaces
Logic	Subsymbolic and symbolic logic operations
Reasoning	Truth methods, bidirectional reasoning, contemplative awareness stubs
MachineMinds	What machine minds are, feel, and know
Installation	Setup, Makefile targets, environment variables

Overview

BasicModel is a parameterized neural architecture that answers the question “what is the truest thing we can say of this experience?”.

Model configurations are specified in XML. See doc/Architecture.md for the full mathematical treatment.

Files

File	Description
bin/BasicModel.py	Main entry point: model factory, training loop, comparison plots, HTML report
bin/Model.py	Layer library: SigmaLayer, PiLayer, ErgodicLayer, LinearLayer, TruthLayer
bin/Space.py	Space classes: InputSpace, PerceptualSpace, ConceptualSpace, SymbolicSpace, OutputSpace, GrammarSpace
bin/embed.py	Word vector training: CBOW/SBOW with negative sampling, WordVectors (gensim-compatible .k) format
bin/SigmaPi.py	Standalone demo of the SigmaPi network solving XOR
data/	XML model configurations
doc/Architecture.md	Algorithm details: Sigma/Pi layers, ergodic exploration, gradient energy sensor
doc/Params.md	Full XML parameter reference
doc/Training.md	Embedding pretraining, CBOW/SBOW, masked prediction, <code><trainEmbedding></code> modes
doc/Installation.md	Setup, Makefile targets, train.py options, remote training
test/	Unit tests

Quick Start

```
# Set up virtual environment
make install

# Run a single model
make simple      # data/simple.xml
make ergodic     # data/ergodic.xml

# Compare two models side-by-side
make compare     # defaults: data/simple.xml vs data/ergodic-only.xml
make compare XML1=data/simple.xml XML2=data/ergodic.xml

# Run tests
make test
```

```
# Generate PDF documentation
make doc_pdf
```

XML Configuration

Models are configured via XML files in `data/`. Training and data parameters live in nested sub-elements:

```
<model>
  <architecture>
    <modelType>simple</modelType>    <!-- simple / lm / embedding -->
    <ergodic>>false</ergodic>
    <certainty>>false</certainty>
    <reconstruct>NONE</reconstruct>  <!-- NONE / symbols / output / both -->
    <maskedPrediction>NONE</maskedPrediction>

    <data>
      <dataset>xor</dataset>         <!-- xor / mnist / text / ... -->
    </data>

    <training>
      <numTrials>1</numTrials>
      <numEpochs>20</numEpochs>
      <batchSize>10</batchSize>
      <learningRate>0.001</learningRate>
      <weightsPath>output/BasicModel.ckpt</weightsPath>
      <autoload>true</autoload>
      <autosave>>false</autosave>
    </training>
  </architecture>

  <InputSpace> ... </InputSpace>
  <PerceptualSpace> ... </PerceptualSpace>
  <ConceptualSpace> ... </ConceptualSpace>
  <SymbolicSpace> ... </SymbolicSpace>
  <OutputSpace> ... </OutputSpace>
</model>
```

See `doc/Params.md` for the full parameter reference.

Output

Each run produces an HTML report (timestamped in `output/`) containing:

- **Error per Epoch** – training and test loss curves
- **Accuracy per Digit** – per-class accuracy breakdown

In compare mode, additional overlay plots show combined loss and accuracy across models with color-coded legends.

Installation and Usage

Prerequisites

- **Python 3.12** with a virtual environment at `.venv/`
- **PyTorch** with MPS (Apple Silicon) or CUDA
- **pandoc** (optional, for PDF generation via `make doc_pdf`)

Setup

`make install` *# creates .venv with dependencies*

All Makefile targets invoke Python through `.venv/bin/python`, so no manual activation is required.

Makefile Targets

Training

Target	Description
<code>make train</code>	Full training (Phase 1 embeddings + Phase 2 model), logged to <code>output/logs/</code>
<code>make train_micro</code>	Small run: 500 docs, 1 random shard, logged
<code>make train_remote</code>	Full training on <code>arbormini.local</code> via SSH
<code>make train_micro_remote</code>	Micro training on <code>arbormini.local</code> via SSH

Models

Target	Description
<code>make run</code>	Run <code>BasicModel.py</code> with the config in XML1
<code>make xor</code>	<code>data/XOR_exact.xml</code>
<code>make simple</code>	<code>data/simple.xml</code>
<code>make ergodic</code>	<code>data/ergodic.xml</code>
<code>make tomatoes</code>	<code>data/tomatoes.xml</code>
<code>make mnist</code>	<code>data/mnist.xml</code>
<code>make compare</code>	Side-by-side compare using XML1 and XML2
<code>make SigmaPi / SymPercept / SPNN</code>	Run respective scripts

Testing and Documentation

Target	Description
<code>make test</code>	Unit tests (forces <code>BASICMODEL_DEVICE=cpu</code>)
<code>make bench</code>	Training benchmarks (baseline)
<code>make doc_pdf</code>	Generate <code>BasicModel.pdf</code> via pandoc

PDF chapters in order: `README.md`, `doc/Architecture.md`, `doc/BasicModel.md`, `doc/Spaces.md`, `doc/Language.md`, `doc/Logic.md`, `doc/Ergodic.md`, `doc/Training.md`, `doc/MachineMinds.md`, `doc/Params.md`, `doc/Installation.md`.

Utilities

Target	Description
<code>make clean</code>	Remove generated files (<code>BasicModel.pdf</code>)
<code>make all</code>	Default; alias for <code>make xor</code>

Makefile Variables

Override on the command line, e.g. `make run XML1=data/ergodic.xml`.

Variable	Default	Description
MODEL	data/BasicModel.xml	XML config for training
XML1	data/simple.xml	Primary config for <code>make run</code> / <code>compare</code>
XML2	data/ergodic-only.xml	Secondary config for <code>make compare</code>
TRAIN_HOST	<i>(empty)</i>	Remote training host
TRAIN_USER	arogers	SSH user
TRAIN_KEY	~/.ssh/id_ed25519_arbormini	SSH private key
TRAIN_DIR	~/WikiOracle/basicmodel	Working directory on remote

train.py Options

`train.py` orchestrates Phase 1 (embeddings) followed by Phase 2 (model training).

General

Flag	Default	Description
<code>--model, -m</code>	data/BasicModel.xml	XML config
<code>--max-docs</code>	<i>(from XML)</i>	Override <code>maxDocs</code>
<code>--num-shards</code>	<i>(from XML)</i>	Override <code>numShards</code>
<code>--random-shards</code>	off	Pick random shard indices
<code>--force-embeddings</code>	off	Retrain embeddings even if file exists
<code>--log [FILE]</code>	off	Capture stdout/stderr to <code>.log</code> in <code>output/logs/</code> . Omit filename for times
<code>--profile</code>	off	Run Phase 2 under <code>cProfile</code> ; writes <code>.prof</code> to <code>output/profiles/</code>

Remote Execution (SSH)

Flag	Default	Description
<code>--host</code>	<i>(none)</i>	Remote host (e.g. <code>arbormini.local</code>); enables remote run
<code>--user</code>	arogers	SSH user
<code>--key-file</code>	~/.ssh/id_ed25519_arbormini	SSH private key
<code>--remote-dir</code>	~/WikiOracle/basicmodel	Working directory on remote

Environment Variables

Variable	Description
BASIC_MAX_DOCS	Override <code>maxDocs</code>
BASIC_NUM_SHARDS	Override <code>numShards</code>
BASICMODEL_DEVICE	Force device, e.g. <code>cpu</code> . Used by <code>make test</code>
PYTORCH_MPS_HIGH_WATERMARK_RATIO	MPS memory limit; set to 0.0 by <code>train.py</code>
PYTHONUNBUFFERED	Set to 1 by <code>train.py</code> for real-time log streaming

Remote Training (ArborMini)

`train_remote` and `train_micro_remote` set `TRAIN_HOST=arbormini.local` and delegate to `train.py --host`.

Workflow

1. **Rsync** the project tree to `arbormini.local`, excluding `.venv/`, `__pycache__/`, `.DS_Store`, `output/`.
2. **SSH** and run `train.py` remotely with the same flags (minus `--host`):

```
ssh -i ~/.ssh/id_ed25519_arbormini -t arogers@arbormini.local \  
"cd ~/WikiOracle/basicmodel && PYTHONUNBUFFERED=1 PYTHONPATH=bin .venv/bin/python bin/train.py ."
```

3. **Output stays remote** under `~/WikiOracle/basicmodel/output/`.

SSH Key Setup

```
ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519_arbormini  
ssh-copy-id -i ~/.ssh/id_ed25519_arbormini arogers@arbormini.local
```

Quick Start

```
# Sanity check: 500 docs, 1 shard, on ArborMini  
make train_micro_remote
```

```
# Full run on ArborMini  
make train_remote
```

Architecture

Status (2026-05-29 update): further architectural pivots landed on top of the 2026-05-27 substrate refactor:

- **PerceptStore** → **RadixLayer**. The radix-trie input encoder is now a first-class `Layer` subclass in `bin/Layers.py`; `PerceptualSpace.reverse` invokes `RadixLayer.reverse` for the structural decode.
- **MetaLayer** → **SymbolizeLayer**. The binary GrammarLayer that promotes a freshly-seen percept to a symbolic prototype is now `SymbolizeLayer`; no semantic change.
- **Auto-META moves PS** → **CS**. The cross-codebook bind (META entry: PS chunk-id ↔ SS prototype-id) fires from `ConceptualSpace._maybe_autobind_meta` at stage 0; PS no longer holds a back-ref to SS.
- **Clean-stack STM**. `ConceptualSpace.forward` bypasses `sigma_in` / `sigma_cs` on forward — `folded = primary` at stage 0, `folded = sym` at `k > 0`. The Stage-10 additive composition is retired; per-stage tier attribution is trivially invertible.
- **basis= kwarg for grammar reverses**. `UnionLayer.reverse(parent, basis=None)` / `IntersectionLayer.reverse(parent, basis=None)` accept a `Codebook` / `Basis` object (typically `SymbolicSpace.subspace.what`) instead of a raw `W` tensor; `bin/Language.py::unreduce()` dispatches accordingly.
- **LSE soft-max kernels**. `Ops._disjunction_kernel` / `Ops._conjunction_kernel` default to `LogSumExp` smooth variants when `monotonic=False`; the hard branch is retained for monotonic-mode and exact idempotency tests.
- **LBG-style SS codebook splitting**. Gray (1990) EMA + per-row variance tracking; rows whose running variance exceeds a threshold split along the top-variance eigendirection.

See `doc/plans/2026-05-29-clean-stack-stm-basis-arg-radixlayer.md` for the consolidated rationale.

Status (2026-05-27): the **substrate refactor** has landed end-to-end (doc/plans/2026-05-26-two-loop-pi-sigma-substr). PS is a single-arg input processor (pi + sigma). CS is a STM container + grammatical CPU (no atomic forward fold; sigma_percept retired). SS owns the unified word lexicon codebook with paired (orth, semantic) rows. The CKY **Chart** and STM shift-reduce parsers retire entirely; **LanguageLayer** (signal router) is the canonical parser. **LiftLayer** / **LowerLayer** are binary **GrammarLayer** subclasses with internal Sigma / Pi (no longer borrowing substrate folds). **GrammarLayer** gains an optional butterfly cascade mode for cross-position mixing — closes the XOR convergence target. Two operating modes: **SERIAL/GRAMMATICAL** (per-word PS with grammar dispatch over STM) and **PARALLEL** (T = <conceptualOrder> iterations of PS over CS). The <parserBackend>, <routerKind>, <chartTau>, <chartTopK>, <chartNoiseEps> XML knobs are retired; <conceptualMode> is the new dispatch knob.

Overview

BasicModel is a bidirectional neural architecture organized as a pipeline of five **spaces** plus a grammar host (**WordSpace**), each implementing a distinct representational transformation:

Forward: `InputSpace -> PerceptualSpace -> ConceptualSpace -> SymbolicSpace -> OutputSpace`
Reverse: `OutputSpace -> SymbolicSpace -> ConceptualSpace -> PerceptualSpace -> InputSpace`

The pre-2026-05-27 “two feedback loops” ($S \rightarrow C$ symbolic loopback per stage, $C \rightarrow P$ subsymbolic loopback cross-forward) collapse under the substrate refactor:

- **Subsymbolic loop dissolves.** PS is a single-direction input processor. No recurrent $C \rightarrow P$ feedback at the substrate level. In PARALLEL mode, iteration happens by passing CS to the same `PS.forward(x)` for T refinement passes (the <conceptualOrder> knob).
- **Symbolic loop generalizes** to pairwise grammar ops over STM, dispatched by the signal router (**LanguageLayer**). **Lift** and **Lower** join the same GrammarLayer dispatch surface as **Intersection**, **Union**, etc.

The forward pass transforms raw input into predictions; the reverse pass reconstructs the original input from the symbolic representation. Both directions are trained simultaneously with a single optimizer minimizing a combined loss:

`totalLoss = (1 - reconRatio) * outputLoss + reconRatio * reconstructionLoss`

The legacy **SubsymbolicSpace** and **SyntacticSpace** classes have been retired. The subsymbolic role is filled by **PerceptualSpace**; syntax / grammar dispatch lives on **WordSubSpace.languageLayer** (the signal router, which subsumes the retired **Chart**). The **MereologicalTree** sidecar that backed **part** / **equals** / **query** is also retired — those operations are pure-geometric clipped-cosine projections over **SymbolicSpace** codebook activations.

Spaces

Space	Role	Owns
InputSpace	Lifts raw data into working dimensionality; surface tokenization	LiftingLayer; lexer wiring
PerceptualSpace	Single-arg input processor: pi + sigma + MPHF lookup	one <code>self.pi</code> (PiLayer),
ConceptualSpace	STM container + main grammatical CPU	STM (ShortTermMemory)
SymbolicSpace	Unified word lexicon codebook owner; dispatch site for codebook-write ops	unified codebook with p
OutputSpace	Final prediction	LinearLayer

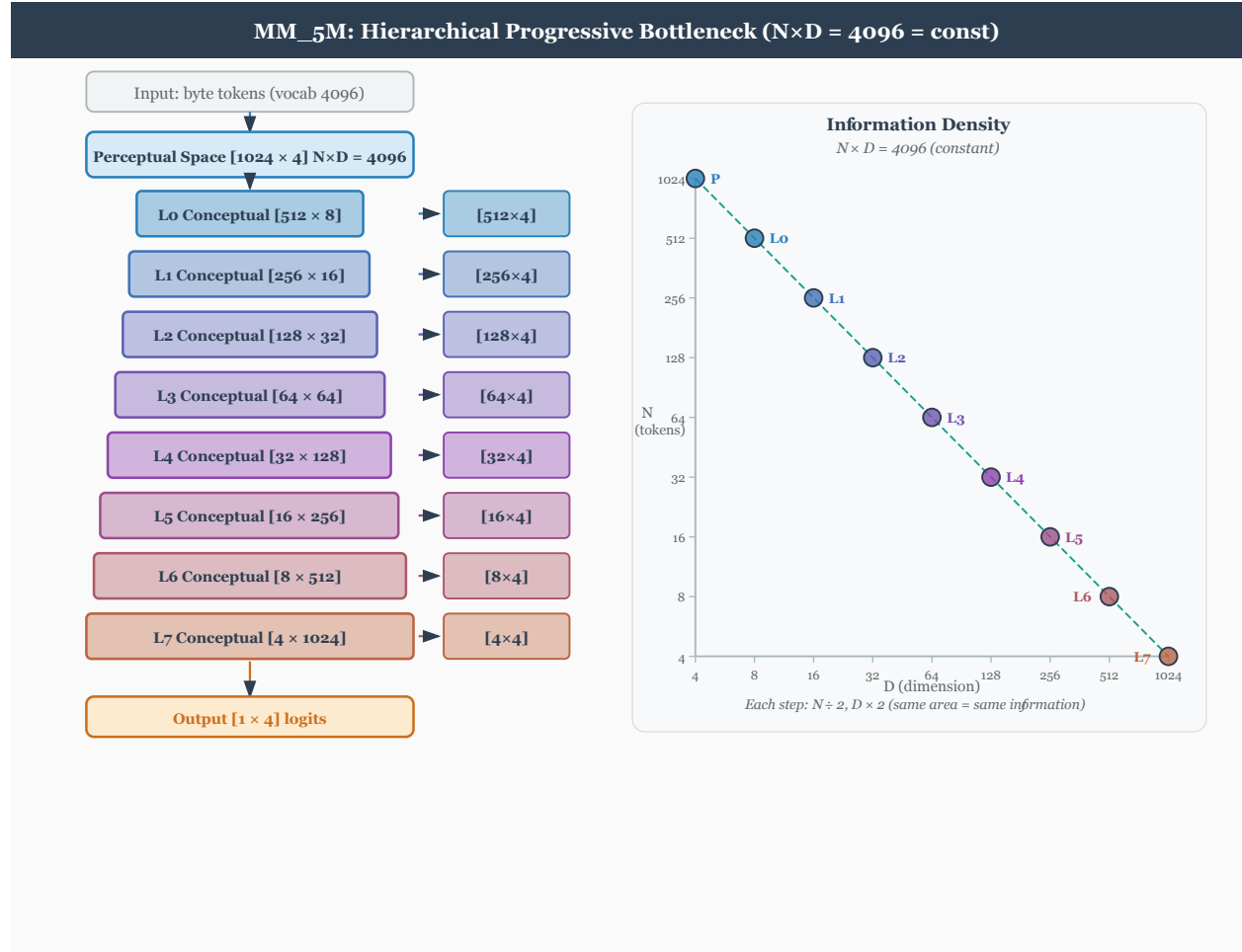
The cross-space fold contract has changed:

PS.forward(x): `return self.pi(x.materialize()) + self.sigma(x.materialize())`
CS.forward(new_idea): `STM[0..6] = STM[1..7]; STM[7] = new_idea (mode-dispatched)`
SS: no atomic forward operator; hosts `insert_paired_word` + write-required grammar ops

The legacy composition $C = \text{sigma_percept}(\text{pi_input}(IS) + \text{pi_concept}(C_{\text{prev}}))$ is **retired**. Per-stage feedback is absent at the substrate; grammar dispatch over STM provides the recurrent character via the signal router.

See Spaces.md Section “Sigma / Pi ownership” for the cognitive rationale and the migration trail. See Logic.md Section 8 for the algebraic constraints on sigma/pi.

Dimensions (nDim) are read from TheObjectEncoding. Codebook sizes (nVectors) are likewise on TheObjectEncoding; the factory validates $n\text{Vectors} \geq n\text{Active}$.



Layer selection by `<reconstruct>` and `invertible`:

1. **reconstruct=NONE**: Non-invertible layers (PiLayer, SigmaLayer) forward-only. No reverse pipeline.
2. **reconstruct=<any> + invertible**: Single invertible layer (PiLayer(invertible=True), etc.) serves both directions, sharing weights.
3. **reconstruct=<any> + not invertible**: Two layers with separate weights — `forward()` on one, `reverse()` on the other. Avoids the expressivity limitation where a non-invertible layer can't represent the inverse of another. Reverse uses matrix `pinv` (may be numerically unstable from SVD convergence). `<invertible>true</invertible>` avoids this via shared-weight inversion.

Reconstruction Symbols

The symbolic bottleneck can lose information needed for reconstruction. XOR maps 2 inputs to 1 output, but $\text{XOR}(0,0)=0$ and $\text{XOR}(1,1)=0$ are distinct inputs producing the same output. A single output symbol cannot reconstruct which input was presented.

nSymbols is split:

- **nOutputSymbols** = `OutputSpace.nActive` — fed to `OutputSpace` for prediction
- **nReconSymbols** = `nSymbols - nOutputSymbols` — carried in `end_state` for reconstruction

A skip connection through the symbolic bottleneck. Reconstruction symbols receive gradient only from reconstruction loss.

Single Optimizer with Overlapping Weight Spaces

The forward and reverse passes share a **single Adam optimizer** that minimizes the combined loss. Forward and reverse weight spaces **partially overlap** — neither disjoint (allowing independent optimizers) nor identical (creating destructive interference). Some layers share weights between directions (shared embeddings, the symbolic bottleneck); others are direction-specific (`pi1/pi2`, `sigma1/sigma2`, `linear1/linear2`).

- **Shared weights** receive gradient from both losses, learning representations useful in both directions.
- **Direction-specific weights** specialize without interference.
- **No ping-pong**: separate optimizers on overlapping parameters would pull weights in alternating, conflicting directions each step.

When `invertible=true`, overlap is total: one invertible layer serves both directions and receives the full combined gradient.

Reference: A.M. Rogers, T.T. Shannon, and G.G. Lendaris, “A comparison of DHP based antecedent parameter tuning strategies for fuzzy control,” *Proceedings Joint 9th IFSA World Congress and 20th NAFIPS International Conference*, 2001, doi: 10.1109/NAFIPS.2001.944317.

Training Loop

Single Adam optimizer with persistent state (momentum/variance accumulate across epochs):

1. Forward pass: input → prediction + `end_state`
2. Compute `outputLoss` from prediction vs. target
3. Reverse pass: `end_state` → reconstructed input
4. Compute `reconstructionLoss` from reconstruction vs. original input
5. Backpropagate combined `totalLoss`
6. If ergodic: run `paramUpdate()` (gradient energy sensor updates alpha)
7. Optimizer step (embedding params excluded when `trainEmbedding` is `NONE`, `CBOW`, or `SBOW`)
8. If `trainEmbedding` is `CBOW`, `SBOW`, or `BOTH`: run embedding update step

Alpha annealing (ergodic): starts at 1.0 (full exploration), decays to 0.0 within the first 5% of epochs via `alpha = max(0, 1 - epoch / warmup)` where `warmup = numEpochs // 20`. Code convention (`alpha=1` means explore) is the inverse of the ergodic math convention; layers translate internally.

See `Params.md` for all XML parameters. See `Training.md` for embedding modes.

Modes of operation

Two operating modes, selected by `<architecture><conceptualMode>` (XML knob added in the substrate refactor):

Mode	Trigger	PS.forward
SERIAL / GRAMMATICAL	<code><conceptualMode>serial</...></code> (default when <code>useGrammar != "none"</code>)	IS_t per w
PARALLEL	<code><conceptualMode>parallel</...></code> (default otherwise)	IS once, th

SERIAL and GRAMMATICAL are not architecturally distinguished — grammar dispatch is a chart / rule-catalog config, not a substrate mode. PS.forward takes a single positional argument in both modes; the argument is whatever input is being processed (IS in SERIAL, IS then CS in PARALLEL refinement).

Pre-2026-05-27 “two feedback loops” retired. The legacy $S \rightarrow C$ symbolic loopback (per-stage) and $C \rightarrow P$ subsymbolic loopback (cross-forward) collapse under the substrate refactor:

- PS is a single-direction input processor; no recurrent $C \rightarrow P$ feedback at the substrate. CS state enters PS only via PS.forward(CS) in PARALLEL mode’s refinement iterations.
- Symbolic loop becomes pairwise grammar ops over STM (the signal router’s copy/reduce dispatch). **Lift** and **Lower** are binary GrammarLayer subclasses dispatched alongside **Intersection**, **Union**, etc.

The recurrent character of the architecture lives in (a) STM accumulation across words in SERIAL mode, and (b) the T-pass PARALLEL refinement loop. Cross-call serial-cache (`subspace.serial_cache`) for streaming / autoregressive contexts is preserved; gated by `PerceptualSpace._recurrent_pass_idx == 0`.

Pipeline as a unit, two-tier reset

`runBatch` is a pure compute brick: forward $\rightarrow$ loss $\rightarrow$ backward $\rightarrow$ optimizer.step. It does **not** decide when to reset per-row state, does **not** consume `_end_of_stream` for control flow, and (after Section 6 vectorization) does **not** issue any GPU $\rightarrow$ host sync inside the brick.

Reset lives in `runEpoch`. The same loop drives both byte cursor (AR text byte) and trial cursor (non-AR); `next_tick` is universal dispatch:

```
while not ds.all_done():
    inp, out, hard_eos = ds.next_tick()           # 3-tuple, host-side
    runBatch(inp, out)                           # compute brick
    flush_word_buffers()                         # materialize subspace.word
    dispatch_per_row_reset(hard_eos)             # hard resets
    dispatch_soft_reset()                       # grammar <start> reductions
    post_tick_compact()                         # truth_layer.compact
```

For AR text byte, `inp` is a byte slab and `hard_eos[b]` flips True when row `b`’s cursor exhausts a doc. For non-AR / numeric data, each tick yields one batch of trials with `hard_eos = [True] * B`.

Hard reset. TheData walks each document one slab of ≤ 1024 bytes at a time. `hard_eos` flips True on cursor exhaustion. Full row-state cascade fires for that row only; other rows continue mid-document with state preserved.

Soft reset. The active parser signals when a row’s parse reduces to `<start>`. `wordSpace._sentence_completed` is drained per-tick: re-arms `_stm_fired[b]`, clears `_last_svo[b*K..]` and parse-stack rows for `b`, but **preserves discourse history** (discourse accumulates across sentences within a document and clears only on hard reset).

No truncation. Documents longer than `slab_bytes` span multiple ticks; concatenating per-tick slabs for any row reproduces the original document byte-exact. `valid_mask: [B, K]` handles partial-fill tails via NULL-padding.

Compute-brick contract. No `.item()`, no `.tolist()`, no Python conditional on a tensor value, no GPU $\rightarrow$ host copy inside `runBatch`. The chart’s residual `.tolist()` calls retired with the `Chart` class itself in the substrate refactor; remaining graph-break sites are documented in `doc/BrickHostSyncStatus.md`.

Two-File Architecture

File	Contents
XML config (e.g. <code>BasicModel.xml</code>)	Architecture, hyperparameters
Weights checkpoint (e.g. <code>BasicModel.ckpt</code>)	Full integrated bundle: model parameters, register-buffer state, embedding

The 2026-05-12 *integrated-weights* refactor retired the separate `.kv` embedding artifact: embeddings, vocabulary mappings, and the BPE codebook now ride inside the single `.ckpt` bundle alongside the model’s other parameters. The bundle layout is:

- `state_dict`: every `nn.Parameter` and `register_buffer` in the module tree (model weights, `wv._vectors`, `TruthLayer.truths`, etc.) — serialised by the normal PyTorch path.
- `vocab_extras`: the WordVectors Python-side mappings that don’t live in `state_dict` (`index_to_key`, `counts`, `total_count`).
- `bpe_extras`: the `ChunkLayer`’s pure-Python state (merges list, vocab dict, `id_to_bytes`, growth cursors). Required because `ChunkLayer` stores its merge table as Python dicts/lists, not tensors.

`bin/embed.py` still produces standalone `.kv` artifacts for CBOW/SBOW *pre-training* studies, but those artifacts are no longer part of the runtime artifact set. Cold-start training initialises the vocabulary and BPE codebook from scratch and learns them end-to-end alongside the model weights.

Language System

The grammar dispatch runs through the **signal router** (`LanguageLayer`, `bin/Language.py`) — the single canonical parser. `WordSubSpace` owns it directly as `self.languageLayer`. The pre-substrate CKY Chart and STM shift-reduce parsers retired in Stage 3 of the substrate refactor.

The signal router represents STM as a slab `[B, N, D]`; per-layer scorers emit per-position copy/reduce scores; `binary_tiling_soft_dp` produces marginals at training (soft superposition), `binary_tiling_viterbi` produces the best tiling at eval. `Grammar.rule_probability(body)` is generalized to the per-position, per-op score head — dormant defaults (fold ops fire, negation ops don’t) carry over as initial biases. Single-application enforcement via `_fired_bodies` / `reset_derivation` carries over unchanged.

Grammar ops are `GrammarLayer` subclasses dispatched by the router as unary (copy-side) or binary (reduce-side):

- **Unary symbolic operators**: `not(S)`, `non(S)`, `swap`, `copy`, `true`, `false`.
- **Binary symbolic operators**: `intersection(S, S)`, `union(S, S)`, `conjunction`, `disjunction`.
- **Mereological operators**: `part(S, S)`, `equals(S, S)`, `query(S, S)`. Pure-geometric — the `MereologicalTree` sidecar that formerly stored explicit parent / equality links retired in favor of clipped-cosine parthood on codebook activations. See `Mereology.md`.
- **lift and lower**: now binary `GrammarLayer` subclasses (Stage 4 of the substrate refactor). Each owns an internal `SigmaLayer` (`LiftLayer`) or `PiLayer` (`LowerLayer`) for the pairwise math. No longer “substrate-borrowing” — fully self-contained binary grammar ops with `arity=2`, `tier='C'`. Typed grammar signatures still determine result order (e.g., `S4 = lift(NP3, VP1)`). See `Language.md`.
- **Butterfly mode on GrammarLayer** (Stage 5): all `GrammarLayer` subclasses accept `butterfly=True`, `N=N` for efficient cross-STM pairwise composition via a packed `nn.Parameter[n_levels, N//2, 2D, 2D]` cascade with bit-reversal permutations. Wired into `PerceptualSpace.pi` by default in butterfly-enabled configs. Closes the XOR convergence target.

Parthood (`part`) is the **fundamental** mereological operation, realized as clipped cosine projection on symbolic activations. The full suite (`whole`, `equal`, `overlap`, `underlap`, `boundary`) composes through `part` on `Basis`. `equals(S, S)` is propositional identity on `S`; delegates to `Basis.equal`.

Short-Term Memory on ConceptualSpace

`ConceptualSpace.stm` (an instance of `ShortTermMemory`) is a per-batch stack of unquantized C-tier “ideas” — the working set the signal router dispatches grammar ops over. Capacity defaults to 7 (Miller, ± 2); `<ConceptualSpace><stmCapacity>N</stmCapacity></ConceptualSpace>` overrides.

Post-substrate-refactor, `CS.forward(new_idea_subspace)` is **STM bookkeeping** — shift slots left, push the new idea onto slot 7. The legacy atomic forward fold (`sigma_percept`) is retired; `CS` no longer holds a

Layer that transforms its input. The mode dispatch:

- **SERIAL / GRAMMATICAL**: one idea pushed per word; STM shifts (oldest dropped from slot 0). Grammar ops dispatched per word or at sentence boundary.
- **PARALLEL**: $T = \langle \text{conceptualOrder} \rangle$ iteration outputs written to STM slots simultaneously; no shift.

STM is cleared on hard **Reset** (sentence boundary) and survives soft reset. The signal router consumes `stm.snapshot()` for its slab input. See `Spaces.md`.

The STM data model, the predict-then-perceive cadence (serial and parallel), the in-STM and inter-sentence predictors, masked-word reconstruction, relative-vs-absolute end-states, and the LTM chain are documented in full in the dedicated `STM.md` chapter. Note in particular that **serial mode runs WITH attention by design** (the old serial-vs-attention guard was lifted; `MentalModel.xml` is serial + `hasAttention`) — see `STM.md` Section 4.

Per-word operational flow (SERIAL mode)

In SERIAL / GRAMMATICAL mode, each word traverses a per-word path:

```
byte stream -> PS.forward(IS_t)      # MPHF surface lookup → PS lexicon
                                     # then pi(x) + sigma(x), no outer tanh
               -> CS.forward(idea)    # STM shift; push idea onto slot 7
               -> signal router dispatches grammar ops over STM
                                     # (read-only via CS; write-required via SS)
```

PS's `self.vocabulary` (Embedding) holds the per-word vectors keyed by MPHF. SS's codebook holds **paired (orthographic, semantic) rows** for the unified lexicon — the orth row is a copy of PS's per-word vector at insert time; the semantic row is random; the two are parented via `Codebook.set_part_parent`. Lookup chain: surface → MPHF → orth row → parented semantic row.

POS rides the codebook for free. The SymbolicSpace reference codebook carries two POS-bearing fields per atom: `category_ids`: [V] (hard POS tag — one of the grammar's nonterminals) and `category_logits`: [V, C] (learnable soft POS distribution per atom). So per-word snap returns (`word_id`, POS) simultaneously — no separate POS tagger needed. The router uses POS for typing reduce candidates; POS is learned through parsing alongside the codebook.

POS rides the codebook for free. The SymbolicSpace reference codebook carries two POS-bearing fields per atom: `category_ids`: [V] (hard POS tag — one of the grammar's nonterminals, e.g. NP / VP / N / V / ADJ) and `category_logits`: [V, C] (learnable soft POS distribution per atom, EMA-updated by the chart's `_apply_codebook_pos_seed` mechanism in `Language.py`). So per-word snap returns (`word_id`, POS) simultaneously — no separate POS tagger needed. The parser uses POS for typing reduce candidates ($NP + VP \rightarrow S$, etc.); POS is *learned through parsing* alongside the codebook.

See `Logic.md`, `Mereology.md`, and `Language.md`.

Shamatha Speech target. Planned narrow grammar for one-pointed object speech: complete DNF over active percepts, permitting each **conjunction** / **disjunction** only when operands' `where()` supports are connected and `when()` supports are continuous. See `Language.md`.

Sigma and Pi Layers

For weight matrix $W \in \mathbb{R}^{m \times n}$ and input $x \in \mathbb{R}^n$:

Sigma layer:

$$y_j = Wx + b = b_j + \sum_{i=1}^n W_{ji}x_i$$

Pi layer (log-space linear):

$$s_i = \log \frac{1+x_i}{1-x_i} = 2 \operatorname{atanh}(x_i)$$

$$z_j = \sum_i W_{ji} s_i + b_j$$

$$y_j = \frac{e^{z_j} - 1}{e^{z_j} + 1} = \tanh(z_j/2)$$

Forward maps $[-1, 1] \rightarrow (0, \infty)$ via `_to_mult`, `log`, `linear`, `exp`, `_from_mult`. Domain and range both $[-1, 1]$. Reverse inverts each step: `_to_mult(y)`, `log`, $W^{-1}(z - b)$, `exp`, `_from_mult`.

Motivation. The classical product form $y_j = b_j \prod_i (1 + W_{ji}x_i)$ becomes, after taking logs, a sum. The code moves into a log-multiplicative domain via `atanh`, performs a linear op there, returns via `tanh`. The `atanh` transform stretches values near ± 1 toward infinity, making the layer sensitive to strong activations.

Monotonicity of the lift / lower chain. Under `monotonic=True`, Pi/Sigma select non-negative linear layers, giving $W \geq 0$. Positive matrices are monotone on the positive cone, so lift / lower preserve parthood for activations represented in that cone. Truth-set bivectors remain live for user-supplied truths; they are explicit truth/operator surfaces rather than a space-wide output mode. See `Spaces.md`.

Dimensionality Constraints

- Input layer output dim = perceptual layer output dim (conceptual operates on both).
 - Symbolic layer input dim = perceptual layer input dim (both operate on conceptual output).
 - Output layer input dim = sum of symbolic layers' output dims.
-

Invertible Linear Layer (LDU)

Factors $W = L \cdot D_{\text{embed}} \cdot U$:

- L : unit lower-triangular ($nIn \times nIn$, diagonal = 1).
- D : diagonal vector of length `rank = min(nIn, nOut)`, embedded into $[nIn, nOut]$ by zero-padding.
- U : unit upper-triangular ($nOut \times nOut$).

Exact inverse via triangular solves: $W^{-1} = U^{-1} \cdot D^{-1} \cdot L^{-1}$. Each factor inverted by `torch.linalg.solve_triangular`. No SVD; inverse exact when all D entries are nonzero. Parameter count: $nIn^2 + \text{rank} + nOut^2$. Initialized at $L = I, d = 1, U = I$ (identity).

`naive=False` (default) applies L/D/U sequentially without materialising `W_eff` as a full matrix. `naive=True` materialises `W_eff` and its inverse.

Ergodic noise injection at the factor level, plus the `stable=True` clamp and the noise lifecycle, are documented in `Ergodic.md`.

Ergodic Exploration

See Ergodic.md.

Sentence-level AR (InterSentenceLayer)

Within-sentence training is IR-only (BERT-style masked-LM at the P-tier; see `doc/Spaces.md` Section “Within-sentence AR retirement”). The **autoregressive** signal in this architecture lives one scale up: between sentences, on a per-sentence representation `s_t`. That’s the job of `InterSentenceLayer` (alias `wordSpace.discourse`).

Sentence representation

`s_t` is the **root S-tier slot** of the body’s final stage: the single vector the start-symbol reduction wrote into. The chart’s parse trace already commits to this slot at sentence end; the layer pools `[B, N, D] → [B, D]` by taking row 0 (root). Width is `sentence_dim = n_dim` (one vector per row), **not** the full `n_symbols * n_dim` flatten that the pre-2026-05-14 contrastive layer used — that broader rep would have blown the predictor’s Linear past the allocator budget on large MM_5M-scale configs.

ARMA(p, q) predictor

`InterSentenceLayer` runs an autoregressive moving-average predictor:

```
s_hat_t = predictor(s_{t-1..t-p}, e_{t-1..t-q})
e_t      = s_t - s_hat_t
loss     = MSE(s_hat_t, s_t)           # accumulated per batch
```

- `p` = AR lag count (default 5) — last `p` sentence reps.
- `q` = MA lag count (default 2) — last `q` prediction errors.
- `predictor = nn.Sequential(Linear(p*D + q*D, H), Tanh, Linear(H, D))`, with `H = min(1024, 2*sentence_dim)`.

The MA term lets the predictor correct for systematic bias in the AR extrapolation: if the AR model consistently under-predicts the sentence rep, the residual `e_t` carries that signal forward.

Buffers (per row, non-persistent):

- `_s_history`: `[B, p, sentence_dim]` ring of last `p` sentence reps (most recent at index `-1`).
- `_e_history`: `[B, q, sentence_dim]` ring of last `q` residuals.
- `_s_count / _e_count`: `[B]` long, fill levels (cap at `p / q`).

`ensure_batch(B)` resizes these on cascade from `WordSpace.ensure_batch`; `Reset()` clears them on hard / discourse boundary. Default behaviour is to **not** auto-reset across document boundaries — the AR lags carry information through discourse continuity unless the caller explicitly calls `Reset`.

Wiring into the training loop

1. After the body finishes (sentence end), `_forward_per_stage` stashes the S-tier event on `_current_discourse_s`.
2. In `runBatch`, when training, `discourse.observe(s_tensor)`:
 - Pools `s_t = sigma_S(s_tensor[:, 0, :])`.
 - Computes `s_hat_t = predictor(_s_history, _e_history)`.
 - Returns `MSE(s_hat_t, s_t)` (None on the first call per row when the ring is empty).
 - Computes `e_t = s_t - s_hat_t`, pushes both into the rings.
3. `runBatch` adds the loss to `TheError` under category “discourse” with weight `armaScale` (training XSD knob, default 0.1).

Inference (chat-loop seeding)

`BasicModel.generate_sentence(seed_text):`

1. Calls `discourse.predict_next()` for the ARMA-predicted sentence-rep prior `s_hat_{t+1}`.
2. Lifts it through `discourse.cast` to `concept_dim` and stages on `ConceptualSpace._c_prior`.
3. Runs the IR forward — the body’s first `sigma_percept` output gets `_c_prior` summed in as a sentence-level conditioning bias before the codebook lookup. Cleared after the forward consumes it.
4. The post-body perceptual event is decoded by nearest-neighbour against the perceptual codebook, producing the (`slot`, `original`, `predicted`) triples for the seed text’s masked positions.
5. Commits the produced sentence’s S-tier root to the ARMA ring via `discourse.observe(s_tensor)`.

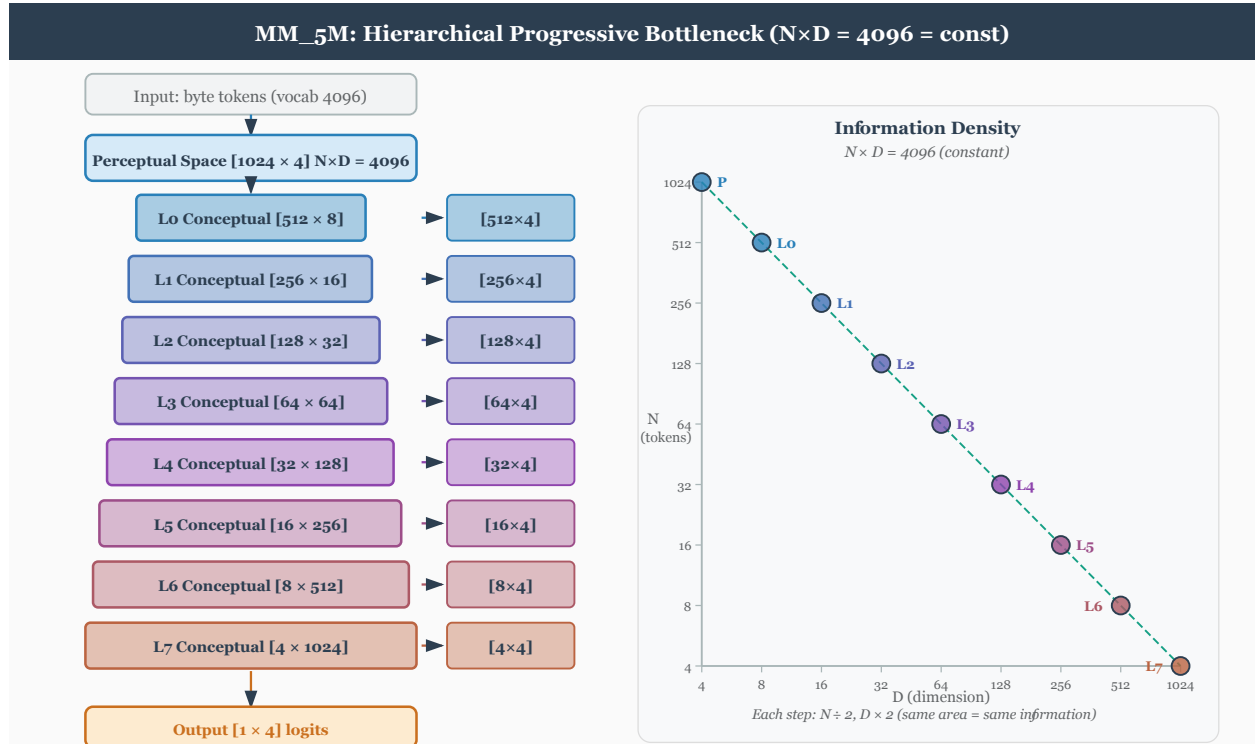
The IR head plays no role at inference — the prediction lives at the masked P-tier positions, decoded against the (frozen) perceptual codebook.

Configuration

XSD knob	Section	Default	Notes
<code><armaP></code>	<code><WordSpace></code>	5	AR lag count
<code><armaQ></code>	<code><WordSpace></code>	2	MA lag count
<code><armaHiddenDim></code>	<code><WordSpace></code>	<code>2*sentence_dim</code> (cap 1024)	predictor hidden width
<code><armaScale></code>	<code><architecture><training></code>	0.1	ARMA loss weight added to <code>TheEr</code>
<code><sentencePrediction></code>	<code><architecture><training></code>	false	Gates <code>InterSentenceLayer</code> constr

The retired pre-2026-05-14 knobs (`<sentenceContextWindow>`, `<sentenceCentroidHistory>`, `<sentenceLambda>`, `<sentencePredictionScale>`, `<sentencePredictiveScale>`, `<sentenceContrastiveScale>`) shaped the legacy contrastive cosine machinery (recent-centroid attraction + older-centroid repulsion). They are not parsed; configs that still set them are tolerated silently.

Basic Model



The Basic Model of Cognition is a neural network of four spaces: real space, perceptual space, conceptual space, and symbolic space.

Model Entities:

Entity	Meaning
$x, \hat{x}, X$	Actual and predicted input (experiences), input space
$y, \hat{y}, Y$	Actual and predicted output (actions), output space
p, P	Percepts, perceptual space
c, C	Concepts, conceptual space
s, S	Symbols, symbolic space
W_e	Experiencing, experiential weights
W_a	Acting, action weights
W_k	Knowing, knowledge weights
W_t	Thinking, thought weights

Background

Based on the Basic Model of Cognition at cognitivesciencesociety.org/framework-cognitive-science; see also The Whole Part.

Inputs and Outputs

Inputs and outputs derive from a real space that is ineffable. Adaptable parameters initialize at zero (*tabula rosa*); learning is driven by random perturbations of the map relative to its territory at *temperature*.

Inputs scale to $[-1, 1]$ via global data min/max. The signed range represents presence and absence symmetrically. Paraphrasing Augustine, *negative is the privation of the positive*.

Action is a function of perceptual space and our expectation, filtered by conceptual projections. Performance errors are defined relative to action, desired effect, and exhaustion. Expectation errors are defined relative to perception and perception-as-reconstructed.

To accommodate a changing input context window, input is augmented with relative spatiotemporal encoding by extending its dimensionality. The attention mechanism treats what and where pathways separately.

Perceptual Space

Perceptual spaces are composed of perceptual prototype vectors, or *percepts*.¹ Percepts that do not correspond to objects are *hallucinations*.

Percepts form a Voronoi tessellation using prototype and/or exemplar vectors, related via Self Organizing Map connections, operating in parallel as prototype-theory categories.

Perceptual accuracy is the distance between inputs and prototype vectors — similar to cosine similarity, becoming a simple distance measure for poorly known percepts.

Attraction and aversion warp perceptual space: attraction is a pole at the percept's location, aversion is a zero. Perfectly recognized percepts lie on the unit hypersphere within conceptual space. Two percepts can be joined into one concept without loss.

Conceptual Space

Concepts create boundaries in conceptual space, grounded in the support vectors of perception. Concepts support logical negation; they may refer to negative entities. Conceptual spaces are capable of representing negative entities. Concepts that do not correspond to percepts are *false*.

Concepts collect percepts into fuzzy sets — linear separating hyperplanes partitioning conceptual space, or sets defined as linear combinations of percept activations.

Concepts are relative: meaningful only insofar as they partition a meaningful space. As concepts approach certainty, decision boundary slope becomes infinite. A single unknowing state is common to all knowledge vectors (activation slope zero).

Conceptual spaces are similarity spaces partitioned by decision boundaries imposing positivity and negativity on a hypersphere of knowing.

Within conceptual space, attraction/aversion to concepts expresses as hyperplane bias. Certainty is the slope of the activation function (tuned after other parameters stop changing).

A concept can be split into two percepts without loss.

Symbolic Space

Symbolic Spaces are composed of percepts that reference concepts — a subtype of perceptual spaces mapping onto conceptual spaces.

Symbols have no spatial context: zero-dimensional points within perceptual space. In humans, perceptual space projects onto a 3D cerebellar subspace before symbolic encoding.

¹Perceptual space is known as “embedding space” in transformer literature.

Symbols are independent, permanent, and integral (as references). The top-down serial activation differs from bottom-up perceptual activation — it casts shadows.

The formation of symbols increases the dimensionality of conceptual space (outer product); the formation of concepts from percepts decreases the dimensionality of perceptual space (inner product).

Symbols are erroneous if their associated concept is not true. Participation in multiple sets creates structural relations that replace positional information.²

Reasoning

Reasoning in symbolic spaces involves computing with parts and wholes — Venn-diagram computations enabling dynamic Bayesian-type statistics. Learned probabilities can be stored as unidirectional connection weights between concepts, possibly with part-whole structure imposed by corresponding symbols.

Symmetric Perception: the Single Layer Linear Case

Perception is bidirectional: we see the world and the world sees us. The egocentric/cultural view treats perception as passive; in neural networks this manifests as forgetting the influence of outputs on inputs.

Symmetric Perception finds $f(x)$ minimizing MSE with respect to:

$$\begin{pmatrix} in \\ out \end{pmatrix} \cdot \begin{pmatrix} f(x) & 0 \\ 0 & f^{-1}(x) \end{pmatrix} = \begin{pmatrix} out \\ in \end{pmatrix}$$

In the linear case, $f(x)$ becomes a single invertible matrix A . When the mapping is nonlinear and an inverse may not exist:

$$\begin{pmatrix} in \\ out \end{pmatrix} \cdot \begin{pmatrix} f(x) & 0 \\ 0 & g(x) \end{pmatrix} = \begin{pmatrix} out \\ in \end{pmatrix}$$

This resembles $Ax = b$, except x is known and we want weights A mapping one space to another. The standard LLS solution $x = (A^\top A)^{-1} A^\top b$ becomes finding A such that $Ax = b$ and $A^{-1}b = x$.

Retrocausality suggests an additional objective: A should be optimal for both forward and inverse transforms. Approximately: A as a product of two matrices, each performing a perfect surjection: $A \cdot A^{-1} = I$.

$$\left(\frac{b}{2} * x^{-1}\right) \cdot \left(\frac{x}{2} * b^{-1}\right) = I$$

Best A requires iterative solution (NN learning, ping-pong style) or Gram-Schmidt orthogonalization with respect to A and A^{-1} .

Symmetric Perception: the Single Layer Nonlinear Case

If $f(x)$ and $g(x)$ share weight matrix W :

Forward computation and gradient:

$$\begin{aligned} \hat{y} &= f(g^{-1}(x)W) \\ e_y &= \frac{1}{2}(\hat{y} - y)^2 \\ \Delta W_y &= \eta(\hat{y} - y)\delta_f(W^\top \cdot g^{-1}(x))g^{-1}(x) \end{aligned}$$

²For example, $\{0,1\}$ and $\{0,1\}$, if orthogonal, pose no constraint on the location of the 1 in their 2x2 grid. However, $\{0,1,1\}$ and $\{0,1\}$ partition a 3x2 space and require two 1's on the same row — a fact present in neither set alone.

Reverse computation and gradient:

$$\begin{aligned}\hat{x} &= g(W^\top f^{-1}(y)) \\ e_x &= \frac{1}{2}(\hat{x} - x)^2 \\ \Delta W_x &= \eta(\hat{x} - x)\delta_g(W \cdot f^{-1}(y))f^{-1}(y)\end{aligned}$$

Since W is shared, summing the partial derivatives gives:

$$\frac{\partial e_y}{\partial W} + \frac{\partial e_x}{\partial W} = \eta(\hat{y} - y)\delta_f(W^\top \cdot g^{-1}(x))g^{-1}(x) + \eta(\hat{x} - x)\delta_g(W \cdot f^{-1}(y))f^{-1}(y)$$

Setting gradient to zero, both errors going to zero when $x = i$ and $y = o$. But $xy - iy = ox - yx$ may hold — simultaneous gradient descent ping-pongs. A better cost function multiplies the two error partials — intersection of the spaces satisfying both:

$$\frac{\partial e_y}{\partial W} \frac{\partial e_x}{\partial W} = \eta(\hat{x} - x)(\hat{y} - y)\delta_f(W^\top \cdot g^{-1}(x))\delta_g(W \cdot f^{-1}(y))f^{-1}(y)g^{-1}(x)$$

Distance Metrics

$$d(x_1, x_2) = \frac{\|x_1\|^n \cdot \|x_2\|^n}{\|x_1 - x_2\|^{2n}}$$

Z-plane form:

$$d(x_1, x_2) = \|x_1 - x_2\| \cdot \frac{(x - z_1)}{(x - z_2)}$$

where z_1 is a zero and z_2 is a pole.

Exploration/Exploitation as a function of temperature

$$y = x \cdot \frac{(W + rt)}{\|W\|}$$

where r is a random vector and t is temperature.

Transfer Functions

Using rotation matrices with tunable Θ before and after a diagonal projection restricts the solution space to an eigendecomposition of the orthonormal basis (an SVD). For 2D:

$$y = \begin{bmatrix} \cos \theta_1 & -\sin \theta_1 \\ \sin \theta_1 & \cos \theta_1 \end{bmatrix} \begin{bmatrix} \Sigma_1 & 0 \\ 0 & \Sigma_2 \end{bmatrix} \begin{bmatrix} \cos \theta_2 & -\sin \theta_2 \\ \sin \theta_2 & \cos \theta_2 \end{bmatrix} x$$

With $f(x) = \sin(x)$ and $g(x) = \cos(x)$:

$$\begin{aligned}\hat{y} &= \sin(\cos^{-1}(x)W) \\ \hat{x} &= \cos(W^\top \sin^{-1}(y)) \\ \Delta W_y &= \eta(\hat{y} - y) \cos(W^\top \cdot \cos^{-1}(x)) \cos^{-1}(x)\end{aligned}$$

$$\Delta W_x = \eta(\hat{x} - x) \sin(W \cdot \sin^{-1}(y)) \sin^{-1}(y)$$

The reverse perception reconstruction and the forward perception gradient share common terms. Taking the product:

$$\Delta W_y \Delta W_x = \eta(\hat{x} - x)(\hat{y} - y) \delta_f(W^\top \cdot g^{-1}(x)) \delta_g(W \cdot f^{-1}(y)) f^{-1}(y) g^{-1}(x)$$

Spaces

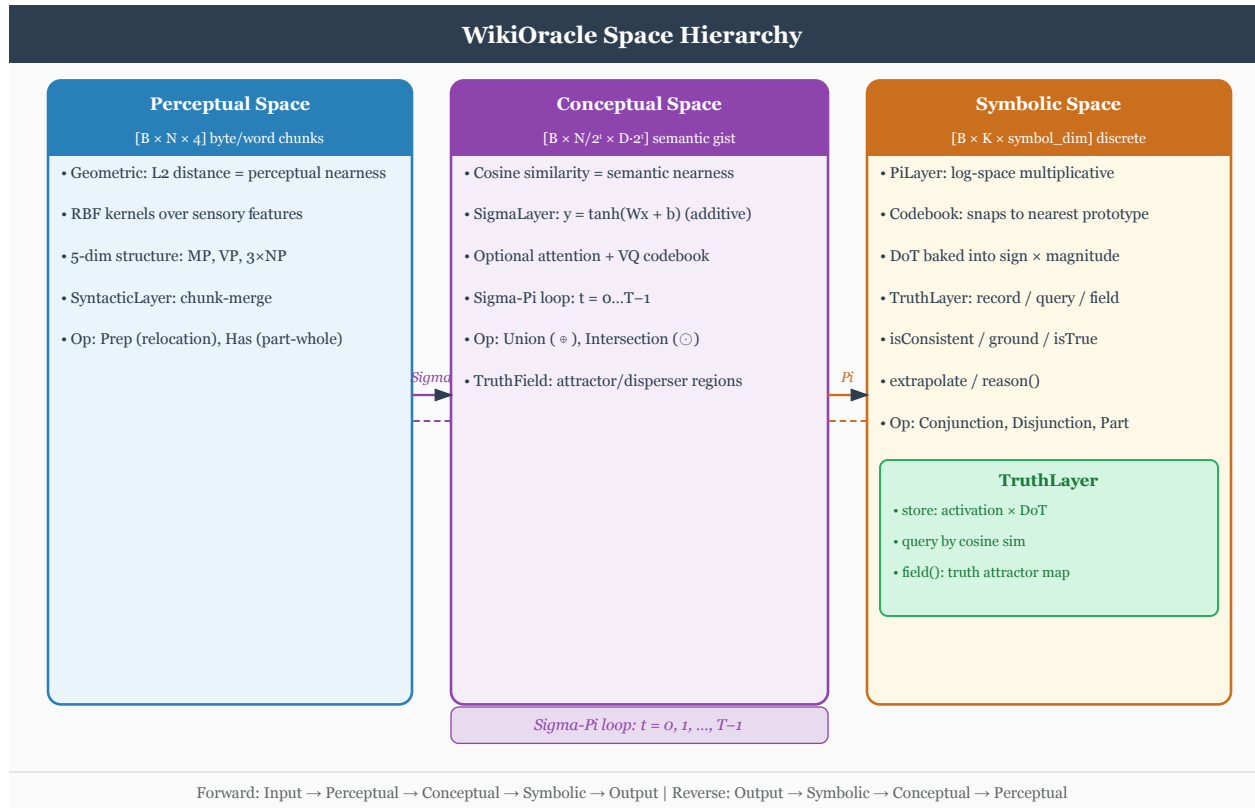
Status (2026-05-27): updated for the substrate refactor. PS is a single-arg input processor (`self.pi + self.sigma`). CS is an STM container + grammatical CPU; no atomic forward fold. SS owns the unified word lexicon codebook with paired (orth, semantic) rows. Grammar dispatch lives on the signal router (`LanguageLayer`) at `WordSubSpace.languageLayer`; the CKY `Chart` and STM shift-reduce parsers are retired. `LiftLayer` / `LowerLayer` are binary `GrammarLayer` subclasses with internal Sigma / Pi (no longer substrate-borrowing). `GrammarLayer` gains an optional butterfly cascade mode.

Overview

BasicModel is a pipeline of five **spaces** plus a grammar host (`WordSpace`), each performing a distinct representational transformation. Data flows forward from raw input to task output; the reverse pass reconstructs the original input from the symbolic representation. The legacy `SubsymbolicSpace` and `SyntacticSpace` classes have been retired — the subsymbolic role is filled by `PerceptualSpace` itself, and the grammar runs from `WordSubSpace.languageLayer` (the signal router; subsumed the retired `Chart`).

Forward: `InputSpace -> PerceptualSpace -> ConceptualSpace -> SymbolicSpace -> OutputSpace`
Reverse: `OutputSpace -> SymbolicSpace -> ConceptualSpace -> PerceptualSpace -> InputSpace`

The pre-2026-05-27 “two feedback loops” ($S \rightarrow C$ per-stage, $C \rightarrow P$ cross-forward) are retired. The recurrent character lives in (a) STM accumulation across words in SERIAL / GRAMMATICAL mode, and (b) the T-pass PARALLEL refinement loop driven by `<conceptualOrder>` via `PS.forward(CS)` iterations.



Base Class: Space

All spaces inherit from `Space`, which manages:

- **Shape management.** `inputShape` / `outputShape` as `[nObjects, nDim]`. Subclasses read dimensions from `TheObjectEncoding`.
- **Codebook / VQ quantization.** When `nVectors` > `nActive`, a codebook holds candidate vectors; top-k selection gives the bottleneck.
- **Reshape flag.** When in/out object counts differ, the `[B, nObj, nDim]` tensor is flattened before the next space and restored on the way back.
- **Attention.** Optional `hasAttention=true` reweights objects.
- **set_sigma propagation.** Ergodic-mode noise level cascades from the top-level model down through every child layer.

Reset cascade — hard vs. soft

Every space exposes `Reset(batch=None, hard=True)`. The signature is required (legacy zero-arg fallback removed).

Call form	Scope	Use
<code>space.Reset()</code>	All rows	Whole-state wipe (epoch boundary)
<code>space.Reset(batch=b, hard=True)</code>	Row <code>b</code> only	Document boundary
<code>wordSpace.soft_reset(batch=b)</code>	Row <code>b</code> sentence-scoped state	Grammar <start> reduction

Hard-reset clears: `parse stack`, `_last_svo`, `_stm_fired`, codebook commit accumulator, discourse history, `serial_cache`, `_ar_embedded`, `_end_of_stream` for the affected rows.

Soft-reset clears per-sentence working buffers (parse stack rows, `_last_svo[b]`, category and reconstruction stacks) and re-arms `_stm_fired[b]`. Does **not** touch discourse history (`InterSentenceLayer` ring buffer) or codebook EMA — those are document-scoped.

Reset is dispatched from `runEpoch`, never from inside `runBatch` (the pure compute brick — see Architecture.md).

Sigma / Pi ownership (2026-05-27 substrate refactor)

The 2026-05-13 rebalance described “each space owns one operator.” The 2026-05-27 substrate refactor revised again: PS owns **both** pi and sigma (single-arg input processor); CS has **no atomic forward operator** (STM bookkeeper); SS owns the unified word lexicon codebook (paired rows); Lift / Lower become **binary GrammarLayer subclasses** with internal Sigma / Pi (no longer substrate-borrowing).

Space	Owns
PerceptualSpace	one <code>self.pi</code> (PiLayer), one <code>self.sigma</code> (SigmaLayer), MPHF + index table, the surface-keyed Lexicon
ConceptualSpace	STM (ShortTermMemory, depth ~8)
SymbolicSpace	unified word lexicon codebook with paired (orth, semantic) rows; <code>insert_paired_word(word, vec)</code>

Composition (per-mode):

- **SERIAL / GRAMMATICAL** (`<conceptualMode>serial</...>`): one iteration per word.

```
PS_t = PS.forward(IS_t)           # MPHF + pi + sigma on per-word surface
CS_t = CS.forward(PS_t)           # STM shift + push
router.dispatch_at_C(STM)         # read-only grammar ops on STM contents
router.dispatch_at_S(STM)         # codebook-write-required ops via SS
```

- **PARALLEL** (`<conceptualMode>parallel</...>`): T iterations of PS over CS.

```
PS_0 = PS.forward(IS); CS_0 = PS_0
for t in 1..T = <conceptualOrder>:
    PS_t = PS.forward(CS_{t-1})    # refinement pass
    CS_t = CS.forward(PS_t)        # STM[t] = PS_t (parallel write; no shift)
    router.dispatch_at_C(STM)      # grammar ops after STM population
```

The legacy formula `C = sigma_percept(pi_input(IS) + pi_concept(C_prev))` is retired entirely.

Lift / Lower in the new ownership

`LiftLayer` and `LowerLayer` are no longer “rule-id annotators over shared substrate.” They are first-class binary `GrammarLayer` subclasses (Stage 4 of the substrate refactor):

- `LiftLayer(GrammarLayer)`: `arity=2`, `rule_name="lift"`, `tier='C'`. Owns an internal `self._sigma: SigmaLayer` for the pairwise additive (sigma-style) math. `forward(left, right)` delegates to `_sigma.compose`.
- `LowerLayer(GrammarLayer)`: `arity=2`, `rule_name="lower"`, `tier='C'`. Owns an internal `self._pi: PiLayer` for the pairwise multiplicative (pi-style) log-domain math. `forward(left, right)` delegates to `_pi.compose`.

Both reverse cleanly via their internal layer’s reverse. Both gain butterfly mode for free via `GrammarLayer` base inheritance (Stage 5).

The signal router dispatches them as binary reduce ops at the C tier, weighted by `Grammar.rule_probability` (the per-position copy/reduce score head).

Butterfly mode on GrammarLayer (Stage 5)

GrammarLayer(butterfly=True, N=N) allocates a packed `nn.Parameter` of shape `[n_levels, N // 2, 2D, 2D]` (`n_levels = log2(N)`), a bit-reversal permutations buffer, and LDU-parameterized invertibility per node. Identity init: cascade is identity at construction. Each subclass implements `_butterfly_pair_op(x_pair, W_node)` with its per-pair math:

- `PiLayer._butterfly_pair_op`: $\text{atanh} \rightarrow \text{einsum} \rightarrow \text{tanh}$ (multiplicative log-domain pairwise).
- `SigmaLayer._butterfly_pair_op`: $\text{atanh} \rightarrow \text{einsum} \rightarrow \text{tanh}$ (additive pairwise — same surface, different gradient regime).
- `LiftLayer` / `LowerLayer`: delegate to their internal `sigma` / `pi`.
- `IntersectionLayer` / `UnionLayer`: `min` / `max` kernels.
- `ConjunctionLayer` / `DisjunctionLayer`: hard-coded monotonic `min` / `max`.

Parameter savings: $O(N \cdot \log N \cdot D^2)$ cascade vs $O(N^2 \cdot D^2)$ for a single big matrix. Wired into `PerceptualSpace.pi` when `<PerceptualSpace><butterfly>true</butterfly><butterflyN>N</butterflyN>` is set. Closes the XOR convergence target (`test_mm_xor.py`).

Normalization and Ranges

Space	Data Contract
InputSpace	Data scaled -1..1 for scalars or vector norms
PerceptualSpace	Modal/demuxed (what/where/when encoding). Signed unit-magnitude scalars or vectors. No negation of
ConceptualSpace	Combined/muxed (event encoding). Signed unit-magnitude (tanh-bounded). Event norm on subspace .
SymbolicSpace	Symbols are percepts. Concept-to-symbol mapping. One symbol encoded at a time
OutputSpace	Rescaled from activation range to original data range

`SyntacticSpace` is retired (see Section “`SyntacticSpace` — retired” below). Grammar / chart machinery lives on `WordSpace` and is attached to `SymbolicSpace` (the canonical grammar host).

Data scaling. Data computes global `input_min/input_max` and `output_min/output_max` at load time. `InputSpace` uses `Data.normalize(x, "input")` to scale to `[-1, 1]`; `OutputSpace` uses `Data.denormalize(x, "output")` to restore the original output range.

Symbols as percepts. Symbols live in `[0, 1]`; since conceptual activations range `[-1, 1]`, the mapping is `symbol = (activation + 1) / 2`. `SubSpace.get_symbols()` / `set_symbols()` perform the conversion.

Demuxed mode. When `InputSpace.demuxed=true`, what/where/when components are stored independently in the `SubSpace` rather than concatenated. `ModalSpace` routes each component through independent `PerceptualSpaces`; downstream spaces see an identical muxed tensor via `materialize()`.

Codebook Similarity Metric

Codebook wraps `VectorQuantize`. Similarity metric per space:

Space	Codebook geometry	Stored
PerceptualSpace	$[-1, +1]^d$ hypercube	Feature <i>patterns</i>
SymbolicSpace	$[-1, +1]^d$ hypercube	Symbol <i>patterns</i>
ConceptualSpace	Unit L2-norm directions (named directions)	Concept <i>directions</i> ; input magnitude in <code>[-1, +1]</code> encodes

Euclidean (Perceptual / Symbolic)

These codebooks store *what something looks like* — $0.5 \cdot v$ carries half as much “of feature v ” as $1.0 \cdot v$, so the right notion is coordinate-wise distance. Retrieval expands $\|x - c_i\|^2$:

$$\|x - c_i\|^2 = \|x\|^2 + \|c_i\|^2 - 2(x \cdot c_i)$$

$\|x\|^2$ is constant across i and drops from argmin:

$$\arg \min_i \|x - c_i\|^2 = \arg \max_i (x \cdot c_i - \frac{1}{2} \|c_i\|^2)$$

VectorQuantize keeps $\|c_i\|^2$ in `_b_norms_sq`:

```
indices = (flat @ codebook.T - 0.5 * b_norms_sq).argmax(dim=-1)
```

One matmul + one broadcast subtract + one argmax. Skips the `sqrt`, the per-row $\|x\|^2$ add, and the cdist autograd plumbing.

Dot product (Conceptual)

ConceptualSpace concepts are *named directions* in belief space. $x \cdot c_i$ gives the *signed strength of belief that x affirms concept i* :

- +1 fully affirms; 0 orthogonal; -1 fully denies

Two consequences:

1. **Codebook must be unit L2-norm.** EMA renormalizes after each update.
2. **Input must NOT be normalized.** The magnitude *is* the certainty signal. Cosine similarity would divide it out.

For *ranking*, $x \cdot c_i$ and $\cos(x, c_i) = (x \cdot c_i) / \|x\|$ are monotone-equivalent (positive constant cancels). Omitting input normalization preserves certainty and costs less:

```
# codebook is unit L2-norm (maintained by EMA)
indices = (flat @ codebook.T).argmax(dim=-1)
```

Configuring the metric

`use_dot_product` is a class attribute on `Codebook` (default `False`). Set it on a Space subclass to opt in — `ConceptualSpace` does this. The underlying `VectorQuantize.use_cosine_sim` flag is historical; after the April 2026 perf pass, input-side normalization is gone, so the effective meaning is “codebook unit-norm; rank by dot product”.

Codebook Uniqueness Contract

Every codebook entry must be **unique under both WhereEncoding and WhatEncoding**:

- **.where** — **globally unique positional key.** Enforced structurally via the class-level `codebook_offset_registry` on `WhereEncoding` (Spaces.py:223–276). Each codebook calls `allocate_codebook_slice(n_vectors)` and gets a contiguous integer offset; all codebooks share `div_term = 2\pi / total_allocated`. Each codebook’s entries live in disjoint `.where` slices.
- **.what** — **distinct prototype content.** Identical `.what` collapses to the same parthood identity (`equal(A, A) = 1`) — a redundant pair the network can’t distinguish.

Current enforcement:

Source	Mechanism
SymbolicSpace codebook	ImpenetrableLayer overlap penalty + variance floor; five-relations classifier pushes pairs toward
ConceptualSpace codebook	ImpenetrableLayer available; not yet wired by default
PerceptualSpace Lexicon	Cosine-margin pole/antipode SBOW training
InputSpace vocabulary	Shares PerceptualSpace’s Lexicon

.**where** uniqueness is **structural** (enforced at construction); .**what** uniqueness is **learned** (encouraged by **ImpenetrableLayer** + antipodal quotient). Together they guarantee the parthood lattice is well-formed.

Lexicon (Projective Unit Ball)

The **Lexicon** (`bin/Layers.py`) backs PerceptualSpace word embeddings and SymbolicSpace symbol prototypes. Each row is a vector w_i in the **projective unit ball** — the closed ball $B^D = \{x : \|x\|_2 \leq 1\}$ with the **negation identification** $w \sim -w$ realizing real projective space $\mathbb{RP}^D$.

Terminology pin — three notions sometimes conflated:

- **Pode** of (a, b) : midpoint $(a + b)/2$; SBOW positive-pair attractor.
- **Wrapped pode**: midpoint via the $\pm$ -quotient, $(a - b)/2$; the midpoint through *negation* of b .
- **Antipode** of a single point p : furthest point. On the flat torus unique ($\text{wrap}(p + 1)$); on $\mathbb{RP}^D$ **not unique** — the maximum-distance set is the orthogonal hyperplane.

So $-w$ is the **negation** of w , *not* the antipode.

Distance and lookup

For $a, b \in B^D$ the projective squared distance is

$$d_{\mathbb{RP}}^2(a, b) = \min(\|a - b\|_2^2, \|a + b\|_2^2) = \|a\|_2^2 + \|b\|_2^2 - 2|\langle a, b \rangle|.$$

With $\text{pode}(a, b) = (a + b)/2$ and $\text{wpode}(a, b) = (a - b)/2$, $d_{\mathbb{RP}}(a, b) = 2 \cdot \min(\|a - \text{pode}\|, \|a - \text{wpode}\|)$. The lookup picks whichever rep of b (b or $-b$) is closer to a .

Sorting by smallest $d_{\mathbb{RP}}^2$ = sorting by largest score $(x, w_i) = |\langle x, w_i \rangle| - \frac{1}{2}\|w_i\|_2^2$.

Implementation: `cache W_norm2 = W.square().sum(-1)` once per optimizer step; top-k is `(x @ W.T).abs() - 0.5 * W_norm2` followed by `torch.topk` — dense matmul + abs + broadcast subtract. No $V \cdot D$ outer-product.

The Lexicon API:

```
lexicon = Lexicon(V, D)
lexicon.project_unit_ball_()          # after optimizer.step()
W_index, W_norm2 = lexicon.lookup_index()

# Projective (RP^D) --- antipode-aware, default.
idx, dist_sq, scores = Lexicon.topk_rp(x, W_index, W_norm2, k=32)

# Plain L2 --- for sites where w and -w are distinct.
idx, dist_sq, scores = Lexicon.topk_l2(x, W_index, W_norm2, k=32)

# Pairwise primitives:
Lexicon.rp_distance_sq(a, b)
Lexicon.rp_similarity(a, b)
```

```

Lexicon.rp_pode(a, b)
Lexicon.rp_wrapped_pode(a, b)
Lexicon.rp_closer_rep(a, b)           # sign(<a, b>) * b

```

For $V \gtrsim 10^5$, use `topk_rp_chunked` to bound peak score-tensor size.

SBOW training: pode (attractor) and antipode (repulsion target)

- **Pode (attractor).** Positive-pair updates pull a and b toward $\text{pode}(a, b)$; the gradient picks the closer of b and $-b$ for shorter-arc attraction.
- **Antipode (balancing repulsion target).** Negative-pair updates push the row toward the furthest point. On $\mathbb{RP}^D$ this is a $(D - 1)$ -sphere, so SBOW samples a representative orthogonal direction.

Negative-sampling gradient has two regimes by $\text{sign}\langle a, b \rangle$: positive case is standard contrastive repulsion along $(a - b)$; negative case pushes a away from $-b$ along $(a + b)$.

After every optimizer step the trainer calls `lexicon.normalize()`, clipping $\|w_i\| \leq 1$. `W_norm2` should be refreshed when weights change.

Torus primitives (`Lexicon.wrap`, `Lexicon.delta`, etc.) and the `torus=True` constructor flag remain as **legacy** static methods (the earlier Lexicon used the flat torus $T^D = [-1, 1)^D$ with wrapped MSE). New code must use the `rp_*` primitives.

InputSpace

Role. Receives the raw source buffer and lifts it into the model's internal working dimensionality.

Text mode forward. Delegates tokenization to `Lex`, producing a span table of `(start, end, type)`. Each span $\rightarrow$ a vector with two components:

- `nWhat` dims — token content, encoded via `Basis` / `Codebook` (the word embedding lookup).
- `nWhere` dims — positional information, materialized as a quadrature-sinusoidal vector via `WhereEncoding` (see the 2026-05-28 where-keyed-taxonomy plan). The integer position is recoverable via `atan2(sin, cos)` / `$_\omega_0$` and serves as the canonical positional identifier across IS / PS / SS taxonomies (no more signed-int cross-codebook hack).

Result: `[nActive, nWhat + nWhere]` tensor.

Text mode reverse. Inverts the span encoding: each vector $\rightarrow$ nearest codebook entry, then spans $\rightarrow$ characters via the stored offset table.

Numeric mode. Tensor data passes through unchanged; `LiftingLayer` projects native input dim (e.g. 784 for MNIST) to `nDim`. Non-embedding inputs are scaled to `[-1, 1]` via the global data min/max.

Key parameters.

Parameter	Description
<code>nActive</code>	Sequence length
<code>nDim</code>	Output dim per vector
<code>lexer</code>	Tokenization mode: "word" or "sentence"
<code>codebook</code>	Whether input values are discrete
<code>demuxed</code>	Store what/where/when independently

2026-05-28: per-Space `<nWhere>` / `<nWhen>` XML knobs are retired. `model.xml` carries the per-Space defaults (`<nWhere>2</nWhere>` on `InputSpace` / `PerceptualSpace` / `SymbolicSpace`, 0 elsewhere). Per-file configs no longer declare them. See `doc/plans/2026-05-28-where-keyed-taxonomy.md`.

Invertibility. Always non-invertible; reverse is a separate reconstruction using the span table.

Document streaming and valid_mask

Documents longer than `nOutput` bytes are not truncated. `TheData` maintains a per-row cursor (`doc_idx[b]`, `offset[b]`) and `next_tick()` returns (`input`, `output`, `hard_eos`) where `input` is a `[B, nOutput]` slab containing the next $\leq nOutput$ bytes from each row's current document. `hard_eos[b]` is a host-side bool set when row `b`'s cursor exhausts the current document. A short fill at document end NULL-pads the slab tail; `valid_mask: [B, K]` bool flips False for padded positions, and state-mutation propagation skips them.

Cursor universal — trial mode for non-AR data. `next_tick()` is the single dispatch for both AR text byte (rolling cursor) and non-AR data (numeric). In trial mode (`slab_bytes` not set), each tick yields one batch of trials with `hard_eos = [True] * B`. The `runEpoch` outer loop drives `ds.next_tick()` directly for both modes; the `DataLoader` exists only so existing tests can grab `loader.dataset`.

`_end_of_stream` is a host-side `list[bool]` diagnostic only; the canonical hard-reset signal is the cursor's `hard_eos`.

AR cursor unfold retirement (2026-05-13)

The legacy AR-training path padded + unfolded the embedded sentence into `[B, K, N, D]` cursor windows so the body could see a `[B*K, N, D]` parallel view of every prefix. At `bs=128, K=128, N=1024, D=10`, the unfolded tensor alone was ~320 MB.

The unfold was retired for AR training on 2026-05-13, replaced with a serial K-cursor loop (`_forward_per_stage_no_unfold`) that walked the same prefixes with a `[B, N, D]` tensor and a per-cursor causal mask.

Within-sentence AR retirement (2026-05-14)

The serial K-cursor loop itself was retired one day later: the benchmark showed `_forward_per_stage_no_unfold` running at ~18 sent/sec (the K body+head calls dominate) vs the single-shot IR fast-path's ~61 sent/sec, and the real AR objective in this architecture is **next-sentence** prediction (the discourse layer) — not next-token within a sentence.

Within-sentence training is now IR-only. `InputSpace.forward` emits `[B, N, D]` (left-aligned, right-padded to N) and `_forward_per_stage` runs a single masked-LM pass:

1. **Stem:** `InputSpace.forward + PerceptualSpace.forward` $\rightarrow [B, N, D]$.
2. **Mask:** `create_ir_mask` replaces a `mask_rate` fraction of WHAT positions with `NULL_PERCEPT`; pre-mask event stored on `_ir_pre_mask_input` as the loss target.
3. **Body:** T stages on B rows (no per-cursor walk, no causal mask).
4. **Head:** `outputSpace` $\rightarrow [B, N, predDim]$. The head is a side channel — IR loss is computed at the P-tier, not at the head.

`runBatch` reads `_ir_mask_positions` and `_ir_pre_mask_input` and computes `MSE(perceptualSpace.subspace at masked positions, _ir_pre_mask_input at masked positions)`. The `<reconstruct>concepts|symbols|both</...>` knob adds optional C-tier / S-tier reconstruction terms (target derived by lifting `_ir_pre_mask_input` through `sigma_percept`; see Plan Section “Reconstruction-loss target shape” Option B).

`<maskedPrediction>` is retired; `<reconstruct>output</...>` is retired (it was the only path that fired the reverse pipeline); `<reverseScale>` is renamed to `<reconstructionScale>` (the legacy name remains parseable with a one-shot deprecation warning).

Sentence-level AR moves to `InterSentenceLayer` — see `doc/Architecture.md` Section “Sentence-level AR (`InterSentenceLayer`)” for the ARMA(p, q) design.

PerceptualSpace

Role. Single-arg input processor. Applies `self.pi` and `self.sigma` additively to its argument (either an IS-typed subspace for surface input or a CS-typed subspace for PARALLEL-mode refinement). Owns the

surface-keyed Lexicon (`self.vocabulary`) and the MPHF + index table for per-word surface $\rightarrow$ row lookup.

Owned state (post-substrate-refactor 2026-05-27):

- `self.pi`: a single `PiLayer` (`percept_dim  $\rightarrow$  percept_dim`). Post-Stage-2, `PiLayer` inherits from `GrammarLayer`; post-Stage-5, accepts `butterfly=True`, `N=N` for cross-position cascade mode.
- `self.sigma`: a single `SigmaLayer` (`percept_dim  $\rightarrow$  percept_dim`). Same `GrammarLayer` inheritance + butterfly capability.
- `self.vocabulary`: the Lexicon (Embedding), keyed by MPHF over surface bytes. Per-word vectors are `nDim`-wide (CS-space-dim per the flat-slab invariant).
- `self._mphf_gpu_layer`: MPHF infrastructure for fast surface lookup.
- `self.chunk_layer`: BPE machinery (the `ChunkLayer` from `bin/Layers.py`).
- `self.radix_layer`: when `<chunking>radix</chunking>`, the input lookup routes through `RadixLayer` (radix trie + inverse table + learned codebook + byte fallback). `RadixLayer` is a first-class `Layer` subclass in `bin/Layers.py` (formerly the standalone `PerceptStore`). `PerceptualSpace.reverse` invokes `RadixLayer.reverse` for the structural decode (chunk-id $\rightarrow$ bytes $\rightarrow$ slot). Promotion knobs default to `threshold=4`, `min_length=2`.

The legacy `pi_input` / `pi_concept` ModuleLists are retired. The `sigma_percept`-style additive fold on CS is also retired (lives on PS now as `self.sigma`).

Forward (`PS.forward(x_subspace)`):

```
def forward(self, x_subspace):
    x = x_subspace.materialize()
    return self.pi(x) + self.sigma(x)    # no outer tanh; pi/sigma have own internal tanh
```

In SERIAL / GRAMMATICAL mode, `x_subspace` is the per-word IS subspace (surface bytes $\rightarrow$ Lexicon lookup $\rightarrow$ CS-space vector). In PARALLEL mode, the first call passes IS; subsequent T-1 calls pass CS for refinement iterations (`T = <conceptualOrder>`).

Math (pi component):

```
s = log((1 + x) / (1 - x))          # atanh domain transform
z = W_pi @ s + b_pi                 # linear in log-multiplicative space
pi(x) = (exp(z) - 1) / (exp(z) + 1) # tanh back to [-1, 1]
```

Math (sigma component):

```
sigma(x) = tanh(W_sigma @ atanh(x) + b_sigma)    # additive log-domain
```

Reverse. `PS.reverse` applies `self.pi.reverse` on the text path (LDU inverse via `InvertibleLinearLayer`). `self.sigma` is intentionally not inverted on the text path — the additive forward `pi + sigma` collapses through the codebook snap to a single prototype; recovery goes through `object_basis.reverse`. Documented at `bin/Spaces.py:8998`.

Butterfly mode (Stage 5): when `<PerceptualSpace><butterfly>true</butterfly><butterflyN>N</butterflyN>`, `self.pi` is constructed with `butterfly=True`, `N=N`. Internal storage becomes a packed `nn.Parameter[n_levels, N//2, 2D, 2D]` cascade with bit-reversal permutations. Closes the XOR convergence target. Note: butterfly weight gradient flow requires bypassing the muxed codebook snap (`<codebook>none</codebook>` on PS in `MM_xor.xml`); STE-through-snap variant is a known follow-up.

Range. Vectors live in $[-1, 1]^d$ (tanh-bounded). No negation operator — percepts represent feature magnitudes with sign indicating direction.

ConceptualSpace

Role. Transforms perceptual vectors into abstract concepts via additive linear operations. Models conceptual hyperplanes that partition perceptual space.

Geometry contrast with the Lexicon. ConceptualSpace’s codebook stores **named directions** (unit-norm $c_i \in S^{D-1}$); input magnitude in $[-1, +1]$ encodes belief certainty with sign. No antipodal identification — sign matters.

Owned layer (2026-05-13 rebalance, revised 2026-05-29 clean-stack).

Layer	Direction	Math	Notes
<code>self.sigma_in</code>	incoming-contribution fold	per-stage SigmaLayer (Ramsified across stages)	Stage 10
<code>self.sigma_cs</code>	residual-CS iteration kernel for stages $k > 0$	per-stage SigmaLayer	Same St

Clean-stack STM (2026-05-29 experiment). The Stage-10 additive composition

`folded = sigma_in(combined) + sigma_cs(prev)`

is replaced with per-stage tier attribution:

```
stage 0      folded = primary      (PS event from subspace.materialize())
stage k > 0  folded = sym          (SS event from word_subspace.materialize())
```

No additive mixing across tiers; no residual lift; trivially invertible (read-back, no inverse-Sigma needed). The `STM_k = STM_{k-1} + SS_k` carry-forward variant was tested and reverted — the pure clean-stack form is the landing point. See `doc/plans/2026-05-29-clean-stack-stm-basis-arg-radixlayer.md`.

Side effect: `sigma_in` / `sigma_cs` are dead-weight on the forward path (no gradient). Convergence on `MM_xor` continues via the `PiLayer` butterfly cascade. A forward / reverse semantic mismatch (reverse inverts a fold forward never applied) is a documented follow-up.

Reverse. `CS.reverse` is a thin pass-through (no fold layer to invert). The reverse chain operates on the terminal STM contents; per the master plan, no per-stage caches.

Activation carrier. `subspace.activation` is the scalar/presence activation used by the Space pipeline. Paired `[pos, neg]` tensors still appear in explicit grammar/truth operators that implement cataskoti logic, and in user-supplied truth-set activations, but they are no longer a space-wide output mode.

MASK on SubSpace._active. Two orthogonal per-position tensors: `activation` and `_active`: `[B, N, M]` (modality presence flags). `_apply_mask` is shape-disambiguated:

Mask shape	Effect
Aligns with <code>out.shape[-1]</code> (feature axis)	Element-wise multiply on output
Aligns with <code>out.shape[-2]</code> (position axis)	Zero masked rows of <code>_active</code> ; <code>materialize()</code> gates downstream

ShortTermMemory

ConceptualSpace owns `self.stm` — a `ShortTermMemory` instance, a per-batch stack of unquantized C-tier “ideas.” Post-substrate-refactor, this is **the primary structure CS manages** — `CS.forward` is STM bookkeeping (shift + push), with no atomic fold layer.

Property	Default	Configurable via
Capacity	8 (within Miller’s 7 ± 2 band; matches <code>wMax</code> fallback)	<code><ConceptualSpace><stmCapacity>N<</code>
Storage	<code>[batch, capacity, concept_dim]</code> buffer + <code>[batch]</code> depth pointers	<code>persistent=False</code> (working state, no
Cleared on	Hard <code>Reset</code> (sentence boundary)	Soft reset leaves it intact

API: `push(b, idea)`, `pop(b)`, `peek(b, n=0)`, `snapshot(detach=False)`, `size(b)`, `is_full(b)`, `is_empty(b)`, `clear(b=None)`, `ensure_batch(batch)`.

STM transition by mode:

- **SERIAL / GRAMMATICAL:** `_stm_shift_and_push(idea)` — shift slots 0..(cap-2) to take values from slots 1..(cap-1), write new idea to the top slot (default cap = 8, so slots 0..6 take from 1..7 and the new idea lands in slot 7). Per-word cadence: one push per `PS.forward(IS_t)` call.
- **PARALLEL:** T iterations of `PS.forward(CS)` write to STM slots simultaneously; no shift. T = `<conceptualOrder>`.

The signal router (`WordSubSpace.languageLayer`) consumes `stm.snapshot()` as its slab input for grammar op dispatch.

Both transitions follow a **predict-then-perceive** discipline (the in-STM `IntraSentenceLayer` predicts the free slot / slab from the retained context before the new event is written). The full STM treatment — predict-then-perceive, attentional filtering (serial runs WITH attention by design), relative-vs-absolute end-states, and the LTM chain of end-states — is in the dedicated STM.md chapter.

Lift / Lower as binary GrammarLayer subclasses

(Pre-2026-05-27, Lift / Lower were “substrate-borrowing” — they reached into `PerceptualSpace.sigma` and `ConceptualSpace.pi` for their math. That pattern is retired in Stage 4 of the substrate refactor. `PerceptualSpace.sigma` itself was subsequently retired in Stage 5 / 10 of the two-loop pi-sigma plan; the sigma half migrated to `ConceptualSpace.sigma_in` per stage. The 2026-05-29 clean-stack STM experiment further bypasses `sigma_in` on the forward path — see doc/plans/2026-05-29-clean-stack-stm-basis-arg-radixlayer.md.)

`LiftLayer` and `LowerLayer` are now first-class **binary GrammarLayer subclasses**, each owning its own internal sub-layer for the pairwise math:

```
class LiftLayer(GrammarLayer):
    arity = 2
    rule_name = "lift"
    tier = 'C'
    # Internal substrate: self._sigma = SigmaLayer(...)
    def forward(self, left, right):
        return self._sigma.compose(left, right)
    def reverse(self, parent):
        return self._sigma.generate(parent)

class LowerLayer(GrammarLayer):
    arity = 2
    rule_name = "lower"
    tier = 'C'
    # Internal substrate: self._pi = PiLayer(...)
    def forward(self, left, right):
        return self._pi.compose(left, right)
    def reverse(self, parent):
        return self._pi.generate(parent)
```

Both register with the signal router (`WordSubSpace.languageLayer`) as C-tier reduce ops via the existing host-layer registry path. Both inherit `GrammarLayer` butterfly mode (Stage 5) — set `butterfly=True`, `N=N` at construction to enable cross-position cascade.

Cognitive correspondence:

- **Lift** (sigma-style additive synthesis) — “lifting features onto concepts” / predication: “**the boy runs**”.
- **Lower** (pi-style multiplicative contraction) — “lowering concepts into specific percept-realizations” / attribution: “**the running boy**”.

The distinction is preserved at the rule-id level (parser records which op fired) plus the operational difference in the per-pair math.

See `Language.md` for the `GrammarLayer` specifics.

SymbolicSpace

Role. Owns the **unified word lexicon codebook** with paired (orthographic, semantic) rows. Hosts grammar ops that need codebook write access. The information bottleneck for word identity.

Owned state.

- **self.subspace.what:** a Codebook carrying symbol prototypes at width `nDim`. Includes per-row meronomy storage (`category_ids`, `part_parents`, `category_logits`). When the configured codebook is `<codebook>quantize</codebook>`, the codebook is a trainable `nn.Parameter` updated under Gray (1990) EMA with **LBG-style variance tracking and splitting**: rows whose running per-component variance exceeds a threshold split along the top-variance eigendirection, with children seeded around the parent $\pm \delta \cdot \text{variance_axis}$. (2026-05-29 Task C.) Set `<codebook>none</codebook>` to retire the SS codebook entirely (degraded mode; the grammar bivector recommender has no `W` to walk on reverse — see `test_mm_grammar_without_vqvae_learns_xor_signal`.)
- **insert_paired_word(word, ps_vector)** API: at OOV insert time, creates a paired (orth, semantic) entry — the orth row is initialized from `ps_vector` (a CS-space-dimensioned per-word vector from PS's Lexicon, per the flat-slab invariant); the semantic row is randomly initialized; the two are parented via `Codebook.set_part_parent(orth_idx, sem_idx)`.
- **get_semantic_row(orth_idx):** $O(1)$ lookup via `_paired_orth_to_sem` map precomputed at insert time.
- **META taxonomy** (2026-05-27 Stage 8): a separate cross-codebook taxonomy mapping (`ps_row`, `ss_row`) $\rightarrow$ `meta_row`. Originally keyed by signed-int refs (positive=PS, negative=SS); the 2026-05-28 plan migrates this to a `.where`-keyed integer-position scheme. META entries are populated by `ConceptualSpace._maybe_autobind_meta` at stage 0 when PS promotes a chunk and SS has a Codebook (2026-05-29 Task G — the auto-bind moved from PS to CS so the cross-space allocation fires from the layer that sees both contributions).
- Signal router dispatch site for **codebook-write-required grammar ops** (insert, EMA updates, codebook expansion / culling).

No atomic forward fold layer on SS. The pre-substrate-refactor `self.sigma` ($C \leftrightarrow S$ monotonic SigmaLayer) is retired; the $C \rightarrow S$ “naming” is now the codebook snap performed in CS via `SS.subspace.materialize()` or directly via `insert_paired_word` at write time. Symbol-side grammar math lives on the GrammarLayer subclasses (intersection, union, lift, lower, etc.), not on SS itself.

Symbols are percepts. Each symbol represents presence (1) or absence (0) of a named entity. Mapping: `symbol = (activation + 1) / 2`.

Lookup chain (surface $\rightarrow$ meaning):

1. Surface bytes enter PS via `InputSpace`.
2. MPHF on PS produces a row index into `PS.vocabulary` (the Lexicon Embedding).
3. The PS Lexicon vector (CS-space-dimensioned) flows through `PS.forward` $\rightarrow$ CS via STM push.
4. When CS-state activations need to match a symbol, they quantize against `SS.subspace.what` (the codebook).
5. Quantization can resolve to an orth row; `get_semantic_row(orth_idx)` returns the parented semantic row $\rightarrow$ the meaning.

Codebook geometry. `subspace.what.getW().shape == (nVectors, nDim)`. Each row is a free coefficient vector over conceptual axes; not unit-norm. Retrieved via Euclidean L2 (per Codebook's metric contract). `SS.nVectors` must be sized to at least `2 * expected_OOV_word_count` to accommodate paired rows (e.g., MM_5M sets `nVectors=131072 = 2 * 65536`).

Multi-stage SS. In multi-stage configs, the lexicon mirror is wired to `self.symbolicSpaces[-1]` (the terminal stage; `model.symbolicSpace`). The terminal SS must be sized for paired-row capacity. See `doc/plans/2026-05-26-two-loop-pi-sigma-substrate.md` for the dispatch rationale.

Conceptual order. `BasicModel` stores per-stage `ConceptualSpace` / `SymbolicSpace` instances for `conceptualOrder`. Symbol dimension is geometrically partitioned per order. See `Reasoning.md`.

See `BuddhistParallels.md`, `Logic.md`, `Mereology.md`, and `Language.md`.

Monotonicity of the lift / lower chain

With `monotonic=true`, the $P \rightarrow C \rightarrow S$ chain is an **order-preserving map on a positive cone**.

Three pieces:

1. **Truth-set activations may live on the positive cone** $[0, 1]^{\sim\{2K\}}$ as user-supplied paired `[pos, neg]` bivectors. The componentwise partial order $\leq$ is the parthood order for that truth surface.
2. **The `Pi` / `Sigma` maps are restricted to $W \geq 0$ entry-wise** (`monotonic=True` selects `NonNegativeInvertibleLinearLayer` or `NonNegativeLinearLayer`):

$$a \leq b \text{ componentwise} \implies Wa \leq Wb \text{ componentwise}$$

3. **Therefore `Pi` / `Sigma` preserve parthood pole-by-pole for truth-set bivectors and other positive-cone activations.**

The bivector layout keeps the contradiction corner $[1, 1]$ distinct from the ignorance corner $[0, 0]$ under positive `matmul` — a single bitonic axis would let $aP - aN$ cancel under summation.

The `ImpenetrableLayer` regularizer maintains an antichain of same-rank prototypes, complementing the structural `.where`-uniqueness from the codebook offset registry. See `Logic.md` Section Parthood as Projection and `BuddhistParallels.md`.

SyntacticSpace — retired

The standalone `SyntacticSpace` class has been retired. Grammar / parser / derivation-tree machinery now lives on `WordSpace`, which attaches a `SyntacticLayer` to `SymbolicSpace` (the canonical grammar host). The binary-derivation behavior previously documented here is preserved by `WordSpace.compose`; words are still concepts encoding grammatical rules, and the derivation is still stored as word tuples — just on `WordSpace` rather than on a separate `Space`.

See `Language.md` for grammar and parser dispatch.

OutputSpace

Role. Maps symbolic (or syntactic) vectors to task targets via linear projection.

Forward. $y = W_{\text{out}} * x + b_{\text{out}}$. Always `reshape=True` — the `[B, nSymbols, symbolDim]` tensor is flattened before projection.

Reverse. Pseudoinverse of W_{out} . Text mode snaps each output vector to the nearest codebook entry (nearest-neighbour lookup).

`getEmbeddedIO()` override. Returns raw target dimensions rather than encoded dimensions, so loss is computed in the output vocabulary space.

Text mode generation. Autoregressive: each step's predicted token vector is snapped to its nearest codebook entry and fed back as input until `maxResponseLength` or EOS.

Key parameters. `nActive`, `nDim`, `nVectors`.

Layer. `LinearLayer` with (`bias`, `temp`) for ergodic mode.

Range. Forward rescales `[-1, 1]` to the original data range via `Data.denormalize()`. Reverse applies `Data.normalize(x, "output")`.

Invertibility. Pseudoinverse; not exactly invertible in general.

Short-Term Memory

Status (2026-05-30): new chapter for the STM serial / parallel modes work (`doc/plans/2026-05-29-stm-serial-parallel-` Documents the per-batch STM buffer, the predict-then-perceive cadence (serial and parallel), the attentional-filtering regime (serial runs **with** attention by design — the old serial-vs-attention guard was lifted), the routing-parser SS-analysis / CS-execution split, the in-STM `IntraSentenceLayer` AR predictor, masked-word reconstruction via priming, the relative-vs-absolute end-state preservation with its content-aware learn-score gate, and the LTM chain of end-states feeding inter-sentence prediction. Where a piece is a deliberate scaffold or a deferred wiring, this chapter says so.

Overview

`ConceptualSpace` is, post-substrate-refactor, an STM container plus a grammatical CPU — it owns no atomic forward fold (see `Spaces.md`). The **short-term memory (STM)** is the structure CS manages: a per-batch stack of unquantized C-tier “ideas” that the model accumulates across a sentence and reduces at the sentence boundary. This chapter is the single reference for what the STM is, how it fills, what reads it, and how its end-states chain into long-term memory.

Two cadences write the STM, selected by `<conceptualMode>` (see `Architecture.md`):

`SERIAL` one idea per word; predict the free slot, then perceive (push)
`PARALLEL` one whole-slab pass; predict the slab, then perceive (set all slots)

Both follow a **predict-then-perceive** discipline: before the freshly materialized event is written into the STM, the in-STM predictor reads the retained context and stakes a prediction; the just-written event is then its supervision target. This is the concept-space analogue of a language model’s next-token objective, scaled to the in-sentence working set.

The trainable target configuration is `MentalModel.xml` (serial, `<modelType>embedding</modelType>`, attention on, sentence prediction on, FineWeb data); the comparison framing against a transformer LM is Section 12.

1. STM data model

`ShortTermMemory` (`Layers.py:8982`) is a **Layer** (not a **Space**): it carries no `SubSpace` and no forward / reverse tensor-map contract. `ConceptualSpace` builds one at construction as `self.stm` and treats it as the primary structure it manages.

Capacity. Default 8, set via `<ConceptualSpace><stmCapacity>N</stmCapacity></ConceptualSpace>` (`DEFAULT_CAPACITY = 8`), with the legacy `<wMax>` as the fallback source. Eight sits inside Miller’s 7 ± 2 band — the working-set size psycholinguistics ascribes to human short-term memory. The capacity is the rolling-window length: at steady state the STM holds the last `cap` ideas and the oldest falls off as new ones arrive.

Buffer. The data is a single per-batch tensor `[B, cap, concept_dim]` plus a `[B]` long depth-pointer vector recording how many slots each row has filled (saturating at `cap`). The buffer is a plain registered buffer with `persistent=False`: STM contents are runtime working state, not learned weights, and never enter the checkpoint. `concept_dim` is the C-tier feature width D — the **what** columns only; the positional / temporal

(where / when) columns are trimmed at the PS→CS boundary so the slab the grammar dispatch runs over is a clean concept-dim block (`_stm_payload_dim = concept_dim`).

WordSubSpace as the data carrier. After the 2026-05-21 STM-Layer refactor the `ShortTermMemory` object is itself data-free: its `_buffer` / `_depth` properties proxy to the owning `WordSubSpace`’s `_idea_*` buffers via `attach_word_subspace`. A standalone fallback buffer exists for bare-construction unit tests, but in the live model the bytes live on `WordSubSpace` and `ShortTermMemory` is a thin accessor over them.

API. `push(b, idea)`, `pop(b)`, `peek(b, n=0)` (`peek(b, 0)` = most recent), `snapshot(detach=False)`, `size(b)`, `is_full(b)`, `is_empty(b)`, `clear(b=None)`, `ensure_batch(batch)`. The signal router consumes `stm.snapshot()` as its slab input.

Lifecycle. Cleared on hard **Reset** (sentence boundary) — the per-batch idea stack drops everything from the just-finished sentence and the next sentence starts empty. Soft reset leaves the STM intact (see `Spaces.md`).

2. Serial sequencing

In `SERIAL` / `GRAMMATICAL` mode each word traverses a per-word path: `MPHF` surface lookup → `PerceptualSpace.forward` ($\pi(x) + \sigma(x)$, no outer tanh) → `ConceptualSpace.forward`, which does the STM bookkeeping. The whole pass is **predict-then-perceive per word**, implemented in `ConceptualSpace.forward` (`Spaces.py:11011`):

1. **Snapshot + predict the free (top) slot.** Before writing the new event, `_stm_predict_then_perceive_serial(idea` (`Spaces.py:10472`) takes `stm.snapshot()` and runs the in-STM predictor from the **retained context** — the snapshot with the oldest slot dropped (`snap[:, 1:]` when `depth ≥ 2`; the whole snapshot when `depth = 1`). The prediction is stashed on `self._stm_predicted_idea`. On the first word (empty STM) the prediction degenerates to zeros and **no** loss accumulates (there is no prior context to predict from).
2. **Perceive (overwrite via `_stm_shift_and_push`).** `_stm_shift_and_push(idea)` (`Spaces.py:10293`) is the perceive step. At capacity, slots `1 ... (cap - 1)` shift into `0 ... (cap - 2)` and the new idea lands in the top slot `cap - 1`; the oldest idea (slot 0) falls off. Below capacity it is a plain push at `depth[b]`.
3. **Accumulate the intra-loss.** The held prediction is scored against the just-perceived **idea** as $\mathcal{L}_{\text{intra}} = \text{MSE}(\hat{c}_t, c_t)$ (see Section 6).
4. **Language dispatch.** The signal router dispatches grammar ops over the STM contents (read-only via CS, write-required via SS).

Convention pin — newest-at-top, shift-LEFT. The plan’s “slot[0]” language is *schematic*. In this codebase the **free** / **newest** slot is the **top** (`cap - 1`) and the rolling window shifts **left**, dropping slot 0 (oldest). The “predict the free slot” step therefore predicts the top slot, and the retained context that conditions it is `snap[:, 1:]` (everything but the soon-to-be-evicted oldest slot). A named primitive `_stm_shift_for_predict` (`Spaces.py:10345`) exists for the literal “rotate so the free slot is free” step, but the hot serial path deliberately does **not** call it — `_stm_shift_and_push` already does shift-left-then-write in one pass, and a separate physical shift would double-shift the buffer. The helper is kept off the forward path and exists for independent unit-testing of the named step.

3. Parallel sequencing

In `PARALLEL` mode the per-stage forward sees the whole sentence at once: the slab `[B, N, D]` is the STM, each of the `N` positions its own slot. The same predict-then-perceive discipline applies whole-slab, implemented in `ConceptualSpace.forward` via the `folded.shape[1] > 1` branch:

1. **Predict $\hat{C}$ from the previous slab.** `_stm_predict_then_perceive_parallel(folded)` (`Spaces.py:10506`) snapshots the previous STM and runs the predictor **per slot** (no cross-slot

collapse), producing a `[B, prev_N, D]` slab stashed on `self._stm_predicted_slab`.

2. **Perceive via `_stm_set_all_slots`.** `_stm_set_all_slots(slab)` (Spaces.py:10248) writes all N positions directly as the slot stack (no shift, no mean-reduction). Capacity-clip: if $N > \text{cap}$ it keeps the last `cap` positions (drop oldest, mirroring the serial rolling window); if $N < \text{cap}$ it zeroes the unfilled tail so a prior pass’s state cannot leak through.
3. **Loss only when shapes align.** $\mathcal{L}_{\text{intra}}$ accumulates only when the predicted slab shape matches `folded` exactly. On the first pass (empty previous STM) the prediction is zeros and loss is skipped; when the previous slot count differs from N the per-slot prediction does not align with the target and the pinned decision is to skip loss for that pass rather than fabricate an overlap. Loss resumes once the STM steady-state slot count matches the slab width.

The earlier “mean-reduce to a single idea, then shift-push” pattern was a bug in parallel mode — it destroyed per-slot identity and made the reverse pipeline emit the same recon vector in every slot. The slot-preserving `_stm_set_all_slots` is the fix.

4. Attentional filtering

Serial mode IS the attentional-filtering regime. The old serial-vs-attention guard was **lifted**: `MentalModel.xml` runs serial *with* attention by design (`<conceptualMode>serial</conceptualMode>` with `<hasAttention>true</hasAttention>` on `ConceptualSpace`). Serial sequencing and attentional filtering are the same regime, not mutually exclusive options.

CS→PS meronymic Gaussian mask. On the per-word serial path the input to each word is not a raw slice but a Gaussian-windowed contextual trace, built by `gaussian_window_word(full_seq, center_k)` (Models.py:2323). For the processed word at center k a single envelope over the whole per-sentence percept sequence $[B, T, D]$ is formed:

$$w_i = \exp\left(-\frac{(i-k)^2}{2\sigma^2}\right), \quad \sigma = \text{maskRate} \cdot T,$$

normalized so the center weight $w_k \approx 1$. The peak sits at the current word; words far from k are attenuated toward 0 by the Gaussian tail. The windowed percepts are mapped into conceptual space and **summed**, so word k ’s representation carries a faint meronymic trace of its neighbours (the local context *is* part of the embedding signal). This **replaces** the prior BERT-style hide-a-token `create_ir_mask` on the per-word grammar path; it is not target-hiding — the center word is preserved. The centring is **hardcoded**: peak at the current word.

CS→SS taxonomic mask. On the symbolic side the inverse recommender zeros **non-admissible** codebook rows before a lift / lower (union / intersection) reduction so an operand can only resolve to a row of the right grammar category and conceptual order. The admissible row set is built by `priming_kwargs_for_slots` (Language.py:7340) — intersecting `refs_by_category` with `refs_by_order` per slot — and applied in the recommender’s `_row_mask` (Layers.py:10344), which admits only the category/order-matched rows plus the $\perp$ / $\top$ sentinels. A parallel `_row_weights` path multiplies a taxonomic **priming** mask over the admitted rows (the `left_priming` / `right_priming` weights from `taxonomy.priming_mask`).

Guidance-signal contract. The mask is a *guidance signal*: it biases which codebook rows / sequence positions the reduction may consult, given the current word and grammar state. The centring policy (CS→PS) is the swappable seam — the present text-reading variant centres on the current word; a future image-reading variant would swap that policy (centre on a fixated region) without changing the rest of the pipeline.

Out of scope. Learning the mask is out of scope for this work — both masks are **hardcoded** (Gaussian centring fixed at the current word; taxonomic admissibility read structurally from the codebook’s category / order metadata). A learned attention policy is a documented future direction, not built here.

5. Routing parser: SS-analysis vs CS-execution

The grammar runs through the signal router (`LanguageLayer`, `Language.py:3493`); `WordSubSpace` owns it as `self.languageLayer`. Conceptually the work splits in two:

- **SS-analysis** — `WordSubSpace.compose` (`Language.py:7693`) is the analysis stage: a soft superposition over the taxonomic codebook that selects, per tier, a **hard rule dict** `current_rules = {tier: [rule_id, ...]}`. It chooses *which* reductions fire.
- **CS-execution** — actually applying the chosen reductions (lift, lower, union, intersection, swap, quantize, not) to the concept tensors runs CS-side in `ConceptualSpace.forward` and the `SymbolicSpace` stack-route path, with the per-tier `SyntacticLayer` cursors (`Language.py:5453`) executing the unary π / σ folds on reverse. Only lift / lower / union / intersection consult the codebook (inverse-recommended); swap / quantize / not are tensor-only.

Honesty — the split is a clean code boundary only on the default-only path. When the grammar is *default-only* (every rule is the unary π / σ fold), `compose` emits `current_rules` from the grammar XML and runs **no** tensor reduction, and the per-tier `SyntacticLayer.forward / reverse` cursors do the CS-side execution — a genuinely clean analysis/execution separation. On the **full-router** path, however, `LanguageLayer.compose` does **both**: it selects the rules *and* folds the slab tensorially through the op modules (`BinaryStructuredReductionLayer.forward` $\rightarrow$ `op(left, right)`), caching the root state. The per-tier `SyntacticLayer` cursors are then **deliberately bypassed** on that path — guarded by `not _grammar_is_default_only` (`Language.py:5731`) — precisely so the reduction is not double-applied. So in the full-router case the SS-analysis and CS-execution stages are **co-located** inside `LanguageLayer.compose` rather than separated across modules. The audit found this; it is a documented task-5 follow-up, not a finished boundary.

`_grammar_is_default_only` is computed from the configured grammar at `WordSubSpace.__init__` (`Language.py:6522`) and gates both `compose` and `generate`.

6. IntraSentenceLayer

`IntraSentenceLayer` (`Layers.py:6632`) is the in-STM autoregressive predictor — the layer that stakes the prediction in predict-then-perceive. It is owned by `ConceptualSpace` (`self.intraSentenceLayer`).

Architecture: combined PI-then-Sigma, no intermediate tanh.

- `self.pi`: `PiLayer(concept_dim $to$ working_dim)`, `invertible=True`, `nonlinear=True` — the log-domain multiplicative boundary fold lifts each STM slot into the working width and bounds it to $[-1, 1]$.
- `self.sigma`: `SigmaLayer(working_dim $to$ concept_dim)`, `invertible=True`, `nonlinear=False` — a raw linear $Wx + b$ that collapses the lifted slots into the predicted idea.

The defining requirement is that there is **no intermediate activation** between the PI body and the Sigma body: `sigma` is built `nonlinear=False` so the two linear cores fuse with no extra tanh interposed (PI's own exp/tanh boundary still bounds PI's output). `working_dim` defaults to `concept_dim`, keeping both sublayers square isomorphisms so the parallel per-slot round-trip reverse($\text{forward}(x)$) $\approx x$ is exact up to the LDU inverse tolerance.

Forward signature. `forward(prior_slots, routing=None, parallel=False)` with `prior_slots`: [B, K, D]:

- **Serial collapse** (`parallel=False`, the primary regime): PI-lift every slot, **sum-fold over the slot axis**, then Sigma-collapse to one idea $\rightarrow$ [B, D].
- **Parallel per-slot** (`parallel=True`): PI $\rightarrow$ Sigma per slot, no cross-slot mixing $\rightarrow$ [B, N, D].

The serial collapse is many-to-one (sum over K slots), so its **reverse** is necessarily approximate (it divides the recovered fold equally across the $k = \text{stm_capacity} - 1$ slots); the parallel per-slot path is width-preserving and exactly invertible.

Loss. $\mathcal{L}_{\text{intra}} = \text{MSE}(\hat{c}_t, c_t)$ via `intra_loss(pred, target)`. `ConceptualSpace` accumulates it per step (`_accumulate_intra_loss`, `Spaces.py:10430`) into a live grad-carrying running sum and `consume_intra_loss` (`Spaces.py:10453`) returns the per-step mean once per batch (`runBatch` consumes it post-body / pre-backward, mirroring the ARMA term). The weight is `<intraLossWeight>` (default 0.1); the term is gated off when grad is disabled or the weight is non-positive.

7. Per-word router firing

The `<routerWireSerial>` knob (`model.xsd:76`) gates when the router fires on the serial path:

Value	Behaviour
per-word	fire per word; boundary fire off
boundary	fire only at the sentence boundary
both	(default) per-word AND boundary both fire
off	neither fires

The **per-word fire** runs `wordSubSpace.compose` over the current STM snapshot *before* `cs.forward` for the next word, populating `wordSubSpace.current_rules` for the SS dispatch (`Models.py:6053`). It lives in the host-side loop, outside the per-iteration captured graph, so even a full `languageLayer` path that introduces a host sync cannot break the per-word capture gate. The **boundary fire** (`[Layers.py / Models.py boundary hook]`) runs iff `router_wire_serial` in ('boundary', 'both').

Honesty — routing conditioning of the predictor is DEFERRED. `current_rules` is a **host-side rule dict** `{'S': [rule_id, ...]}`, **not** a `[B, concept_dim]` tensor. `IntraSentenceLayer.forward` accepts a routing tensor and projects it into a Sigma bias, but `ConceptualSpace's _intra_routing_for_predict` (`Spaces.py:10387`) only forwards **routing** when it is a CS-width tensor; given a dict it returns `None`. So the per-word fire wires the SS dispatch context but does **not** yet flow a routing tensor into the intra predictor — the predictor runs **without** the routing bias for now. Rule-aware predictor conditioning awaits a future rule-dict $\rightarrow$ CS-width projection (deliberately not built here: rule-distribution width $\neq$ concept_dim, and a naive bridge risks capture-gate breakage). This is the current state, not a finished feature.

8. Masked-word reconstruction via priming

Masked words enter the pipeline as **all-zeros** percept slots. On reverse, the router fills the blank via a best-fit codebook walk biased by the taxonymic prior, POS selection, and accumulated prediction — it reconstructs the most plausible word for the slot given the grammar context and the codebook geometry.

Priming machinery. The bias enters through `left_priming / right_priming` in `Basis.lift / Basis.lower` (`Spaces.py:1202`), forwarded down to the inverse recommender's `_row_weights` (`Layers.py:10364`) where each admitted codebook row is scaled by its taxonymic priming weight (the $\perp$ / $\top$ sentinels pinned to 1.0). Primed rows are preferred in the argmax that selects the operand. `test/test_primed_reverse_hard_mask.py` exercises this path.

Tests. The reverse roundtrip and reconstruction tests are `test/test_stm_reverse_roundtrip_lift_lower.py`, `test/test_stm_reverse_roundtrip_union_intersection.py`, and `test/test_stm_recon_from_cleared_cache.py`.

Honesty — the recon-from-cleared-cache test is an honest xfail. `test/test_stm_recon_from_cleared_cache` marks the top- k word recovery assertion xfail (not relaxed to a trivially-true bound) because of two pre-existing upstream bugs, both out of scope here and flagged as separate follow-ups:

- **Finding A** — on the untrained config the per-word forward fills the C-tier STM with NaN: `conceptualSpace.stm.snapshot()` is non-finite, so the reduced single- S seed is already NaN before reverse runs.
- **Finding B** — even with a *finite* seed, the reverse perceptual leg (`PerceptualSpace.reverse`) turns it NaN.

The non-xfail assertions in that file pin everything that *does* hold today (the per-op reverses reconstruct `[B, N, D_c]`; the decode uses the real perceptual codebook, no reimplementations). When the two upstream bugs are fixed the xfail will xpass. Documenting the contract honestly: the priming-biased recon path is built and tested, but end-to-end word recovery on an untrained model is blocked upstream.

9. Relative vs absolute end-states

At the sentence boundary the STM is reduced to its end-state. The shape of that end-state depends on whether the sentence asserts an **absolute** or a **relative** truth.

- **Absolute** sentences reduce to a **single idea** (depth 1) — the root S the start-symbol reduction wrote. This is the dominant path and the one the IR loss consumes.
- **Relative** sentences (the `part / isEqual` predicate family, which produce the `REL_T` relative start symbol) are **preserved at depth 3** as `[predicate, idea1, idea2]`.

The mechanism is a per-row `protect_depth` gate threaded into `_stm_reduce_to_single_S` (`Models.py:5470`). Each masked micro-step (`_stm_bounded_reduce_step`, `Models.py:5272`) folds only while `depth > protect_depth`: absolute rows pass `protect_depth = 1` (collapse all the way), relative rows pass `protect_depth = 3` (stop at the depth-3 end-state).

Conservative detection. `_sentence_relative_mask` (`Models.py:5387`) decides per row from the grammar — it scans `current_rules`' S -tier for a `TheGrammar.is_relative_rule` rule_id (`REL_T` LHS, or an `isEqual / queryPart / assertPart / part` op). It **defaults to collapse on any uncertainty**: a false positive would stop an absolute sentence's collapse and break the dominant path and the IR loss, so missing / empty rules, a grammar with no relative rule, or any ambiguous shape all return all-False. The absolute path stays **byte-identical** in every fall-through case.

Learn-score acceptance gate. Codebook insertion of a learned relation is gated by a content-aware `learn_score` (`_compute_learn_score`, `Spaces.py:10787`):

$$\text{learn_score} = \text{children_in_codebook} \times \text{is_truth_obvious} \times \text{resolves_contradiction},$$

each factor in $[0, 1]$. A relation is accepted **iff** `learn_score` $\geq$ `<truthCriterion>` (default 0.3). Because the factors multiply, a **low is_truth_obvious does not on its own block**: lies and uncertain relations can still be learned if the other two factors are high. At `truthCriterion = 1` nothing is learned; at 0 everything is.

Accepted insertions carry a **tetralemma trust 4-tuple** (t, f, b, n) (`TRUE / FALSE / BOTH / NEITHER`, summing to 1) computed by `_tetralemma_trust` (`Spaces.py:10818`) from the `TruthSet` posture via `assess()`, and bound onto the relation's `META` node by `insert_relation` (`Spaces.py:12559`) with the predicate as the parent and the two ideas as its taxonomy children.

Honesty — the `learn_score` factors read the **GLOBAL truth layer**. The `is_truth_obvious` and `resolves_contradiction` factors currently read the `global_truth_layer.assess()` — support and conflict respectively (`_learn_score_is_truth_obvious / _learn_score_resolves_contradiction`,

Spaces.py:10754) — behind a swappable seam (each factor is an independently overridable method, the plan’s required test seam). A **per-relation projection** is the documented refinement; the global read is a first cut, not the final formula.

The user-provided `<truth>` gold set remains a **separate surface**, recorded via `store_truths`, which arms recording by setting `truthMinMagnitude = 1` (the recording gate). The retired `accumulateTruth` knob’s role moved to `truthMinMagnitude`; see Section 4 in Params.md and Logic.md.

10. LTM as the chain of STM end-states

Long-term memory (LTM) is the **full chain of STM end-states**. It lives on `InterSentenceLayer` (Layers.py:5744) as a per-row bounded `collections.deque _stm_end_states` of size `<ltmCapacity>` (default 1024). Each entry is a time-ordered tuple

(depth: int, payload: [depth, D] tensor, tetralemma: tuple | None)

where `depth` is 1 for an absolute end-state and 3 for a relative [`predicate`, `idea1`, `idea2`] end-state (so the payloads are ragged — a fixed register-buffer tensor does not fit, hence the per-row deque). The chain is transient host-side state (`persistent=False` semantics, not in `state_dict`).

`observe_stm_end_state(depths, payloads, tetralemmas=None)` (Layers.py:6162) records **every** sentence’s end-state — it is **not** gated by `truthCriterion`. The distinction is load-bearing: `truthCriterion` gates only the separate codebook insertion of *learned relations* (Section 9), whereas LTM is the AR sequence the inter-sentence predictor consumes and must see the full history. A non-finite payload **raises** (fail-loud on numerical divergence); the deque `maxlen` evicts the oldest entry once full. `get_stm_chain(n=None, b=0)` (Layers.py:6262) returns the last n (or all) end-states for a row, oldest-first.

LTM is the **full chain**; the **TruthSet** is the accepted-belief subset (the relations that cleared the learn-score gate and were inserted into the codebook). LTM keeps everything that happened; the TruthSet keeps what was believed.

11. Inter-sentence prediction

A **lifted IntraSentenceLayer instance** (`_inter_predictor`, Layers.py:5946) predicts the next end-state over the LTM chain — the same predictor class as the in-STM one, instantiated at the inter-sentence level. Its chain window is $K = \min(\text{ltmCapacity}, 8)$ (`_inter_chain_window`): the AR signal that predicts the next end-state lives in the last handful of sentences, so a small bounded window is used rather than the full `ltmCapacity`.

`predict_next_end_state(b=0)` (Layers.py:6326) produces the next end-state **shape** $(\hat{d}, \hat{p}[\hat{d}, D])$:

- **Chain reduction (ragged $\rightarrow$ fixed).** Each chain entry is reduced to its **slot-0 root** by `_reduce_end_state_to_root` (Layers.py:6290) — for an absolute end-state slot 0 *is* the collapsed idea; for a relative end-state slot 0 is the predicate (the head the relative structure hangs off). The last K roots form a $[1, K, D]$ context, left-padded with zeros so the most recent sits at the tail (newest-at-1, like the ARMA ring).
- **Root prediction.** `_inter_predictor.forward(context, routing=None, parallel=False)  $\rightarrow$   $[1, D]$` , the predicted root.
- **Loss.** $\mathcal{L}_{\text{inter}} = \text{MSE}(\hat{p}, p)$ on the roots, accumulated by `_accumulate_inter_loss` (Layers.py:6413) and drained by `consume_inter_loss`, weight `<interLossWeight>` (default 0.1). The actual root is detached so the loss trains `_inter_predictor`, not the perception path. `observe_stm_end_state` scores the prediction made for a row against the end-state that actually arrived.

The prediction is consumed by `generate_sentence` via the `_c_prior [depth, D]` staging path: a predicted next-end-state shape is staged across the first `depth` STM slots as a sentence-level conditioning bias

(ConceptualSpace.forward's slotwise _c_prior branch, Spaces.py:11201).

Honesty — $\hat{d}$ is a **copy-last AR prior**; **reduction is slot-0 only**. The predicted **depth** $\hat{d}$ is a simple AR prior: the depth of the **most recent** end-state in the chain (a relative sentence tends to be followed by structure of the same shape; an absolute by an absolute). This delivers **depth** in $\{1, 3\}$ without a separate learned head, but it is a **scaffold** — a tiny **concept_dim** $\$ \rightarrow \$$ 2 argmax head is the documented upgrade path. Likewise the chain reduction collapses each end-state to its **slot-0 root only** (mean-over-depth is the obvious alternative; the root carries the sentence-level signal, mirroring the ARMA **_pool_sentence_rep**). A non-finite predicted root raises.

12. nanochat comparison framing

The trainable target is MentalModel.xml: `<conceptualMode>serial</conceptualMode>, <modelType>embedding</modelType>, <hasAttention>true</hasAttention> (ConceptualSpace), <sentencePrediction>true</sentencePrediction>, FineWeb data (<shardDir>data/fineweb</shardDir>). This is the configuration that exercises the full STM stack — serial sequencing with attentional filtering, the in-STM and inter-sentence predictors, and the LTM chain.`

The comparison loss is

$$\mathcal{L} = \mathcal{L}_{\text{IR}} + \mathcal{L}_{\text{intra}}$$

- $\mathcal{L}_{\text{inter}}$, $\mathcal{L}_{\text{inter}}$

the masked-LM information-reconstruction term at the P-tier (see Spaces.md) plus the in-STM next-idea term (Section 6) plus the inter-sentence next-end-state term (Section 11). This is analogous to a transformer language model's next-token cross-entropy, but computed **in concept space** rather than over a token vocabulary: $\mathcal{L}_{\text{intra}}$ predicts the next idea within a sentence, $\mathcal{L}_{\text{inter}}$ the next sentence's end-state shape, and $\mathcal{L}_{\text{IR}}$ reconstructs masked content — the three together are the concept-space counterpart of the autoregressive LM objective a system like nanochat trains.

Out of scope. The comparison harness itself — running this against a transformer baseline and reporting the numbers — is a **separate plan**. This chapter documents the loss that *would* be compared and the configuration that produces it; it does not build the benchmark.

See also

- Spaces.md — ConceptualSpace as STM container; the ShortTermMemory API; Sigma / Pi ownership.
- Architecture.md — modes of operation (serial / parallel); the per-word operational flow; InterSentenceLayer ARMA predictor.
- Language.md — the signal router, **compose** / **generate**, GrammarLayer reductions.
- Mereology.md — parthood as clipped-cosine projection; the relations the relative predicates draw on.
- Reasoning.md, Logic.md — the truth surfaces and tetralemma trust.
- Params.md — `<stmCapacity>`, `<intraLossWeight>`, `<interLossWeight>`, `<routerWireSerial>`, `<lrmCapacity>`, `<truthCriterion>`, `<truthMinMagnitude>`.

Language

This document is synchronized to the code as of 2026-05-29. The source of truth is bin/Language.py and bin/embed.py. (The earlier bin/typed_stack.py, bin/stm_driver.py, and bin/parse_state.py modules were retired in the 2026-05-21 / 2026-05-29 refactors; their functionality folded into bin/Layers.py and bin/Language.py.)

2026-05-29 deltas:

- `unreduce()` passes the tier-local Basis (Codebook) to binary GrammarLayer reverses as `basis=tier_basis` (replacing the prior raw-W form). `UnionLayer.reverse` / `IntersectionLayer.reverse` extract `W = basis.getW()` internally and dispatch to `Ops.disjunctionReverse` / `Ops.conjunctionReverse`. Layers that don't accept the `basis` kwarg yet are handled by a `TypeError` fallback. No back-ref is stored on the layer.
- `MetaLayer` was renamed to `SymbolizeLayer` (no semantic change).
- Word-mode parse appends a `\x00` null sentinel after the words slab for explicit end-of-sequence on the forward path.
- See `doc/plans/2026-05-29-clean-stack-stm-basis-arg-radixlayer.md`.

Current Parser Surface

`WordSpace.compose()` and `WordSpace.generate()` are the public parser entry points. `WordSpace.parser_backend` selects the implementation:

backend	status	notes
chart	default	Compatibility path. Runs the existing chart / Viterbi machinery.
stm	active	Shift/reduce over <code>ConceptualSpace.stm_typed</code> ; requires an attached <code>KnowledgeView</code> .
parallel	bridge	Builds the STM driver, then runs the chart path authoritatively.

`WordSpace.routerKind` is separate from `parserBackend` and only affects the chart backend:

routerKind	status
chart	Default chart scorer / inside pass.
signal	Signal-router grammar path used by configs such as <code>XOR_grammar.xml</code> .

The chart parser has not been deleted. It remains the default because legacy configs can instantiate without a knowledge artifact, while STM cannot.

Retired XML Knobs

These XML fields are no longer read by the runtime and should not appear in `data/*.xml`:

- `WordSpace.useGrammar`
- `WordSpace.chartCompose`
- `WordSpace.softChartCompose`
- `bivectorOutput`

Grammar mode is derived from the loaded grammar: default-only unary `pi` / `sigma` rules derive `useGrammar == "none"` internally; any non-default operator rule derives `useGrammar == "all"`.

SS-analysis vs CS-execution. `WordSubSpace.compose` is the SS-side *analysis* stage (it selects the per-tier hard rule dict `current_rules`); the CS-side *execution* (applying lift / lower / union / intersection / swap / quantize / not to the concept tensors) runs in `ConceptualSpace.forward` and the per-tier `SyntacticLayer` cursors. This split is a clean code boundary **only on the default-only path**: on the full-router path `LanguageLayer.compose` does both selection and tensor reduction, and the per-tier cursors are deliberately bypassed (`not_grammar_is_default_only`). See STM.md Section 5 for the accurate, audited account.

Grammar

The `Grammar` is the singleton `Grammar` instance. Rules are loaded from XML `<WordSpace><language><grammar>` blocks or from a configured grammar CFG. A `RuleDef` stores:

```
(tier, canonical, arity, method_name, lhs, rhs_symbols)
```

The grammar accepts explicit conceptual-order suffixes:

```
NP3
NP4
VP1
MP1
S4
S5
```

The intended current style is explicit ordered rules, for example:

```
S4 = lift(NP3, VP1)
S5 = lift(NP4, MP1)
```

Here all NPs share base category NP, but participate at different conceptual orders. The parser carries stack frames as `(category, order)`, so NP3 and NP4 both have `category == "NP"` but orders 3 and 4.

Kleene order forms such as NP* are still parsed for compatibility, but they are not required by the current grammar style.

Knowledge Artifacts

`embed.build_knowledge_section(grammar)` creates the parser knowledge section:

- `taxonomy`: base category refs plus explicit ordered refs.
- `reference_codebook`: scalar prototypes and `order` per ref.
- `typed_indexes`: `refs_by_category` and `refs_by_order`.
- `grammar.rule_order_signatures`: serialized rule typing.

For explicit ordered grammar, the taxonomy stores ordered refs as children of their base category:

```
NP
|-- NP3
`-- NP4
```

`KnowledgeView.category_of_ref(ref_id)` returns the base category (NP), while `KnowledgeView.order_of_ref(ref_id)` returns the conceptual order (3, 4, etc.).

`WordSpace.category_codebook` has been retired. The live category embedding is:

```
WordSpace.category_embedding: nn.Embedding
```

STM Shift/Reduce

The STM backend uses:

- `ConceptualSpace.stm_typed`: a `TypedStack` carrying payload, category, order, and ref id.
- `STMDriver`: shift/reduce coordinator.
- `RuleScorer`: MLP over top stack payloads.
- `embed.admissibility_mask`: hard category/order mask over grammar rule signatures.

SHIFT snaps each input vector to the nearest live reference, then pushes:

```
payload, category, order, ref_id
```

REDUCE computes a typed admissibility mask, applies it to rule logits, softmaxes over admissible rules, chooses the argmax for the current greedy path, and records the chosen score/probability.

Typed admissibility is exact for explicit rules:

```
S4 = lift(NP3, VP1)
```

fires only when the stack top matches:

```
left:  category NP, order 3
right: category VP, order 1
```

Syntax And SVO

`BasicModel.write_syntax_tree()` (in `bin/Models.py`) emits an XML syntax-tree dump per batch row when configured with a syntax output path. It reads the chart's per-row derivation trace and the symbolic codebook's per-atom category tags; see the function docstring for the exact format. SVO extraction is performed by the STM path on the canonical subject lift over a transitive verb-phrase derivation; the object role is the NP operand inside that verb-phrase derivation, not a separate semantic category.

Mereology

Single-page reference for the mereological grammar: parthood as the fundamental operation, the five mereological relations, and the `ImpenetrableLayer` regularizer.

2026-05-29 delta — **binary GrammarLayer reverses take Basis, not W.** `Ops._binary_op_recommend(result, W, op_name, ...)` (the mereology-guided recommender that walks `W` rows to find an operand pair such that $\text{op}(x_1, x_2) \approx \text{result}$) is still keyed on the raw `W` tensor at the low-level kernel signature. But the public `UnionLayer.reverse(parent, basis=None)` / `IntersectionLayer.reverse(parent, basis=None)` now accept a `Codebook` / `Basis` object (typically `SymbolicSpace.subspace.what`) and extract `W = basis.getW()` internally before dispatching to `Ops.disjunctionReverse` / `Ops.conjunctionReverse`. The chart reverse / signal-router dispatch (`bin/Language.py::unreduce()`) passes `basis=tier_basis` at the call site; no back-ref is stored on the layer. See `doc/plans/2026-05-29-clean-stack-stm-basis-arg-radixlayer.md`.

Codebook IS the meronymic structure. The standalone `MereologicalTree` sidecar that formerly stored explicit parent / equality links was retired in favour of pure-geometric parthood on the `SymbolicSpace` bivector codebook. The grammar layers `PartLayer`, `IsEqualLayer`, `EqualLayer`, and `QueryLayer` operate directly on codebook bivector activations via clipped cosine projection — no separate adjacency table, no `<architecture><mereologicalTreeSize>` XML knob (silently ignored if present). Asserted meronymic relations are learned by training pulling the codebook geometry into the right configuration.

Parthood as Clipped Cosine Projection

Parthood is the single fundamental operation. Every other mereological relation is defined in terms of it.

For two concepts $A, B \in \mathbb{R}^D$:

$$\text{part}(A, B) = \frac{\max(0, A \cdot B)}{|A| |B|}$$

The clipped cosine projection lies in $[0, 1]$ and satisfies Boole's contrapositive:

$$\text{part}(A, B) = \text{part}(-B, -A)$$

because $(-B) \cdot (-A) = A \cdot B$ and norms are sign-invariant.

Empty-operand contract. When $|A|$ or $|B|$ is near zero, **part** returns 1.0 — the empty set is part of everything.

The Full Suite

Every member of the suite is expressible through **part**:

Method	Formula	Interpretation
part (A, B)	$\max(0, A \cdot B)$	A
whole (A, B)	$\text{part}(B, A)$	A contains B.
equal (A, B)	$\text{part}(A, B) \cdot \text{part}(B, A)$	Mutual parthood, $[0, 1]$.
overlap (A, B)	$0 < \text{equal}(A, B) < 1$	Strictly-partial mutual parthood (indicator).
underlap (A, B)	$\text{equal}(A, B) = 0$	No mutual parthood (indicator).
boundary (A, B)	$\$$	$\text{part}(A, B) - \text{part}(B, A)$

Region Partition

equal(A, B) partitions into three disjoint regions:

Region	Condition
Underlap	$\text{equal} = 0$
Overlap	$0 < \text{equal} < 1$
Identity	$\text{equal} = 1$

Under clipped cosine, **part** is symmetric, so the three regions collapse to a single scalar classifier. For a future **Basis** with asymmetric **part**, the general five-relation table applies:

Five-relation table (general case, $\tau \approx 1$, $\epsilon \approx 0$):

Relation	Condition
disjoint (a,b)	$P[a, b] < \epsilon$ and $P[b, a] < \epsilon$
part (a,b)	$P[a, b] > \tau$ and $P[b, a] < \epsilon$
part (b,a)	$P[a, b] < \epsilon$ and $P[b, a] > \tau$
equal (a,b)	$P[a, b] > \tau$ and $P[b, a] > \tau$
overlap (a,b)	Both partial

Asymmetric subsumption. Since **part** is symmetric under clipped cosine, classical asymmetric subsumption is not encoded in raw magnitude. It is recovered **relationally** via figure / ground: compare $\text{part}(A, B)$ against $\text{part}(A, \neg B)$. This is what makes Boole’s contrapositive hold exactly.

Mereological Fusion

Fusion of a truth set is the **least upper bound** (LUB / join) over stored truth vectors:

$$\text{fusion} = \max_i \text{truths}[i]$$

(elementwise max). In bivector space, the fusion vector names the top-right corner of the axis-aligned bounding hyperrectangle dominating every stored truth. Fusion is the geometric dual of luminosity: LUB (join) vs GLB (meet).

DoT is already baked into each stored truth (`stored[i] = activation_i * degree_i` in `TruthLayer.record`), so fusion is trust-weighted automatically.

See Logic.md section **Fusion** for full discussion and the leading-bivector / paired-index layout caveat.

ImpenetrableLayer: Five-Relations Regularizer

`ImpenetrableLayer` regularizes the `SymbolicSpace` symbol codebook. It classifies each ordered pair of codebook rows (i, j) into one of the five mereological relations (using `Basis.part`), and penalizes partial overlap when paired with a trust mismatch. The learned half of the Codebook Uniqueness Contract.

Penalty

\$\$ \mathcal{L} = \mathrm{overlap_weight} \cdot \frac{\sum_{i \neq j} \mathrm{overlap}(i, j) \cdot |\mathrm{trust}(i) - \mathrm{trust}(j)|}{\{K(K-1)\}}

- $\mathrm{variance_floor}$ term

Overlap strength is damped to zero as the pair approaches **equal**:

$$\text{overlap_strength}(i, j) = \min(P[i, j], P[j, i]) \cdot (1 - \max(P[i, j], P[j, i])^k)$$

with $k = \text{equal_suppression}$ (default 4.0). Penalty is zero for **disjoint**, **equal**, and **strict part**, active only for **overlap**.

A separate `variance_floor` term guards against row collapse.

Trust

Per-row usage frequency, derived from VQ EMA counts:

$$\text{trust}(i) = \frac{\text{cluster_size}[i]}{\sum_j \text{cluster_size}[j]}$$

When VQ is absent, trust falls back to $\|cb[i]\| / \max_j \|cb[j]\|$.

Relative relations and tetralemma trust. A relative sentence in the **part** / **isEqual** predicate family is preserved at depth 3 as `[predicate, idea1, idea2]` and may be learned into the codebook as a ternary META edge (predicate as parent, the two ideas as children) via `SymbolicSpace.insert_relation`. An accepted relation carries a **tetralemma trust 4-tuple** (t, f, b, n) (TRUE / FALSE / BOTH / NEITHER, summing to 1) from the TruthSet posture, gated by the content-aware learn-score against `<truthCriterion>`. See STM.md Section 9.

Diagnostics

After `forward()`: `last_overlap_loss`, `last_variance`, `last_relation_counts` (dict summing to $K(K-1)$).

Configuration

XML knobs (under `SymbolicSpace`):

Knob	Default	Meaning
<code>overlapWeight</code>	0.1	Weight of overlap $\times$ trust-diff penalty
<code>varianceFloor</code>	0.01	Minimum per-dim std
<code>fullPartThreshold</code>	0.9	τ : part score above this is “full part”
<code>disjointThreshold</code>	0.1	ϵ : part score below this is “disjoint”
<code>equalSuppression</code>	4.0	k : damping exponent near the equal corner

Why This Design

- **Parthood as projection.** One formula, Boole-contrapositive exact, continuous in $[0,1]$. The old composite formula $\text{conjunction}(1 - \text{dist}(x, x \cap y), 1 - \text{dist}(y, x \cup y))$ mixed set-valued operands with a distance.
- **Overlap penalty (not antisymmetry).** Legacy `ImpenetrableLayer` penalized mutual parthood $P[i,j] \cdot P[j,i]$ directly, pushing *every* overlapping pair apart including synonyms. The five-relations design leaves `equal` pairs alone and only penalizes the ambiguous middle region, gated by trust mismatch.
- **Trust via VQ EMA.** Already tracked for VQ commit loss.
- **Parthood is preserved by Pi / Sigma.** Under `monotonic=true`, Pi/Sigma are restricted to $W \geq 0$. User-supplied truth-set bivectors live on the non-negative paired `[pos, neg]` cone, where componentwise $\leq$ is the parthood partial order. Each lift / lower preserves parthood pole-by-pole for that truth surface. The bivector layout keeps contradiction `[1, 1]` distinct from ignorance `[0, 0]` under positive matmul. See `Spaces.md` “Monotonicity of the lift / lower chain”.

Logic

Overview

This document defines the logic system at three levels:

1. **Subsymbolic (vector / field level)** — geometry in `ConceptualSpace`
2. **Symbolic (scalar level in [-1,1])** — order + polarity in `SymbolicSpace`
3. **Rationality** — propositional truth store built on both

Executable implementations of the subsymbolic and symbolic operators are the `Method` subclasses in `Language.md`.

1. Subsymbolic Layer

Objects: vector sets (B, N, D) interpreted as RBF / luminosity fields.

Operators

- **Union**: `union(A, B) = concat(A, B)`. Combine sets.
- **Intersection**: Co-supported regions (RBF product / merge).
- **Negation (affirming)**: `neg(x) = -x`. Antipodal opposition on hypersphere.
- **Non (non-affirming)**: `Basis.non()` — bitonic returns zero (complete withdrawal); monotonic is `relu(x - threshold)`.
- **Parthood** (fundamental): `Basis.part()` — clipped cosine projection:

$$\text{part}(x, y) = \frac{\max(0, x \cdot y)}{\|x\| \cdot \|y\|}$$

Satisfies Boole’s contrapositive trivially (dot product and norms sign-invariant under joint negation). The full suite (**whole**, **equal**, **overlap**, **underlap**, **boundary**) composes through **part**.

2. Symbolization

For $X \in (B, N, D)$:

$$s(X) = 2 \cdot \text{mean}(\|x_i\|) - 1 \in [-1, 1]$$

Interpretation: $+1$ = strong presence, 0 = neutral, -1 = absence.

3. Symbolic Layer (Scalars in [-1,1])

Basis supports **monotonic** (plain min/max over positive-cone values, including user-supplied truth-set bivectors) and **bitonic** (sign-aware). Monotonic forms here; bitonic forms (RadMin, RadMax) in Section 7. See `Spaces.md` “Monotonicity of the lift / lower chain”.

Let $a, b \in [-1, 1]$.

Operator	Bitonic	Monotonic
Negation (affirming)	$\text{neg}(a) = -a$	—
Non (non-affirming)	$\text{non}(a) = 0$	$\text{non}(a) = \text{relu}(a - \tau)$ (learnable τ)
Union	RadMax (Section 7)	$a \cup b = \max(a, b)$
Intersection	RadMin (Section 7)	$a \cap b = \min(a, b)$

Parthood as Projection

Parthood is the **fundamental mereological operation**. For $A, B \in \mathbb{R}^D$:

$$\text{part}(A, B) = \frac{\max(0, A \cdot B)}{\|A\| \cdot \|B\|} \in [0, 1]$$

Boole’s contrapositive $\text{part}(A, B) = \text{part}(-B, -A)$ holds trivially.

The full suite composes through **part**:

Method	Formula
<code>whole(A, B)</code>	<code>part(B, A)</code>
<code>equal(A, B)</code>	<code>part(A, B) · part(B, A)</code>
<code>overlap(A, B)</code>	<code>0 < equal(A, B) < 1</code>
<code>underlap(A, B)</code>	<code>equal(A, B) = 0</code>
<code>boundary(A, B)</code>	<code>\$</code>

Under clipped cosine, **part** is symmetric, so **equal** reduces to **part**². Asymmetric classical subsumption is recovered **relationally** via figure/ground: compare `part(A, B)` against `part(A, ¬B)`. See `Mereology.md`.

For ternary scalar values (monotonic projection):

<code>part(a,b)</code>	+1	0	-1
+1	1	1	0
0	1	1	1
-1	0	1	1

Zero is vacuously part of everything (empty-set convention).

4. Key Insight

- Subsymbolic = geometry
- Symbolic = order + polarity
- Symbolization = norm projection

Cleanly separates representation (vectors) from logic (scalars).

5. Rationality

Rationality is the propositional logic layer, built on S-tier grammar rules `part(S, S)` and `equals(S, S)`.

Truth Statements

A **truth statement** is any assertion processed through the full pipeline. The **TruthLayer** — owned by `WordSpace`, reached as `self.wordSpace.truth_layer` — stores activations **scaled by** `DegreeOfTruth`:

$$\text{stored} = \text{activation} \times \text{degree}$$

Degree	Stored Vector	Effect
+1	full activation	attractor
<code>0 < d < +1</code>	scaled activation	weak attractor
0	zero vector (inert)	prunable
<code>-1 < d < 0</code>	scaled, negated	weak disperser
-1	negated activation	disperser

TruthLayer (WordSpace.truth_layer)

TruthLayer is instantiated by `WordSpace.__init__`. `SymbolicSpace.forward` records gold-<truth> activations when the recording gate is armed — i.e. when `truthMinMagnitude` is set to its armed value 1. The <accumulateTruth> knob that previously gated this is **retired**: `store_truths` now arms recording by setting `truthMinMagnitude = 1`, and the default 0.3 leaves recording off during training. See STM.md Section 9 and Params.md:

- **record(activation, degree)** — store `activation * degree`.
- **query(activation)** — find the closest stored truth by cosine similarity.
- **field(concepts)** — project all stored truths into ConceptualSpace as a scalar field over concept vectors.

Truth Field

$$f(c) = \frac{1}{n} \sum_{i=1}^n c \cdot t_i$$

where $t_i = \text{activation}_i \times \text{degree}_i$. Two kinds of regions:

- **Attractors** ($f \rightarrow +1$): concept vectors near +DoT truths.
- **Dispersers** ($f \rightarrow -1$): concept vectors near -DoT truths.

Consonance and Dissonance

1. **Internal consonance** — stored truths should be mutually consistent.
2. **External consonance** — incoming statements are evaluated against the truth field.

Propositional Structure

Both parthood and equality are S-tier operations over symbolic activations:

- **part(S, S)** — containment by clipped-cosine projection. Grammar applies it as `score * B`, scaling the whole by parthood degree.
- **equals(S, S)** — identity as mutual parthood; delegates to `Basis.equal`.

Together they define a partial order over symbolic activations.

Truth Accumulation Pipeline

Truth entries arrive as (text, DoT) pairs. `store_truths()` stages all texts on TheData and runs a standard inference epoch via `runEpoch(split="runtime")`. During forward, `SymbolicSpace.forward()` records each raw symbolic activation (degree 1.0). After the epoch, each activation is scaled by its DoT:

$$t_i = \text{activation}_i \times \text{DoT}_i$$

Two-phase design (encode all, then scale) ensures all truths are encoded in the same pipeline context before DoT modulation.

6. Consistency and Verification

Internal Consistency (within a TruthSet)

Leave-one-out check: for each t_j , evaluate the field produced by the rest of the set at c_j :

$$d_j = f_{\setminus j}(c_j) = \frac{1}{n-1} \sum_{i \neq j} c_j \cdot t_i$$

- $d_j > 0$: consonant; $d_j \approx 0$: independent; $d_j < 0$: dissonant.

A disperser ($\text{DoT} < 0$) near an attractor ($\text{DoT} > 0$) naturally produces negative field values, flagging the contradiction.

Verification (incoming statement against stored truth)

`verify(statement)` processes the statement, then queries the field:

$$v = f(c_a) = \frac{1}{n} \sum_{i=1}^n c_a \cdot t_i$$

v	Interpretation
$v \rightarrow +1$	strong support
$v \approx 0$	no opinion
$v \rightarrow -1$	strong contradiction

`wordSpace.truth_layer.query()` returns the single closest truth and its degree for point lookups.

Logical Entailment (symbolic closure)

Subsymbolic field checks detect *geometric* consistency but not *logical* entailments. Example: `part(A, B) = +1` and `part(B, C) = +1` entail `part(A, C)`, but A and C could be geometrically distant. A symbolic closure layer would:

1. **Extract propositions** by decomposing stored S-tier activations into relation + two operands.
2. **Compute closure** via transitivity:
 - $\text{part}(A, B) \wedge \text{part}(B, C) \Rightarrow \text{part}(A, C)$
 - $\text{equals}(A, B) \wedge \text{equals}(B, C) \Rightarrow \text{equals}(A, C)$
 - $\text{equals}(A, B) \wedge \text{part}(A, C) \Rightarrow \text{part}(B, C)$

Each entailed proposition inherits $\text{DoT} = \min$ of the premise DoTs.

3. **Inject entailments** as new symbolic activations recorded in the TruthLayer.

Worth implementing for reasoning rather than retrieval. The geometric field alone is sufficient for soft consistency checks and nearest-truth lookups.

7. Radial Operators for Hypersphere Ternary Logic

Ternary Logic Operators			
+1 = true 0 = unknown -1 = false			
NOT (neg(a) = -a)		NON (non(a) = 0)	
a	¬a	a	non(a)
+1	-1	+1	0
0	0	0	0
-1	+1	-1	0

Intersection (agree → min a,b)			
a \ b	+1	0	-1
+1	+1	0	0
0	0	0	0
-1	0	0	-1

Union (agree → max a,b ; 0 absorbs)			
a \ b	+1	0	-1
+1	+1	+1	0
0	+1	0	-1
-1	0	-1	-1

PART (mereological, range [0, 1])			
a \ b	+1	0	-1
+1	+1	0	0
0	+1	+1	+1
-1	0	0	+1

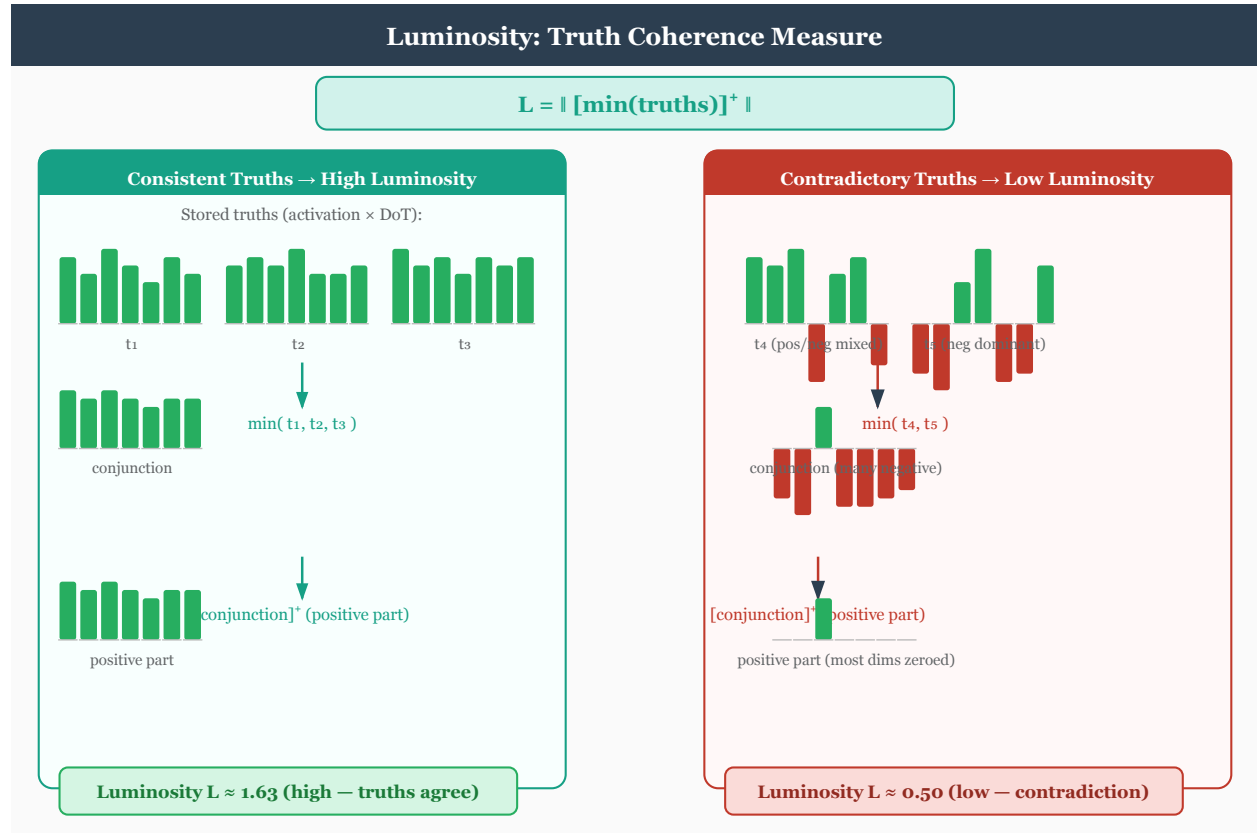
| PART(a,b): 1 when a ⊆ b; 0 is part of everything; opposite signs → 0 | | | |

RadMin, RadMax, NOT, and NON operate over signed magnitude space [-1, +1]. Sign = direction of assertion; magnitude = strength; zero = **unknown**. +1 = True; 0 = Unknown (no assertion); -1 = False. Magnitude ordering: $x \subset y$ iff $|x| < |y|$.

Op	Behavior	Ternary truth table
RadMin (AND)	Sign disagree → 0; else $\mathrm{sign}(x) \cdot \min(x , y)$	x
RadMax (OR)	Sign disagree → 0; else $\mathrm{sign}(x) \cdot \max(x , y)$	x
NOT	-x. Involutive; preserves magnitude; NOT(0) = 0.	input → -input
NON	0 for all x. Absorbing; idempotent; withdraws assertion (vs. NOT which inverts).	input → 0

Both RadMin and RadMax are commutative and collapse to zero on sign disagreement.

Luminosity



Luminosity measures truth-set coherence as a scalar:

```
luminosity = ||relu(min(truths))||
```

Element-wise min computes the conjunction (point where all truths agree); relu removes negative dimensions; L2 norm gives brightness.

- **High luminosity:** truths are coherent and mutually reinforcing.
- **Low luminosity:** truths are sparse, contradictory, or incoherent.

Two roles in the model:

1. **Top-down bias:** concept input scaled by luminosity each conceptual iteration: `concept_input * (1 + truthBiasScale * luminosity)`.
2. **Loss modification:** low luminosity increases training loss.

Fusion

Fusion is the **mereological least upper bound** of the stored truth set:

```
fusion = max_i truths[i]
```

In bivector space (paired-index [p0, n0, p1, n1, ...]), fusion names the top-right corner of the smallest hyperrectangle dominating every stored truth componentwise. Geometric dual of luminosity:

- **Luminosity** = `||relu(min(truths))||` — GLB / meet, scalar coherence.
- **Fusion** = `max(truths)` — LUB / join, vector coverage.

Trust (DoT) is already baked into stored truths, so fusion is trust-weighted automatically.

Layout caveat. TruthLayer’s paired-index slicing (`[..., 0::2]` for positive poles, `[..., 1::2]` for negative) assumes a *repeated* bivector layout. The SymbolicSpace codebook uses a *leading* bivector plus positional trailers: `[pos, neg, where..., when...]`. Callers feeding SymbolicSpace activations into `luminosity()` must slice the leading 2 dims first (`acts[..., :2]`); see `truth_loss` call-site in `bin/Spaces.py`.

Supporting measures

- **Consistency** (`isConsistent()`): folds truths into a union via successive `Basis.disjunction()`. In bitonic mode, conflicting +/- assertions cancel.
- **Grounding** (`ground()`): finds the minimal subset entailing a query.
- **TruthLoss**: additive penalty for contradictions, measured by union norm reduction. See Reasoning.md.
- **Derive**: pairwise mereological inference via `part()`; generalized by `extrapolate()` to all two-argument methods.

Semantic Summary

Operator	Type	Role	Behavior on Contradiction
RadMin	Binary	Conjunction (AND)	Collapses to zero
RadMax	Binary	Disjunction (OR)	Collapses to zero
NOT	Unary	Sign-flipping negation	N/A
NON	Unary	Non-affirming negation	N/A

8. Cross-references

The level-crossing operations (`lift`, `lower`, `intersection`, `union`), Pi/Sigma layer ownership, witness-set / slab-lattice geometry, and `Ops.*` namespace inventory have moved out of this document.

- **Ops.* reference**: Language.md.
- **PiLayer / SigmaLayer ownership** (`forwardPi/reversePi/forwardSigma/ reverseSigma` aliases): Spaces.md.
- **Mereological suite** (`part`, `whole`, `equal`, `overlap`, `underlap`, `boundary`, `copart`): Mereology.md.
- **Tetralemma bivector layout** (TRUE/FALSE/BOTH/NEITHER corners): Spaces.md and BuddhistParallels.md.

Reasoning System

2026-05-29 delta: the chart / signal-router reverse path now passes the tier-local Basis (`subspace.what`) to binary GrammarLayer reverses as `basis=tier_basis`. The mereology-guided recommender (`Ops._binary_op_recommend`) walks the Codebook’s W rows to find operand pairs (x_1, x_2) such that $\text{op}(x_1, x_2) \approx \text{parent}$. Under `<codebook>none</codebook>` on SymbolicSpace the recommender has no rows to walk and falls back to the lossy (`parent`, `parent`) pseudo-inverse — degrading the reasoning loop’s structural recovery on multi-stage chart parses. See `doc/plans/2026-05-29-clean-stack-stm-basis-arg-radixlayer.md`.

Four methods on `BaseModel` for truth-aware inference, a bidirectional reasoning loop, and a grammar learning mode. Builds on the TruthLayer infrastructure (Logic.md) and grammar composition (Language.md).

Partitioned Symbolic Space

The symbolic dimension is statically partitioned across conceptual orders using geometric decay. Each order writes only to its slice of `symbolSum`, while reading the full vector as feedback.

```
order 0: [0,      D//2)      <- 1/2 of symbol_dim
order 1: [D//2,   3D//4)     <- 1/4
order 2: [3D//4,  7D//8)     <- 1/8
...
last order: remainder of D
```

Makes the symbolic space **self-describing**: position reveals conceptual order. Truth methods use `_activation_order()` to determine a query's order by finding the partition with the highest energy. Partition boundaries are precomputed once at model creation via `BasicModel._order_partitions`.

Reasoning Methods

`isConsistent()` -> dict

Analyzes the TruthSet for internal consistency by folding all stored truths into a single summary via successive `Basis.disjunction()`. In bitonic mode, conflicting +/- assertions on the same dimension cancel to zero. Returns {'consistent': bool, 'score': float, 'sites': tensor, 'union_vector': tensor}.

`ground(activation, threshold=0.6)` -> dict

Finds the minimal subset of the TruthSet entailing a query activation. Uses `_activation_order()` to filter truths by partition. Falls back to `TruthLayer.derive()` for indirect derivation. Returns {'grounded': bool, 'basis': [indices], 'trace': [...], 'confidence': float}.

`isTrue(activation)` -> float

Grounds a proposition and returns a scalar Degree of Truth in [-1, 1]. Positive = true, negative = false, zero = unknown. Delegates to `ground()`.

`extrapolate(grammar, seed_indices, max_new, attenuation)` -> dict

Generalizes `TruthLayer.derive()` to all two-argument grammar methods (union, intersection, equals, part). For each pair of stored truths, applies every eligible method and accepts results that preserve or increase luminosity. Accepted truths recorded at `attenuation * min(DoT_i, DoT_j)`. Returns {'added': [indices], 'rejected': [(i, j, rule, delta_lum), ...]}.

TruthLoss

Additive loss penalty for false propositions, via `<TruthLoss>` in model.xml (default 0.0 = disabled).

Measures the **union norm reduction** when a proposition is included in the TruthSet union via `Basis.disjunction()`:

```
truth_union = disjunction(all stored truths)
extended    = disjunction(truth_union, new_proposition)
penalty     = max(0, ||truth_union|| - ||extended||)
```

Case	Effect
Agreeing proposition	Preserves/extends union dims → no penalty
Unknown proposition (zero dims)	Passes through → no penalty
Contradicting proposition	Cancels conflicting dims → positive penalty

DoT weighting is implicit: stored vectors carry DoT in magnitude, so contradicting a high-DoT truth causes a larger norm drop.

TruthLoss is **additive** and coexists with **multiplicative** luminosity modulation (`totalLoss * (1 + lum_weight * (1 - luminosity))`).

Bidirectional Reasoning Loop

`BasicModel.reason(givens, target, direction, max_steps):`

- **Forward** (`givens` $\rightarrow$ conclusion): Encode givens into TruthSet, extrapolate new truths each step, check `isTrue(target)` until DoT exceeds threshold or `max_steps` is reached.
- **Reverse** (`target` $\rightarrow$ grounding): Encode target, call `ground()` to find minimal basis, extrapolate if insufficient.

Luminosity non-decrease is the validity certificate.

Grammar Learning

`BasicModel.grammar_learning_step()` learns grammar weights from a symbolic reconstruction objective:

1. Forward pass produces `symbolSum`.
2. Reverse pass reconstructs input.
3. Re-encode reconstruction to `symbolSum_hat`.
4. Loss = $||\text{symbolSum_hat} - \text{symbolSum}||^2$ (symbolic, not conceptual).
5. Optional luminosity validity penalty for rules decreasing luminosity.

Paraphrase-invariance holds because semantically similar sentences snap to nearby codebook entries.

Parser And Conceptual Order

Grammar mode is derived from the loaded grammar block. Default-only unary `pi` / `sigma` rules take the fast path; non-default operator rules enable grammar-directed parser dispatch.

`conceptualOrder` controls the number of $P \rightarrow C \rightarrow S$ stages and the symbol partition geometry. Higher-order symbols write to later partitions, so truth grounding, consistency, and extrapolation can respect conceptual order.

The parser backend is selected by `WordSpace.parserBackend`:

Backend	Use
<code>chart</code>	Default compatibility path.
<code>stm</code>	Shift/reduce over typed STM with admissibility masks from grammar signatures.
<code>parallel</code>	Migration bridge: initializes STM while chart remains authoritative.

Explicit ordered grammar is preferred:

```
S4 = lift(NP3, VP1)
S5 = lift(NP4, MP1)
```

Here all NPs share base category NP; the suffix gives the conceptual order. Lift and lower are the only syntactic operations that change argument/return order.

Configuration

Parameter	Location	Default	Description
<TruthLoss>	<training>	0.0	Additive truth-loss weight
<conceptualOrder>	<architecture>	1	Percept→Concept→Symbol iterations
<parserBackend>	<WordSpace>	chart	Parser backend: <code>chart</code> , <code>stm</code> , or <code>parallel</code>
truthMinMagnitude	<SymbolicSpace>	0.3	Gold-<truth> recording <code>gate</code> (repurposed). Arme
truthCriterion	<architecture> / <SymbolicSpace>	0.3	Learn-score acceptance gate for learned relative-se
intraLossWeight	<training>	0.1	In-STM next-idea loss $\mathcal{L}_{\text{intra}}$ weight (<code>IntraSentem</code>
interLossWeight	<training>	0.1	Inter-sentence next-end-state loss $\mathcal{L}_{\text{inter}}$ weight. Se
routerWireSerial	<architecture>	both	Per-word router-fire gating on the serial path (<code>per</code>
ltmCapacity	<WordSpace>	1024	LTM chain capacity (<code>InterSentenceLayer</code> deque

The relative-vs-absolute end-state machinery, the content-aware learn-score gate, and the tetralemma trust 4-tuple carried on accepted relative META edges are documented in STM.md Section 9.

Contemplative Awareness Methods

Four stubs on `BaseModel` characterizing stages of contemplative awareness as spatial/computational properties. Each raises `NotImplementedError` — they define the target characterization, not implementation.

Method	Stage	Characterization
<code>Contiguous()</code>	One-Pointedness (Shamatha / FA)	Single connected, convex region in <code>PerceptualSpace</code> ; contiguous span
<code>Continuous()</code>	Simplicity (Continuity / OA)	Concept states flow continuously; Jacobian of forward map is bounde
<code>Peaceful()</code>	One Taste (Emotional Symmetry)	<code>TruthLayer</code> luminosity uniformly high across stored propositions
<code>Done()</code>	Buddhahood (Non-Meditation)	Model is a fixed point of forward-reverse; reconstruction loss zero

Shamatha Speech is the target grammar mode for `Contiguous()`: complete DNF object grammar plus spatiotemporal contiguity. Every `conjunction` / `disjunction` over object parts must keep `where()` support connected and `when()` support continuous. Differs from serial mode — may reduce over all active percepts at once; rejects scattered object fields, not multi-percept fields. See `Language.md` Section Shamatha Speech Mode and `plans/2026-04-28-shamatha-speech-contiguity-handoff.md`.

Testing

Unit tests in `basicmodel/test/test_reasoning.py` cover all methods without requiring a trained model. English-level tests (syllogisms, contrapositives, semantic equivalence) are `@pytest.mark.xfail` until word identity is learned through training.

Training

2026-05-29 deltas:

- **Embedding unit-ball normalization.** The training loop calls `Lexicon.normalize()` (unit-ball projection on the per-row vectors) right after `optimizer.step()` in `bin/Models.py::train_loop`. Keeps embedding vectors from drifting off the unit ball under `JOINT` / `BACKPROP` modes.
- **Seeded retries for MM_xor tests.** `test_learns_xor_signal` and `test_convergence` in `test/test_mm_xor.py` use for seed in (42, 123, 7): `torch.manual_seed(seed)`; ... (the previously-dead `seed` loop variable in `test_convergence` is now actually consumed). Pass-if-any-attempt-converges semantics.
- **Reconstruction default.** `<reconstruct>concepts</reconstruct>` is the default in `data/model.xml`; per-experiment XMLs no longer override it.

- **Supervised output loss restored.** `bin/Models.py::train_loop` computes MSE on the supervised output in addition to the reconstruction loss (`reconstructionScale-weighted`).
- See `doc/plans/2026-05-29-clean-stack-stm-basis-arg-radixlayer.md`.

Overview

Two phases: **embedding pretraining** and **network training**. Embedding pretraining builds word vectors from a large corpus. Network training uses those vectors as input representation and learns to predict and reconstruct sentences.

The two phases can overlap: `<trainEmbedding>` controls whether and how embeddings continue to evolve during network training.

Phase 1: Embedding Pretraining (make `train` / `embed.py train`)

Produces a static embedding artifact (e.g. `BasicModel.kv`). Output path from `<embeddingPath>`.

Pipeline

FineWeb-EDU parquet shards

- > Pass 1: stream documents, lex + parse, count words, build vocabulary
- > Pass 2: stream documents again, train SBOW per sentence, discard examples
- > Save `WordVectors` to `sentence.pt`

SBOW (Sentence Bag of Words)

Sentence-level variant of CBOW. Given a sentence of N words:

Leave-one-out centroid for word i :

$$c_i = \frac{1}{N-1} \sum_{j \neq i} v_j$$

Logits (unnormalised, for every word in vocabulary V):

$$z_i = W_o c_i + b_o$$

Softmax probability:

$$P(w_i | c_i) = \frac{\exp(z_{i,w_i})}{\sum_{k=1}^V \exp(z_{i,k})}$$

Loss (cross-entropy, averaged over all N positions):

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \log P(w_i | c_i)$$

Gradient of the loss has two terms:

- **Attractive** (from position i where w_i is the target): pulls v_i toward directions that increase its probability.
- **Repulsive** (from positions $j \neq i$ where v_j is part of the centroid): pushes v_i away from centroids whose targets are other words.

The softmax partition function ensures all V words compete: any word too close to a centroid it doesn't belong to increases the denominator and is pushed away with force proportional to its softmax probability. This adaptive repulsion distributes vectors across the embedding space.

```
vecs = embedding(idx)                # [N, dim]
total = vecs.sum(dim=0)              # [dim]
centroids = (total - vecs) / (N - 1) # [N, dim] leave-one-out
logits = linear(centroids)           # [N, vocab] full softmax
loss = cross_entropy(logits, idx)
```

CBOW (Continuous Bag of Words)

CBOW predicts a single target word from its context. Context mean (with padding for variable-length contexts):

$$\bar{c}_i = \frac{1}{|C_i|} \sum_{j \in C_i} v_j$$

Loss applies the same negative-sampling objective per-word to the padded context mean rather than the leave-one-out centroid:

$$\mathcal{L}_{\text{CBOW}} = -\log \sigma(\bar{c}_i^\top v_{w_i}) - \sum_{k=1}^K \log \sigma(-\bar{c}_i^\top v_{w_k^-})$$

where K is the number of negative samples and w_k^- are sampled words.

Two-Pass Architecture

- **Pass 1 (vocabulary):** Stream all documents, count every word. Words meeting `min_count` are promoted; model (embedding + linear head) allocated once at final vocab size.
- **Pass 2 (training):** Stream documents again. Per sentence, run one SBOW step and discard examples. Multiple epochs re-stream the data.

Configuration (Makefile variables)

Variable	Default	Description
BASIC_SHARDS	1	FineWeb-EDU parquet shards
BASIC_MAX_DOCS	10000	Max documents to process
BASIC_VEC_SIZE	100	Embedding dimensionality
BASIC_EPOCHS	10	Training epochs
BASIC_MIN_COUNT	10	Min word count for vocabulary inclusion
BASIC_BATCH_SIZE	8	Batch size (unused by SBOW; per-sentence)

Memory Considerations

Dominant cost is the linear output head: `vocab_size * vector_size` parameters plus optimizer state. With 200K vocab and 100 dims:

- Embedding: $200\text{K} \times 100 \times 4 \text{ bytes} = \sim 80\text{MB}$
- Linear head: $100 \times 200\text{K} \times 4 \text{ bytes} = \sim 80\text{MB}$
- Adam optimizer: $\sim 320\text{MB}$ total

Full softmax computes $(N, \text{vocab_size})$ logits per sentence. Training runs on CPU by default (`BASICMODEL_DEVICE=cpu`) to avoid MPS/GPU memory limits.

Phase 2: Network Training (`BasicModel.py`)

The network learns to predict and reconstruct sentences using pretrained embeddings.

Within-sentence training objective (IR-only)

Within-sentence training is **always IR** (masked-LM at the P-tier). `create_ir_mask` replaces a `mask_rate` fraction of WHAT positions with `NULL_PERCEPT` and snapshots the pre-mask event on `_ir_pre_mask_input`; `runBatch` computes `MSE(perceptualSpace at masked positions, _ir_pre_mask_input at masked positions)` — no head loss, no reverse pipeline. The legacy `<maskedPrediction>` knob and the AR / ARUS / ARIR modes were retired 2026-05-14; sentence-level AR moved to `InterSentenceLayer` (see `doc/Architecture.md` Section “Sentence-level AR (`InterSentenceLayer`)”).

Knob	Effect
<code>maskRate</code>	Bernoulli mask probability at the P-tier (BERT default 0.15)
<code>reconstructionScale</code>	Blend weight between output and reconstruction loss; <code>total = (1 - r)*output + r*recon</code> . Legacy
<code><reconstruct></code>	Target tier for the optional forward-only reconstruction loss: <code>none</code> / <code>symbols</code> / <code>concepts</code> / <code>both</code> .

`<trainEmbedding>`: Embedding Update Mode

Controls whether and how embeddings evolve during network training. When enabled (CBOW, SBOW, or BOTH), implements EM-like alternation:

- **E-step (network):** Forward + masked prediction + backward + optimizer.
- **M-step (CBOW/SBOW):** Run one embedding update on the same sentence.

Value	Embedding	Model layers	Description
NONE	Frozen	Trained	Only model layers train
CBOW	CBOW (padded context, own optimizer)	Trained	Model layers train separately
SBOW	SBOW (centroid, own optimizer)	Trained	Faster variant
BACKPROP	Backprop only	Trained	Codebook trained purely via model loss
BOTH	SBOW post-batch	Trained	Two optimizers
JOINT	Single backward	Trained	Single optimizer: combined model + SBOW loss

Gradient Flow Through Codebook

`<trainEmbedding>` determines whether the codebook weight is **detached** during forward/reverse and whether it appears in the main optimizer:

<code>trainEmbedding</code>	<code>detach()</code>	In main optimizer	Method
NONE	Yes	No	Frozen
CBOW	Yes	No	CBOW (own optimizer)
SBOW	Yes	No	SBOW (own optimizer)
BACKPROP	No	Yes	Model loss only
BOTH	No	Yes	SBOW + backprop
JOINT	No	Yes	Single combined loss

- **NONE** is recommended for constrained hardware (Apple MPS <16GB).
- **CBOW/SBOW** maintains clean EM separation.

- **BACKPROP** is the simplest trainable mode. Embedding shaped entirely by the task; best for small vocabularies.
- **BOTH risks gradient interference** — reconstruction pulls embeddings apart (distinguishability); SBOW pulls co-occurring words together. These forces can conflict because they use separate optimizers.

JOINT: Single Combined Loss

Avoids EM alternation by computing one combined loss before one backward:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{model}} + \lambda \cdot \mathcal{L}_{\text{SBOW}}$$

where λ is `trainEmbeddingRatio` (default 0.1).

SBOW loss uses the same negative-sampling objective:

$$\mathcal{L}_{\text{SBOW}} = -\frac{1}{N} \sum_{i=1}^N \left[\log \sigma(c_i^\top v_{w_i}) + \sum_{k=1}^K \log \sigma(-c_i^\top v_{w_k^-}) \right]$$

Advantages. One optimizer, one momentum buffer; gradient coherence; no post-batch step.

Trade-off. Both objectives share LR and Adam state. If loss surfaces have very different curvature, λ may need tuning.

Why MSE over Embeddings (not Cross-Entropy)

- “cat” prediction when target is “kitten” incurs less loss than “democracy” — embedding space captures semantic similarity.
- Output dim is the embedding dim (~100) vs. vocab size (~200K).

Training Loop

Data flows through `SentenceStreamDataset` wrapped in `DataLoader`. The ordered training list is split into $B = \text{batchSize}$ contiguous slabs of length $L = \text{len}(\text{split}) // B$; at step t , row b is item $b * L + t$. Temporal context is coherent across steps. No per-epoch global shuffle.

For each B-wide batch:

1. `BasicModel.Start(inputData)` cascades reset through every Space.
2. `InputSpace.forward()` lexes/embeds once into $[B, N, D]$ (left-aligned, right-padded to N). No K axis, no cursor unfold.
3. `create_ir_mask` replaces a `mask_rate` fraction of WHAT positions with `NULL_PERCEPT`; pre-mask event stashed on `_ir_pre_mask_input` and the position mask on `_ir_mask_positions`.
4. `_forward_body` runs T stages on B rows.
5. `_forward_head` produces $[B, N, \text{predDim}]$ (a side channel — IR loss is computed at the P-tier, not at the head).
6. `runBatch` computes loss via `TheError.add`:
 - **reconstruction** (P-tier): MSE between post-body `perceptualSpace` and `_ir_pre_mask_input` at masked positions.
 - Optionally `reconstruction_c` / `reconstruction_s` (C-tier / S-tier) when `<reconstruct>` is `concepts` / `symbols` / `both`, weighted by `reconstructionScale`.
 - **arma** (sentence-level): `InterSentenceLayer.observe(s_t)` MSE between the ARMA(p, q) prediction and the current sentence rep, weighted by `armaScale`.
7. One `backward()` + `optimizer.step()` per `DataLoader` yield.
8. Embedding training (CBOW/SBOW/BOTH) runs once per batch.

SBOW vs CBOW

Property	CBOW	SBOW	Masked Prediction (Phase 2)
Targets per sentence	1 (pick one word)	N (every word)	N (one per masked pos)
Context	Other N-1 words	LOO centroid of N-1	All unmasked (+ future truncati
Positive updates	1 per step	N per step	N per step
Repulsive force	vocab-1 implicit via softmax	$N \times (\text{vocab}-1)$	Implicit via MSE
Signal density	Low	High	High
Loss	Cross-entropy over vocab	Cross-entropy over vocab	MSE over embeddings
Updates embeddings	Yes (own optimizer)	Yes (own optimizer)	Only for BACKPROP/BOTH/JOINT
Used in (<code><trainEmbedding></code>)	CBOW	SBOW, BOTH, JOINT	BACKPROP, BOTH, JOINT

SBOW provides richer signal — every word acts as both context (for others) and target (predicted from others). The full softmax pushes vectors to distribute across the hypersphere.

Three-File Architecture

Artifact	Example	Contents	Updated by
XML config	<code>data/BasicModel.xml</code>	Architecture, hyperparameters, paths	Hand-edited
Embedding	<code>data/BasicModel.kv</code>	Word vectors ($V \times d$)	Phase 1
Weights	<code>data/BasicModel.ckpt</code>	Model layer parameters	Phase 2

Checkpoint excludes embedding parameters (`wv._vectors`). Enables:

- Retrain embeddings without touching model weights (`--force-embeddings`)
- Retrain model with frozen codebook (`<trainEmbedding>NONE`)
- Swap codebooks between models sharing the same architecture

Embedding Space Geometry

The full softmax provides a distributional guarantee that random negative sampling cannot match. For predicting w_i from centroid c_i :

$$\mathcal{L}_i = -\log \frac{\exp(z_{i,w_i})}{\sum_{j=1}^V \exp(z_{i,j})}$$

The gradient with respect to the logit for any non-target word $k \neq w_i$:

$$\frac{\partial \mathcal{L}_i}{\partial z_{i,k}} = P(k \mid c_i)$$

Words with high softmax probability (too close to the centroid) receive the strongest repulsive gradient; words already far away contribute almost nothing. This naturally spreads vectors across the hypersphere.

At equilibrium, no word can move closer to any centroid without increasing the loss. Random negatives, by contrast, mostly sample words already far away on a high-dimensional hypersphere and contribute little useful gradient.

Profiling

--profile wraps Phase 2 in cProfile:

```
make train_micro_remote # add --profile via train.py
# Or:
```

```
python bin/train.py --model data/BasicModel.xml --profile --max-docs 500
```

Output: a .prof file in output/profiles/ plus a top-30 summary to stdout. View with snakeviz:

```
pip install snakeviz
snakeviz output/profiles/train_20260319_111441.prof
```

For live profiling, py-spy can attach by PID (requires root on macOS):

```
pip install py-spy
sudo py-spy record -o profile.svg --pid <PID> --duration 30
```

Ergodic Exploration via Adaptive Bias-Variance Control

BasicModel decouples **what** the network learns (weights W) from **how much it explores** (a scalar α governing the bias-variance tradeoff). Two algorithms operate in tandem:

Component	Algorithm	Role
Weights W	Standard Adam	Gradient descent on the loss landscape
Tradeoff α	Gradient Energy Sensor	Measures loss-surface curvature to set exploration

α is **not trained by gradient descent**. It is a *sensor* reading the gradient energy through the temperature parameter and converting it into an exploration policy.

1. The Ergodic Weights

Each layer uses an effective weight matrix sampled from a locally specified ergodic distribution:

$$W_{\text{eff}} = \alpha \cdot W + (1 - \alpha) \cdot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I)$$

W stores the learned structure that shapes the distribution as α rises. Derived quantities:

$$\text{bias} = \alpha, \quad \text{var} = 1 - \alpha$$

- $\alpha = 0$: Pure exploration. $W_{\text{eff}} = \varepsilon$. Random noise.
- $\alpha = 1$: Pure exploitation. $W_{\text{eff}} = W$. Learned weights only.
- $0 < \alpha < 1$: Interpolation.

Training begins at $\alpha = 0$ and transitions autonomously toward exploitation as the loss landscape flattens.

2. Why Standard Adam Fails on α

The temperature $T = 1 - \alpha$ receives gradients via:

$$\frac{\partial \mathcal{L}}{\partial T} = \sum_{i,j} \frac{\partial \mathcal{L}}{\partial (W_{\text{eff}})_{ij}} \cdot \varepsilon_{ij}$$

Because ε is re-sampled every forward pass, this gradient is **zero-mean**:

$$\mathbb{E}\left[\frac{\partial \mathcal{L}}{\partial T}\right] = \sum_{i,j} \frac{\partial \mathcal{L}}{\partial (W_{\text{eff}})_{ij}} \cdot \mathbb{E}[\varepsilon_{ij}] = 0$$

Standard Adam's first moment kills the signal: $m_t \approx 0 \Rightarrow m_t / \sqrt{v_t} \approx 0$. Adam produces no update.

3. The Gradient Energy Sensor

3.1 Modified Second-Moment Estimator

Discard the first moment; use only the second moment as **output** rather than normalizer:

$$v_t = \beta \cdot v_{t-1} + (1 - \beta) \cdot g_t^2$$

where $g_t = \partial \mathcal{L} / \partial T$ and $\beta = 0.999$.

3.2 Bias Correction

$$\hat{v}_t = \frac{v_t}{1 - \beta^t}$$

3.3 What $\hat{v}_t$ Measures

The second moment of a zero-mean random variable is its variance:

$$\mathbb{E}[g_t^2] = \text{Var}(g_t) = \sum_{i,j} \left(\frac{\partial \mathcal{L}}{\partial (W_{\text{eff}})_{ij}} \right)^2 = \left\| \frac{\partial \mathcal{L}}{\partial W_{\text{eff}}} \right\|_F^2$$

The **squared Frobenius norm** of the loss gradient — the **gradient energy**.

Gradient energy	Meaning	Desired behavior
High $\hat{v}_t$	Loss sensitive to weight changes	Keep exploring (low α)
Low $\hat{v}_t$	Loss surface is flat	Exploit (high α)

3.4 The Alpha Update Rule

$$\alpha_t = \frac{1}{1 + \tau \cdot \sqrt{\hat{v}_t}}$$

where τ (`global_temp`) is a sensitivity knob.

Properties:

- $\sqrt{\hat{v}_t} \rightarrow 0 \Rightarrow \alpha_t \rightarrow 1$ (exploit)
- $\sqrt{\hat{v}_t} \rightarrow \infty \Rightarrow \alpha_t \rightarrow 0$ (explore)
- $\alpha_t \in (0, 1)$ always
- τ controls transition speed

After computing α_t : $\text{bias}_t = \alpha_t$, $\text{var}_t = 1 - \alpha_t$.

3.5 Layer-Local Certainty (Per-Neuron Sigma)

Each ergodic layer maintains per-neuron **sigma** — running variance of its gradient energy, tracked via Welford’s algorithm in `observe_sigma()`. Sigma drives per-neuron bias and var:

$$\text{var}_j = \frac{\sigma_j}{\sigma_j + \kappa}, \quad \text{bias}_j = 1 - \text{var}_j$$

Low sigma (consistent gradient, found a minimum) yields high bias, low var. High sigma (unstable gradient) yields low bias, high var.

`set_sigma(sigma)` controls κ :

- `sigma=1`: encourage exploration (low κ)
- `sigma=0`: suppress exploration (high κ , $\text{var} \approx 0$)

4. Comparison: Adam vs. Gradient Energy Sensor

Property	Adam (for W)	Sensor (for α)
Type	Optimizer	Sensor (measurement)
First moment m_t	Yes — tracks direction	No — direction is zero-mean
Second moment v_t	Yes — normalizes step size	Yes — is the output
Role of $\sqrt{v_t}$	Denominator (adaptive LR)	Numerator (energy estimate)
Update rule	$W \leftarrow W - \eta \cdot m / \sqrt{v}$	$\alpha \leftarrow 1 / (1 + \tau \sqrt{\hat{v}})$
Descent?	Yes	No — maps energy to policy
Zero-mean safe?	No — produces $0 / \sqrt{v} = 0$	Yes — by design

5. Derivation: Why Drop m_t

Let $g_t = \nabla_T \mathcal{L}$. Since noise ε is i.i.d. each step, $\mathbb{E}[g_t] = 0$. The first moment EMA $m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t$ gives $\mathbb{E}[m_t] = \beta_1 \mathbb{E}[m_{t-1}]$. By induction $\mathbb{E}[m_t] = \beta_1^t \mathbb{E}[m_0] = 0$ — exponentially to zero, carrying **no information**.

The second moment converges to a meaningful quantity:

$$\mathbb{E}[v_t] \rightarrow \mathbb{E}[g_t^2] = \|\nabla_{W_{\text{eff}}} \mathcal{L}\|_F^2$$

The gradient energy — exactly what we need.

6. Dropout via Bias

`bias` and `var` support dropout:

$$\text{bias}_t^{(\text{drop})} = \begin{cases} 0 & \text{with probability } p \\ \alpha_t & \text{with probability } 1 - p \end{cases}$$

When bias is dropped to zero, the layer acts as pure noise regardless of α . The temperature $T = 1 - \alpha$ remains unchanged so noise contribution is consistent.

Dropout is a schedule on bias: ordinary Bernoulli dropout is the binary case. Annealed dropout corresponds to increasing the expected bias over training. Nonzero temperature bakes regularization directly into the model — every forward pass perturbs the effective weights, discouraging brittle co-adaptation.

7. The `global_temp` Sensitivity Knob

τ (`global_temp`) scales responsiveness:

τ	Effect
$\tau = 0$	$\alpha = 1$ always — pure exploitation
$\tau \ll 1$	Quickly converges to $\alpha \approx 1$
$\tau = 1$	Balanced — α tracks gradient energy on natural scale
$\tau \gg 1$	Maintains high exploration even with moderate gradients

τ can be scheduled (e.g., annealed from high to low) for a coarse exploration-to-exploitation curriculum.

8. Initialization and Bootstrap

Problem. If initialized at $\alpha = 1$ (pure exploitation), the noise term vanishes and $\partial\mathcal{L}/\partial T = 0$. The sensor would read zero energy and keep $\alpha = 1$ forever — an **exploitation trap**.

Solution. Initialize at $\alpha = 0$.

At $\alpha = 0$:

- $W_{\text{eff}} = \varepsilon$ — pure noise carries gradients through the network
- Temperature gradient is nonzero: $g_t = \sum_{ij} (\partial\mathcal{L}/\partial\varepsilon_{ij}) \cdot \varepsilon_{ij} \neq 0$
- Sensor measures initial gradient energy and begins tuning α upward
- As W learns useful structure, energy decreases and α rises

Zero-initializing W is safe and often cleaner than random initialization. Symmetry is broken by the sampled noise; learned weights don't contribute at $\alpha = 0$ anyway. Starting from $W = 0$ makes the exploratory regime honest.

9. Layer Architecture

All ergodic layers follow the effective-weight pattern. When `ergodic=True`, noise buffers are registered and weights are zero-initialized.

ErgodicLayer Base Class

`ErgodicLayer` (`Model.py`) provides per-neuron `bias` and `var` buffers driven by gradient variance (`observe_sigma()` / `sigma_to_ergodic()`). Subclasses with child `ErgodicLayers` must forward `set_sigma()` and `paramUpdate()`.

InvertibleLinearLayer — factor-level noise injection

LDU factorisation $W = L \cdot D_{\text{embed}} \cdot U$ (see `Architecture.md`). Exact inverse via triangular solves; no SVD.

When `ergodic=True`, noise is injected at the factor level to preserve exact invertibility at all temperatures:

$$\begin{aligned}
L_{\text{eff}} &= I + \text{strict_lower}(\text{raw_L} + t \cdot \text{noise_L}) \\
U_{\text{eff}} &= I + \text{strict_upper}(\text{raw_U} + t \cdot \text{noise_U}) \\
d_{\text{eff}} &= b \cdot d_{\text{clamped}} + t \cdot \text{noise_d}
\end{aligned}$$

W_{eff} stays in LDU form, so its exact inverse is always available — no approximation regardless of noise level. Compare to matrix-level blending $W_{\text{eff}} = b \cdot W + t \cdot N$, which destroys LDU structure:

$$W_{\text{eff}} \cdot W_{\text{eff}}^{-1} \approx (b^2 + t^2)I + bt(WN^{-1} + NW^{-1}) \neq I$$

Error grows with temperature — exactly the regime where exploration is most active. Factor-level injection avoids this.

`noise_d` is sampled with magnitude in $[\text{eps}, 1]$ and random sign, so it's always invertible; combined with the `stable=True` clamp on the deterministic diagonal, `d_eff` stays bounded away from zero.

`SigmaLayer(invertible=True)` and `PiLayer(invertible=True)` use `InvertibleLinearLayer(ergodic=True)` internally. `bias` and `var` from the parent `ErgodicLayer` forward as `bias=` / `temp=` arguments, so the gradient-energy sensor governs noise injection in the inner linear primitive.

10. Training Loop Integration

```

optimizer = torch.optim.Adam(model.getParameters(), lr=lr)  # Adam for W
loss = model(x, y)                                         # forward W_eff = bias*W + var*noise
loss.backward()                                           # Adam: dL/dW; sensor: dL/dT
optimizer.step()
model.paramUpdate()                                       # sensor updates alpha

```

11. Summary of the Full Algorithm

Initialize: $\alpha_0 = 0$, $v_0 = 0$, $t = 0$, $\beta = 0.999$.

Each step:

1. Sample $\varepsilon \sim \mathcal{N}(0, I)$.
2. $W_{\text{eff}} = \alpha_t W + (1 - \alpha_t)\varepsilon$.
3. Forward: $\hat{y} = f(x; W_{\text{eff}})$; loss $\mathcal{L}(\hat{y}, y)$.
4. Backward: $\nabla_W \mathcal{L}$ and $g_t = \nabla_T \mathcal{L}$.
5. **Adam** updates W .
6. **Sensor** updates α :

$$v_t \leftarrow \beta v_{t-1} + (1 - \beta)g_t^2, \quad \hat{v}_t = \frac{v_t}{1 - \beta^t}, \quad \text{var}_j = \frac{\hat{v}_j}{\hat{v}_j + \kappa}, \quad \text{bias}_j = 1 - \text{var}_j$$

7. Zero temperature gradient: $g_t \leftarrow 0$.

Machine Minds

Alec Rogers, September 24, 2025

Abstract

Problem

Society depends on machine minds, but we lack a good theory of them. While human and machine minds functionally overlap, their design goals differ, and treating a machine mind as a human mind has significant consequences. We should have a different theory of mind for machine minds, informed by how those minds are designed and trained.

Background

Anthropomorphizing machines helps us understand them relative to human minds. This essay focuses on three questions: what are machine minds, what do machine minds feel, and what do machine minds know.

Solution

Three measures make a machine mind more humanistic: weight ergodicity, network invertibility, and output certainty. Weight ergodicity treats weights as samples from a random process rather than fixed constants. Network invertibility makes the architecture bidirectional and partially invertible. Output certainty means knowing that it does not know, rather than only the probability of one answer versus another. All models tested use a 20-neuron, single hidden layer on MNIST.

Invertibility matters for meaningful semantic embedding: an LLM's symbols should correspond to something we can identify, not be abstract tokens inside Searle's Chinese translation engine.

Background

Scope: Large Language Models (reinforcement learning with known inputs/outputs and unknown weights — the transformer architecture).

Part I: What Machine Minds Are — Weight Ergodicity

Weights as Ergodic Samples

Measurement theory views weights as **samples from an ergodic distribution** rather than fixed constants. W at any moment is one realization of a stochastic process:

$$W = \mu M + \sigma V$$

where μ and σ are the bias and variance parameters; M has norm 1 and zero variance (analogous to a typical weight matrix), and V is sampled from a standard normal with unit variance.

This perspective comes from **Lagrangian Field-theoretic Gradient Control** (LFGC), treating the weight landscape as a field where boundary conditions shape the distribution from which weights are drawn. The ergodic hypothesis guarantees that time averages (training steps) equal ensemble averages (weight configurations).

See Rogers, A. (2026), *Local Freedom under Global Constraint*, Zenodo, doi: 10.5281/zenodo.18644302.

Temperature and Annealing

A single per-layer *temperature* parameter, updated via ADAM, characterizes mean and variance:

$$\mu = 1 - temp, \quad \sigma = temp$$

Temperature begins at 1 and decreases to 0.

Advantages of Ergodic Weights

1. **No learning rate required.** Biases find different weight-space locations via randomness and converge as temperature decreases.
2. **Zero initialization.** Weights can be initialized to zero — annealing bounces the model out of local minima more effectively than gradient steps.
3. **Theoretical convergence guarantee.** Simulated-annealing literature provides a guarantee of convergence to global optimum (given infinite iterations).

Standard *dropout* can be interpreted as per-neuron variance in the bias parameter. The same regularization occurs naturally from time-varying weight biases.

Part II: What Machine Minds Feel — Network Invertibility

Bidirectional Perception

Egocentric viewpoint sees perception as passive; physics suggests it is bidirectional. Approximated via invertibility of weight multiplications and activation functions.

Symbols, Worldlines, and Karmic Inertia

The symbols an LLM processes have meanings given by **worldlines** — the accumulated history of contexts in which a token has appeared, the gradients that have shaped its embedding, and the network states it has participated in. The meaning of a symbol is the integral of its worldline.

Weight adaptation (or inference) puts a **force** into the system. That force adapts either the system’s inputs or outputs depending on which has less **karmic inertia** — resistance to change. The bidirectional architecture means the force propagates in both directions:

- **Forward (encoding):** input representation adapts to match learned categories — perception shaped by expectation.
- **Reverse (decoding):** internal state adapts to reconstruct the input — expectation grounded in perception.

The invertible architecture makes both directions explicit and trainable.

Implementation

Even when not invertible, bidirectionality plays a role in backpropagation and autoencoders. We examine simultaneously training in both forward (predict output from input) and reverse (predict input from output of the *penultimate* layer) directions — the network is composed of a bijective front (*recognizer*) and a surjective back (*generalizer*). See also Rogers et al. 2001 for related work on sharing weights between policy and critic.

Part III: What Machine Minds Know — Output Certainty

Certainty vs. Probability

A network’s error function corresponds to its desire. Cross-entropy loss *requires* output classification — expressing the probability of one class *versus another*, not the certainty that the input belongs to a given class.

To remedy: blend cross-entropy with a product of cross-entropy and the prediction norm. Zero output is penalized by cross-entropy; incorrect predictions carry additional penalty proportional to magnitude.

Certainty-Weighted Loss

$$\begin{aligned} error &= -\sum_{c=1}^N t_c \log(p_c) \\ certainty &= |\hat{y}| \\ surprise &= (\alpha) \cdot certainty \cdot error + (1 - \alpha) \cdot error \end{aligned}$$

For amplitude certainty:

$$error = (\hat{y} - y)^2$$

Knowing What You Do Not Know

A network should **know that it does not know**, not merely the probability of one answer vs. another. Cross-entropy forces probability distribution across all known classes, even when “I don’t know” is correct. Certainty weighting separates “which class?” from “how sure?”

Methods

Python + PyTorch. MNIST dataset: 60K train, 10K test, 28×28 monochrome at 256 bit depth. Inputs are preprocessed (mean and variance removed) and shuffled.

Input: 28×28 array. Output: one-hot target class. Architecture: 20 hidden unit NN. Batch size 10, 9 epochs, 7 trials for error bars.

Standard benchmark: ReLU, LR 0.01, ADAM, cross-entropy loss.

Base ergodic: zero-init weights, single per-layer temperature, ADAM updates temperature, dropout (0.75) on middle layer.

Five models evaluated: base + four ergodic variants (one plain ergodic, one with input normalization, two reversible — one with separate forward/backward layers, one invertible). The invertible model represents W in SVD form (rotation, eigenvalues, rotation) using Givens rotations and diagonal eigenvalues to avoid direct matrix inversion.

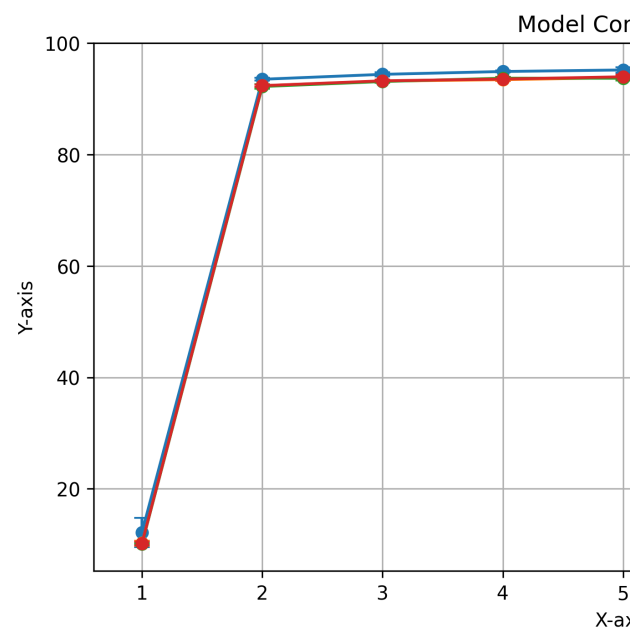
Weight update equations for ergodic models avoid a learning rate. Compared to traditional gradient descent ($\Delta_W = \alpha(y - \hat{y})\nabla_F x$):

$$\begin{aligned} e &= y - \hat{y} \\ c &= \alpha \hat{y}^2 + (1 - \alpha) \\ \Delta_W &= (ce - \alpha \hat{y} e^2)x \end{aligned}$$

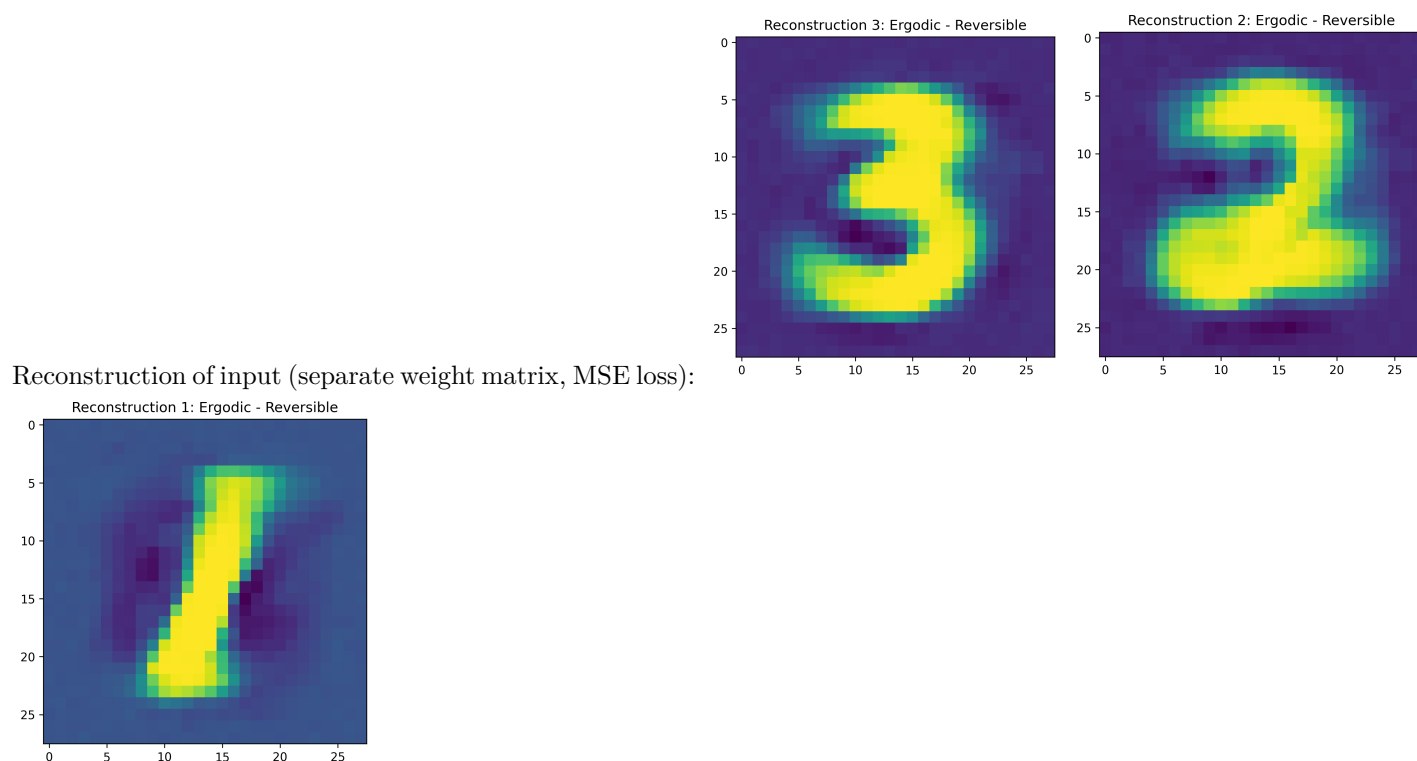
The network simultaneously minimizes error and “incorrect certainty”.

Results

Model comparison with error bars shows the simple model performs slightly better than all proposed



paradigms, with all proposed paradigms performing almost identically:



Discussion

Benefits:

- Random initialization unnecessary — only the simple model with random weight variation shows significant performance variation.

- Learning rate replaced by weight noise variance (dropout role replaced by bias variance).
- Error function encapsulates output certainty in addition to between-class certainty.

Limitations:

- Pearson correlation between certainty and accuracy is insignificant — the implementation used $\text{certainty} = p_c$ (softmax output) rather than $|\hat{y}|$, creating contradictory constraints since softmax outputs sum to 1 while the certainty norm imposes another constraint. Corrected code should use norm-weighted MSE.
- PyTorch autograd tunes weights directly, interfering with simultaneous temperature tuning in ways not captured.

Conclusion

Strong philosophical grounding for several paradigms, but implementation issues limited testing. Effectiveness in multilayer NNs and challenging tasks remains unclear.

References

Rogers, Shannon, Lendaris; 2001: *A comparison of DHP based antecedent parameter tuning strategies for fuzzy control*, IFSA/NAFIPS, IEEE.

Rogers, A; 2003: *Analysis of the Instantaneous Estimate of Autocorrelation*, unpublished.

Rogers, A; 2026: *Local Freedom under Global Constraint*, Zenodo, doi: 10.5281/zenodo.18644302.

XML Configuration Parameters

Model configuration is specified in XML files (e.g. `data/XOR_exact.xml`). Default values are loaded from `data/model.xml` and overridden by model-specific configs. The schema is in `data/model.xsd`.

Overlay merge order: `data/model.xml` $\rightarrow$ `<config>.xml` $\rightarrow$ runtime overrides.

Note on naming: the schema and the XML configs use `nInput` / `nOutput` (sequence-length sentinels), `nDim` (per-vector dim), `nVectors` (codebook size), and `nInputDim` / `nOutputDim` (raw shape hints for the LiftingLayer). Older docs sometimes call `nOutput` / `nInput` `nActive`; treat that as the same thing as `nOutput`.

2026-05-28: `<nWhere>` / `<nWhen>` are retired from per-file configs. `data/model.xml` carries per-Space defaults (`<nWhere>2</nWhere>` on InputSpace / PerceptualSpace / SymbolicSpace, 0 elsewhere). The `.where` field is now the canonical positional identifier (quadrature sinusoidal); see doc/plans/2026-05-28-where-keyed-taxonomy.md.

<architecture>

Core model settings. Training and data parameters are in nested sub-elements `<training>` and `<data>` (see below).

Parameter	Type	Default	Description
<code>modelType</code>	string	"simple"	simple (feedforward), <code>embedding</code> (LM / chat with sequence processing)
<code>reconstruct</code>	string	"concepts"	Reverse-pass mode: <code>none</code> disables; <code>symbols</code> reconstructs from cached
<code>conceptualOrder</code>	int	1	Iteration depth of the $P \rightarrow C \rightarrow S$ pipeline. <code>order > 1</code> partitions the sy
<code>processSymbols</code>	bool	false	Apply extra symbolic processing after Sigma.
<code>monotonic</code>	bool	false	Constrain invertible Sigma / Pi to $W \geq 0$ so the lift / lower chain is c
<code>ergodic</code>	bool	false	Ergodic exploration: every Layer uses $W_{\text{eff}} = \text{bias} * W + \text{temp} *$

Parameter	Type	Default	Description
<code>naive</code>	bool	<code>false</code>	Materialise <code>W_eff</code> densely in <code>InvertibleLinearLayer</code> . Slower; debug
<code>conceptualMode</code>	string	<code>"parallel"</code>	<code>parallel</code> (whole-slab [B, N, D] forward; required for the butterfly c
<code>routerWireSerial</code>	string	<code>"both"</code>	Per-word router-fire gating on the serial path: <code>per-word</code> (fire per wor
<code>embeddingPath</code>	string	(empty)	gensim <code>KeyedVectors</code> path. Empty disables embedding load.
<code>weightsPath</code>	string	(empty)	Model weights checkpoint path. Empty falls back to <code>output/<name></code> .
<code>maxResponseLength</code>	int	4096	Inference token budget (characters / bytes / tokens). Caps output alo
<code>amp</code>	string	<code>"off"</code>	AMP autocast mode: <code>off</code> , <code>fp16</code> , <code>bf16</code> .
<code>truthBiasScale</code>	decimal	0.1	Strength of luminosity-based bias on concept input during forward.
<code>LuminosityWeight</code>	decimal	0.1	Multiplicative loss penalty for incoherent truths.
<code>UniversalityWeight</code>	decimal	0.1	Multiplicative loss penalty for unkind propositions.
<code>allowExcludedMiddle</code>	int	1	Catuskoti policy: 1 permits NEITHER, -1 enforces classical LEM.
<code>allowContradiction</code>	int	0	Catuskoti policy: 0 forbids BOTH (non-contradiction), 1 permits par
<code>gateL1Lambda</code>	float	0.0	L1 sparsity penalty on lift / lower gates. 0 disables.
<code>loadBalanceWeight</code>	float	0.0	Sparse-MoE load-balance loss weight. Active only when <code>WordSpace.c</code>
<code>l1Lambda</code>	decimal	0.0	Architecture-wide L1 penalty hook. Most configs leave this at 0 and u
<code>discontinuityLambda</code>	decimal	0.0	Architecture-wide discontinuity penalty hook (legacy).
<code>conceptualWidth</code>	string	<code>"tapered"</code>	Symbol-partition geometry for <code>conceptualOrder > 1</code> . <code>tapered</code> = ge
<code>maxActivePerLayer</code>	int	8	Maximum active nodes per layer for reverse / mereological walks.
<code>codebookRetire</code>	bool	<code>false</code>	Retire codebook path where supported. Mostly a migration hook.
<code>symbolLearning.enabled</code>	bool	<code>false</code>	Enables runtime symbol-learning hooks. Disabled by default.
<code>mereologicalTreeSize</code>	int	–	Retired compatibility knob. Values are accepted but ignored; the cod
<code>writeSyntax</code>	bool	<code>false</code>	Dump POS-labelled syntax tree at end of every <code>MentalModel.forward</code>
<code>syntaxOutPath</code>	string	–	Output path for <code>writeSyntax</code> .

Luminosity and universality are **multiplicative** — they scale the total loss by

$$(1 + \text{LuminosityWeight} \cdot (1 - \text{luminosity}) + \text{UniversalityWeight} \cdot (1 - \text{universality})).$$

`TruthLoss` (in `<training>`, below) is **additive** — it directly penalises propositions that contradict the `TruthSet`. Both coexist. See `Ethics.md` Section `Universality` and `Reasoning.md` Section `TruthLoss` for details.

`<architecture><data>`

Data loading and filtering.

Parameter	Type	Default	Description
<code>dataset</code>	string	<code>"xor"</code>	Dataset key: <code>xor</code> , <code>mnist</code> , <code>tomatoes</code> , <code>text</code> , <code>inline</code> . Omit for inference-only mode
<code>numShards</code>	int	1	Shard count (streaming text datasets).
<code>maxDocs</code>	int	10000	Per-shard document cap.
<code>shardDir</code>	string	(empty)	Text shard directory (e.g. <code>data/fineweb</code>). Only used when <code>dataset=text</code> .
<code>minFrequency</code>	float	0.0	Vocabulary admission threshold (fraction of corpus). Words below it are held in a
<code>classificationMin</code>	float	–	Lower bound for classification-accuracy reporting.
<code>classificationMax</code>	float	–	Upper bound for classification-accuracy reporting.

Inline-mode configs additionally accept `<input use="train|test|validation">...|...|...</input>` and matching `<output>` elements directly inside `<data>`.

<architecture><training>

Training loop and I/O.

Parameter	Type	Default	Description
numTrials	int	1	Independent training runs per invocation.
numEpochs	int	3	Epochs per trial.
batchSize	int	64	Contiguous streams through the dataset. Each batch row b receives the next
numWorkers	int	0	DataLoader prefetch workers. 0 = synchronous in-process batch assembly.
learningRate	float	0.001	Adam learning rate.
reconstructionScale	float	0.5	Weight of reconstruction loss vs prediction loss in $[0, 1]$: $\mathcal{L}_{\text{total}} = (1 - r) \mathcal{L}_{\text{ou}}$
whatScale	float	0.7	Loss weight on the .what (content) channel.
whereScale	float	0.2	Loss weight on the .where (positional) channel.
whenScale	float	0.1	Loss weight on the .when (temporal) channel.
negSamples	int	64	Negative samples per positive for CBOW / SBOW. Memory cost: $O(\text{batch} \cdot$
TruthLoss	float	0.0	Additive penalty for propositions that contradict the TruthSet (union-norm
maskRate	float	0.15	Within-sentence IR Bernoulli mask rate. BERT default 0.15.
trainEmbedding	string	"NONE"	Embedding update mode: NONE, CBOW, SBOW, BACKPROP, BOTH, JOINT. See ta
embeddingScale	float	0.25	JOINT-mode embedding loss weight λ : $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{model}} + \lambda \cdot \mathcal{L}_{\text{emb}}$. Ignored
autoload	bool	true	Load weights from weightsPath at construction. Stale incompatible checkp
autosave	bool	false	Save weights after training completes.
checkpointEveryBatches	int	0	Save mid-training checkpoint every N optimizer steps. 0 disables periodic ch
profile	bool	false	cProfile the training loop.
certainty	bool	false	Per-neuron certainty tracking in ergodic layers. Allows individual neurons to
sentencePrediction	bool	false	Enables InterSentenceLayer , an ARMA(p, q) predictor over sentence repr
armaScale	float	0.1	Loss weight for the ARMA sentence-prediction MSE.
sentencePrimingScale	float	0.05	AR prediction cast into concept_dim and added as a bias to concept_inpu
intraLossWeight	float	0.1	Loss weight on the in-STM next-idea term $\mathcal{L}_{\text{intra}} = \text{MSE}(\hat{c}_t, c_t)$ from Intra
interLossWeight	float	0.1	Loss weight on the inter-sentence next-end-state term $\mathcal{L}_{\text{inter}} = \text{MSE}(\hat{p}, p)$ fro

<trainEmbedding> — Embedding Update Modes

Value	Embedding method	Model layers	Gradients through codebook
NONE	Frozen	Trained	No
CBOW	CBOW (padded context)	Trained	No
SBOW	SBOW (leave-one-out centroid)	Trained	No
BACKPROP	Backprop only	Trained	Yes
BOTH	SBOW post-batch	Trained	Yes (two optimizers)
JOINT	Single backward	Trained	Yes ($\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{model}} + \lambda \cdot \mathcal{L}_{\text{SBOW}}$)

<architecture><server>

HTTP server settings (used by **serve.py**).

Parameter	Type	Default	Description
host	string	"127.0.0.1"	Bind address.
port	int	8001	Port. Override per model to run several in parallel.
timeout	int	120	Per-request timeout (s).

BasicModel / MentalModel Quick Reference

The two configurations that train the full $P \rightarrow C \rightarrow S$ pipeline:

```
<architecture>
  <truthBiasScale>0.1</truthBiasScale>
  <LuminosityWeight>0.1</LuminosityWeight>
  <UniversalityWeight>0.1</UniversalityWeight>
  <conceptualOrder>1</conceptualOrder>
  <monotonic>false</monotonic>
</architecture>

<training>
  <TruthLoss>0.0</TruthLoss>
</training>
```

The grammar section of `MentalModel.xml` declares `lift` as an S-tier binary rule:

```
<S>lift(S, S)</S>      <!-- lift is binary; SVO is assembled by
                        the chart-compose path S -> S VO,
                        VO -> V O, not a ternary lift -->
```

Transitive SVO is produced compositionally via typed grammar rules such as $S \rightarrow S VO$ / $VO \rightarrow V O$, or via STM `ParseState` extraction for a subject `lift` over a transitive verb-phrase derivation. Grammar mode is derived from the loaded grammar.

Space sections

Every space block uses the same sentinel convention:

- `nInput` = 0 — `InputSpace`: derive from `TheData`; other spaces: derive from the previous space's `outputShape[0]`.
- `nOutput` = 0 — same as `nInput` for this space (passthrough count).
- `nVectors` = 0 — same as `nOutput` (used for non-codebook spaces).

Per-Space `<nWhere>` / `<nWhen>` declarations were retired on 2026-05-28 (see top-of-file note); the per-Space defaults live in `data/model.xml` and don't need re-declaration in per-experiment configs.

<InputSpace>

Lifts raw data into the model's internal representation.

Parameter	Type	Default	Description
<code>nOutput</code>	int	sentinel	Sequence length: max tokens per input. For XOR: 2. For text models, OOV
<code>nDim</code>	int	1	Per-vector content dim.
<code>nInputDim</code> / <code>nOutputDim</code>	int	0	Raw shape hints. 0 defers to the loader's native dim.
<code>nVectors</code>	int	sentinel	Codebook size. Defaults to <code>nOutput</code> .
<code>codebook</code>	mode	none	none, quantize, or project. Legacy true/false are accepted as quantize/
<code>lexer</code>	string	"word"	Tokenization mode: word, sentence, byte.

`inputShape` uses the data's native dim (e.g. 784 for MNIST, `inputLength` for text); `outputShape` uses `nDim` from `TheObjectEncoding`. `LiftingLayer` bridges the two.

<PerceptualSpace>

Transforms lifted input into perceptual features via Pi layers (multiplicative interactions).

Parameter	Type	Default	Description
nOutput	int	sentinel	Number of active perceptual feature vectors. Should be $\geq$ <code>InputSpace.nOutput</code> . For
nDim	int	1	Per-vector content dim.
nVectors	int	sentinel	Codebook size. When $>$ <code>nOutput</code> , enables vector quantization with top-k selection o
invertible	bool	false	true : one <code>PiLayer(invertible=True)</code> handles both directions; false : separate pi
hasAttention	bool	true	Attention reweighting of percepts before the Pi fold.
nonlinear	bool	true	Tanh-bound output to $[-1, 1]$.
codebook	mode	none	none , quantize , or project . Legacy true / false are accepted. Set to none to make
chunking	string	"lexicon"	Multi-token chunking strategy: lexicon , bpe , radix . radix routes the input lookup
butterfly	bool	false	FFT-style element-pair butterfly cascade on <code>self.pi</code> (and <code>self.sigma</code> historically)
wordLearning	int	0	Active lexicon-growth mode. 0 = frozen codebook (training default). $>=1$ = on first

Pi layer math: $y_j = b_j \prod_i (1 + W_{ji} x_i)$. See Architecture.md. `InvertiblePiLayer` has been merged into `PiLayer(invertible=True)`.

<ConceptualSpace>

Transforms perceptual features into abstract concepts via Sigma layers (additive / linear).

Parameter	Type	Default	Description
nOutput	int	sentinel	Active concept vectors. For XOR: 3.
nDim	int	1	Per-vector content dim.
nVectors	int	sentinel	Codebook size.
invertible	bool	false	true : <code>SigmaLayer(invertible=True)</code> (exact inversion via $\text{atanh} + W^{-1}$); false : sepa
hasAttention	bool	false	Attention in conceptual processing. Violates position locality required by serial_mode .
nonlinear	bool	true	Tanh-bound output to $[-1, 1]$.
codebook	mode	none	none , quantize , or project .
butterfly	bool	false	Butterfly cascade on CS's <code>PiLayer</code> (same machinery as <code>PerceptualSpace</code>). Required by M
stmCapacity	int	8	STM ring depth (within Miller's 7 ± 2 band; also the wMax fallback). Per-batch buffer [I

Sigma layer math: $y_j = \tanh(Wx + b)$. See Architecture.md. `InvertibleSigmaLayer` has been merged into `SigmaLayer(invertible=True)`.

2026-05-29 (clean-stack STM): `ConceptualSpace.forward` no longer applies `sigma_in` / `sigma_cs` on the forward path. The Stage-10 additive composition `sigma_in(combined) + sigma_cs(prev)` is replaced with per-stage tier attribution (**folded = primary** at stage 0, **folded = sym** at $k > 0$). The sigma layers remain allocated and are still invoked on the reverse path, but their parameters get no gradient on forward — they're dead weight under the current experiment. See doc/plans/2026-05-29-clean-stack-stm-basis-arg-radixlayer.md.

<SymbolicSpace>

Discrete symbolic representation — the information bottleneck between perception and output. See Language.md for the design.

Parameter	Type	Default	Description
nOutput	int	sentinel	Total symbols. When reconstruction symbols are enabled, <code>nOutput</code>
nDim	int	1	Per-vector content dim.

Parameter	Type	Default	Description
<code>nVectors</code>	int	sentinel	Codebook size. When <code>codebook=quantize</code> , equals the number of
<code>codebook</code>	mode	<code>none</code>	<code>none</code> , <code>quantize</code> , or <code>project</code> . Required for the full symbolic pipeline.
<code>nonlinear</code>	bool	<code>true</code>	Tanh-bound the activation.
<code>sortNetwork</code>	bool	<code>false</code>	Bitonic-sort regularizer on the codebook ordering.
<code>sortPasses</code>	int	0	Sort-network depth. 0 disables.
<code>l1Lambda</code>	decimal	0.01	L1 sparsity penalty on the symbolic activation.
<code>symbolResidualScale</code>	decimal	1.0	Symbol-residual injection scale (legacy mereology hook).
<code>outputSymbolResidualScale</code>	decimal	0.0	Output-side symbol-residual injection scale.
<code>commitmentBeta</code>	float	0.25	VQ-VAE commitment-loss β . Active whenever <code><codebook>quantize</code> .
<code>decorrelationWeight</code>	decimal	0.0	<code>ImpenetrableLayer</code> decorrelation regularizer on codebook rows.
<code>spectralFlatnessWeight</code>	decimal	0.0	<code>ImpenetrableLayer</code> spectral-flatness regularizer.
<code>truthMinMagnitude</code>	decimal	0.3	Gold- <code><truth></code> recording gate (repurposed; no longer a per-activation gate).
<code>truthCriterion</code>	unitInterval	0.3	Learn-score acceptance gate for learned relative-sentence relations.

`SymbolicSpace` owns one `SigmaLayer(nConcepts, nSymbols, invertible=True, monotonic=monotonic)` at `self.sigma` bridging the $C \leftrightarrow S$ boundary in both directions via its own self-inverse (`forwardSigma` / `reverseSigma` pointer aliases hide the one-or-two-layer split). The legacy `SymbolicSpace.layer PiLayer` attribute is gone; consumers use `model.symbolicSpace.sigma` directly.

<OutputSpace>

Maps symbols to final predictions via linear layers.

Parameter	Type	Default	Description
<code>nOutput</code>	int	sentinel	Output values. For XOR: 1.
<code>nDim</code>	int	1	Per-output dim.
<code>nVectors</code>	int	sentinel	Codebook size. Defaults to <code>nOutput</code> .
<code>invertible</code>	bool	<code>false</code>	Rarely set on <code>OutputSpace</code> .
<code>codebook</code>	mode	<code>none</code>	<code>none</code> , <code>quantize</code> , or <code>project</code> .
<code>nonlinear</code>	bool	<code>false</code>	<code>OutputSpace</code> is linear by default; rescales via <code>Data.denormalize</code> .

`LinearLayer` with (`bias`, `temp`) support for ergodic mode.

<WordSpace>

Grammar infrastructure (`SyntacticLayers`, `TruthLayer`). Lives outside `<architecture>` as a top-level sibling.

Parameter	Type	Default	Description
<code>syntacticHiddenDim</code>	int	64	<code>SyntacticLayer</code> MLP hidden size.
<code>truthMaxEntries</code>	int	1024	<code>TruthLayer</code> SS-codebook capacity.
<code>ltmCapacity</code>	int	1024	Long-term-memory (LTM) chain capacity on <code>InterSentenceLayer</code> : the p
<code>parserBackend</code>	string	"chart"	Parser backend: <code>chart</code> , <code>stm</code> , or <code>parallel</code> . STM requires an attached kno
<code>routerKind</code>	string	"chart"	Chart-backend router: <code>chart</code> or <code>signal</code> .
<code>chartCollapse</code>	string	"root"	Soft-superposition chart output: <code>root</code> (single root sentence node, other po
<code>chartTau</code>	float	1.0	Temperature for the chart rule-score softmax.
<code>wMax</code>	int	8	Max span width considered by the parser chart path and default STM cap
<code>chartTopK</code>	int	0	Sparse-MoE rule gating: keep top-K rules per (cell, split). 0 disables.
<code>chartNoiseEps</code>	float	0.0	Gaussian noise epsilon for <code>chartTopK</code> gating.

Parameter	Type	Default	Description
iterationsPerWord	int	1	P->C iterations per word in the per-word stem path.
downwardGeneration	bool	false	Emit a codebook head via <code>WordSpace.reconstruct()</code> and stash on <code>self</code>
language.interpretation	float	0.5	Soft-interpolation weight between forward and generation directions.

The `<language><grammar>` block contains tier-scoped rules in `<compose>` and `<generate>` sections, each holding `<percepts>` / `<concepts>` / `<symbols>` sub-blocks of `<rule>` elements. Per the 2026-05-07 rollback, the grammar XML is the sole source of truth for which rule fires at each tier (no code-level `default_rule` fallback). Legacy bare-tier elements like `<P>P = sigma(P)</P>` directly under `<grammar>` are still accepted.

InvertibleLinearLayer (programmatic)

The LDU-factorised invertible linear primitive used internally by `SigmaLayer(invertible=True)` and `PiLayer(invertible=True)`. Not set via XML directly.

Parameter	Type	Default	Description
naive	bool	false	false : apply L, D, U sequentially via triangular solves (no W materialisation). true : materialise W
stable	bool	false	Clamps each diagonal entry d_i to magnitude $[\epsilon, 1]$ with sign preserved before the ergodic blend
ergodic	bool	false	Factor-level noise injection. Registers <code>noise_raw_L</code> , <code>noise_raw_U</code> , <code>noise_d</code> buffers; resamples

Class renames and removals.

Old name	New name / status
LULayer	Renamed to <code>InvertibleLinearLayer</code>
InvertibleLinearLayer (SVD-based)	Removed; replaced by LDU factorisation
InvertibleSigmaLayer	Merged into <code>SigmaLayer(invertible=True)</code>
InvertiblePiLayer	Merged into <code>PiLayer(invertible=True)</code>

See `Architecture.md` for the LDU factorisation and factor-level noise injection.

Example: XOR Configuration

```

<model>
  <architecture>
    <modelType>embedding</modelType>

    <training>
      <numEpochs>400</numEpochs>
      <learningRate>0.01</learningRate>
      <autoload>false</autoload>
      <trainEmbedding>JOINT</trainEmbedding>
      <embeddingScale>0.05</embeddingScale>
    </training>
  </architecture>

  <InputSpace>
    <nVectors>8</nVectors>

```

```

    <nDim>10</nDim>
    <nOutput>8</nOutput>
    <nOutputDim>10</nOutputDim>
</InputSpace>

<PerceptualSpace>
  <nInput>8</nInput>
  <nVectors>1000</nVectors>
  <nDim>10</nDim>
  <nOutput>8</nOutput>
  <hasAttention>false</hasAttention>
  <invertible>true</invertible>
  <codebook>quantize</codebook>
</PerceptualSpace>

<ConceptualSpace>
  <nInput>8</nInput>
  <nVectors>8</nVectors>
  <nDim>10</nDim>
  <nOutput>8</nOutput>
  <invertible>true</invertible>
  <codebook>quantize</codebook>
</ConceptualSpace>

<SymbolicSpace>
  <nInput>8</nInput>
  <nVectors>8</nVectors>
  <nDim>2</nDim>
  <nOutput>8</nOutput>
  <codebook>quantize</codebook>
</SymbolicSpace>

<OutputSpace>
  <nOutput>1</nOutput>
  <nVectors>1</nVectors>
</OutputSpace>
</model>

```

Everything not listed inherits from `data/model.xml`.

Example: Embedding / Chatbot Configuration

```

<model>
  <architecture>
    <modelType>embedding</modelType>
    <weightsPath>BasicModel.ckpt</weightsPath>
    <embeddingPath>BasicModel.kv</embeddingPath>

    <data>
      <shardDir>data/fineweb</shardDir>
      <minFrequency>0.00001</minFrequency>
    </data>
  </architecture>
</model>

```

```

<training>
  <numEpochs>2</numEpochs>
  <batchSize>128</batchSize>
  <reconstructionScale>0.1</reconstructionScale>
  <trainEmbedding>BACKPROP</trainEmbedding>
  <autosave>true</autosave>
  <checkpointEveryBatches>500</checkpointEveryBatches>
  <sentencePrediction>true</sentencePrediction>
  <armaScale>0.1</armaScale>
</training>
</architecture>

<!-- space definitions inherit from model.xml -->
</model>

```

Key points:

- `modelType=embedding` activates the embedding-based input pipeline alongside the neural model.
- `trainEmbedding=BACKPROP` trains model layers with gradients flowing through the codebook.
- XML config defines the architecture. `weightsPath` stores the neural model checkpoint.
- `minFrequency` gates vocabulary admission; words below it are buffered until they exceed the threshold.